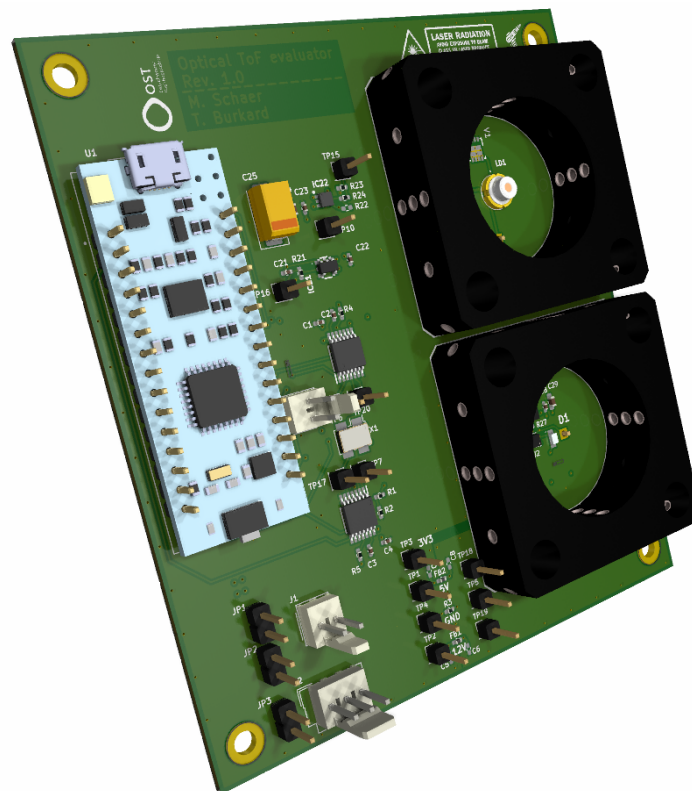


DIY Optical ToF Distance Measurement

CAS Sensors and Sensor Signal Conditioning

Matthias Schär, Timon Burkard
OST – Ostschweizer Fachhochschule

Translated by DeepL. Original german version: https://github.com/MatiMcFly/DIY_ToF/tree/main/docs/documentation/documentation.pdf



CAS Sensors and Sensor Signal Conditioning at the OST – Ostschweizer Fachhochschule

Title	DIY Optical ToF Distance Measurement
Student	Matthias Schär, Timon Burkard
Degree	CAS Sensors and Sensor Signal Conditioning
Semester	HS24
Lecturer	Prof. Guido Keel, Michael Lehmann

Abstract

This project work deals with the development of a Do It Yourself (DIY) system for optical Time of Flight (ToF) distance measurement.

A Time to Digital Converter (TDC) is used to precisely measure the signal propagation time. For test purposes, a second TDC is used, which is operated purely electrically to determine the length of different copper cables.

The work includes the development of the hardware, including laser drivers, photodetectors and transimpedance amplifiers, as well as the implementation of the firmware for signal evaluation. Simulations and measurements were carried out to optimize accuracy and minimize interference.

The electrical measurements show that the TDC achieves a temporal resolution in the range of several tens of picoseconds. This allows cable lengths to be determined to within a few centimeters.

The optical measurements show that a rough distance measurement up to about 30 cm is possible when a mirror is used as a target is used. In order to measure the distance to a wall, further improvements to the demonstrator are necessary.

Rapperswil, 19. März 2025

© Matthias Schär, Timon Burkard, OST – Ostschweizer Fachhochschule

Table of Contents

1	Introduction	11
1.1	Topic	11
1.2	Objective	11
1.3	Structure	11
2	Theory	13
2.1	Time of Flight	13
2.2	Photocurrent	14
2.2.1	Calculation with RLD94PZJ5 and BPV23NF	14
2.2.2	Calculation with RLD65NZX1 and NJL6401R-3	15
2.2.3	Calculation with receiving lens	15
2.2.4	Expected photocurrent	16
2.3	Transimpedance amplifier	18
3	Concept	19
3.1	Signal chain	19
3.2	Time measurement	20
3.3	Approach	21
4	Simulations	22
4.1	Laser Driver	22
4.2	Transimpedance amplifier	23
5	Realization	26
5.1	Circuits	26
5.1.1	Selective Input Voltage	26
5.1.2	Nucleo Board	27
5.1.3	TDC Electrical Signal	27
5.1.4	TDC Optical Signal	28
5.1.5	Oscillator For TDCs	29
5.1.6	Power Supply Separation	30
5.1.7	Laser Driver	31
5.1.8	Photo Receiver	31
5.1.9	Decoupling Capacitors	32
5.2	Layout	33
5.3	3D View	34
5.4	Fabrication	34
5.5	Firmware	35
6	Results	38
6.1	Electrical Measurements	38
6.1.1	GPIO Toggle with HAL	38
6.1.2	GPIO Toggle without HAL	41
6.1.3	Different cable lengths	44
6.1.4	Mode 1 vs. Mode 2	46
6.1.5	Timer Output	48
6.1.6	Different Clock Configurations	50
6.1.7	Different cable lengths with DSO	52

6.2	Optical Measurements	57
6.2.1	Signal Path Transmitter	57
6.2.2	ToF measurements via mirror	59
6.2.3	ToF measurements via wall	65
7	Conclusion	66
7.1	Outlook	66
7.2	Lessons Learned	67
7.3	Acknowledgements	67
8	Appendix	68
8.1	Outline	68
8.2	TDC Driver	69
8.3	Python Scripts	77
8.4	Schema	82
8.5	List of Components	85
8.6	Layout	86
8.7	Component Placement	90
8.8	3D View	92
8.9	Photos of the demonstrator	93

List of Acronyms

ADC Analog to Digital Converter. 20

AMS Analog/Mixed Signal. 22

CPU Central Processing Unit. 38, 39, 41, 42, 47, 48

DIY Do It Yourself. 2

DSO Digital Storage Oscilloscope. 6, 7, 9, 38, 40, 41, 43, 44, 52, 53, 55–57, 60–62

FW Firmware. 35

GPIO General Purpose Input/Output. 6, 8, 10, 28, 35, 38–45, 47–50, 52, 57

HAL Hardware Abstraction Layer. 6, 8, 10, 38–44, 47

HSI High Speed Internal. 6, 49–51

IC Integrated Circuit. 20, 27, 36

MCU Microcontroller Unit. 20, 21, 27–29, 36, 38, 41, 47, 48, 57, 60, 65, 66

PCB Printed Circuit Board. 25, 33, 34, 59, 66, 67

PLL Phase-Locked Loop. 6, 50–52

SPI Serial Peripheral Interface. 27, 28, 36

TDC Time to Digital Converter. 2, 6, 9–11, 21, 27–31, 36, 38, 40, 42, 43, 46–49, 52–55, 57, 61–63, 66, 69, 73, 77

TIA Trans-Impedance Amplifier. 6–8, 18, 23–25, 30, 31, 33, 64–67

ToF Time of Flight. 2, 6, 10, 11, 13, 19, 20, 23, 26, 34, 35, 66, 67, 78–80

UART Universal Asynchronous Receiver/Transmitter. 39, 42

USB Universal Serial Bus. 27

List of Figures

1	Visualization of a: direct ToF and b: indirect ToF	13
2	Lens from QIOPTIQ	16
3	Operational amplifier circuit as a TIA	18
4	Block diagram	19
5	Conceptual signal path	20
6	Concept operation TDC purely electric	21
7	Simulation Laser Driver - Schematic	22
8	Simulation Laser-Driver - Plot	23
9	Simulation TIA - Schema	23
10	Simulation TIA - Plot	24
11	Simulation TIA - Plot without C_F	25
12	Selective Input Voltage	26
13	Nucleo Board	27
14	TDC Electrical Signal	28
15	TDC Optical Signal	29
16	Oscillator for TDCs	30
17	Power Supply Separation	30
18	Laser Driver	31
19	Photo Receiver	32
20	Decoupling Capacitors	32
21	Inner Layer 1 with notes on the supply concept	33
22	3D View Top	34
23	Demonstrator from the front without lens	35
24	Structure of the firmware project	35
25	Pin configuration of the STM32F042K6	37
26	GPIO Toggle with HAL - Histogram	39
27	GPIO Toggle with HAL - Boxplot	40
28	GPIO Toggle with HAL - DSO	40
29	GPIO Toggle with HAL - DSO (Zoom)	41
30	GPIO Toggle without HAL - Histogram	42
31	GPIO Toggle without HAL - Boxplot	43
32	GPIO Toggle without HAL - DSO	43
33	GPIO Toggle without HAL - DSO (Zoom)	44
34	Different cable lengths	45
35	TDC Mode 1	46
36	TDC Mode 2	47
37	Default clock configuration	49
38	Clock configuration HSI with PLL 16 MHz	51
39	Clock configuration HSI48 48 MHz	51
40	PLL-Jitter	52
41	Different cable lengths - measurement values TDC	53
42	DSO recording with a cable length of 1 m	55
43	DSO recording with a cable length of 6 m	56
44	Measurement of the signal path sender: <code>start_op</code> , <code>LD_pulse</code> , FET gate	57
45	Measurement signal path sender: Current through laser diode	58
46	Measurement setup with mirror	59
47	Laser beam pattern	59

48	Laser alignment	60
49	Laser alignment adjusted	60
50	Initial mirror measurement with DSO	61
51	Different distances to the mirror - plot	62
52	Different distances to the mirror - plot with linear interpolation	63
53	TIA output and comparator threshold at 12 cm	64
54	TIA output and comparator switching threshold at 30 cm	65
55	Outline	68
56	Schema S.1/3	82
57	Schema S.2/3	83
58	Schema S.3/3	84
59	PCB Layout Top	86
60	PCB Layout Inner 1	87
61	PCB Layout Inner 2	88
62	PCB Layout Bottom	89
63	PCB component placement top	90
64	PCB component placement bottom	91
65	3D View Top	92
66	3D View Bottom	92
67	Demonstrator from above	93
68	Demonstrator from above without lens	93
69	Demonstrator from the front	94
70	Demonstrator from the front without lens	94
71	Demonstrator from below	95

List of Equations

1	Incoming light power	14
2	Radiation intensity	14
3	solid angle	14
4	photocurrent	14
5	Values of the RLD94PZJ5	14
6	Values of BPV23NF	14
7	Numerical results with RLD94PZJ5 and BPV23NF	14
8	Values of the RLD65NZX1	15
9	Values of NJL6401R-3	15
10	Numerical results with RLD65NZX1 and NJL6401R-3	15
11	Magnification of the receiving area by lens	16
12	Increased transmission power of the laser diode	16
13	Photocurrent scenario 1	17
14	Laser diode radiation model	17
15	Photocurrent scenario 2	17
16	Übertragungsfunktion TIA	18
17	Maximum spatial resolution of the TDC7200	20
18	TIA time constant	24
19	Laser diode series resistor	31
20	GPIO Toggle with HAL	39
21	GPIO Toggle without HAL	42
22	Back-calculate to cable length	45
23	Mode 1 vs. Mode 2	48
24	GPIO-switching delay time	49
25	GPIO Toggle with timer output	50
26	GPIO-switching delay times	52
27	Calculation of current through laser diode	58
28	Calculation of current through laser diode (actual)	58
29	Theoretically expected change in runtime	62

List of Tables

1	Different cable lengths	45
2	Calculated cable lengths	46
3	Mode 1 vs. Mode 2	48
4	Different clock sources	51
5	Different cable lengths - results TDC	53
6	Different cable lengths - results DSO	53
7	Different cable lengths - theoretical expectation	54
8	Different cable lengths - TDCns difference compared to theory	54
9	Different cable lengths - TDC [m] according to theory	54
10	Different cable lengths - TDC [m] according to the 0.9 c theory	55
11	Different distances to the mirror - statistical quantities	62
12	Bill of Material	85

List of Codes

1	TDC Init and Enable with Driver	36
2	TDC Read/Write with Driver	36
3	TDC Measurement with driver	36
4	GPIO Toggle with HAL	38
5	GPIO Toggle with HAL	39
6	GPIO Toggle without HAL	41
7	GPIO Toggle ohne HAL	42
8	Mode 1	47
9	GPIO Toggle with timer output	49
10	TDC Driver (Header)	69
11	TDC Driver (Source)	73
12	Python Analysis	77
13	Python Analysis (Multi ToFs)	78
14	Python Analysis (Multi Logs)	79
15	Python Live Plot	80

1 Introduction

This chapter provides an introduction to the topic of the present project work.

This work is being written by Timon Burkard and Matthias Schär. The supervising lecturers are Prof. Guido Keel and Michael Lehman.

1.1 Topic

The work deals with the development of demonstrator electronics for the visualization of optical Time of Flight (ToF) for distance measurement.

A measurement chip, the TDC7200 from Texas Instruments, is put into operation. This is done in two steps: first, purely electrical measurements are to be made possible, for example to determine the length of connected cables. Furthermore, the optoelectronics needed for optical distance measurement are to be located on the demonstrator.

1.2 Objective

The students have set themselves the goal of developing electronics that can be used to carry out the experiments. The control of the measuring equipment and the design of the optoelectronics should also be largely carried out independently.

The aim is to create a demonstrator that can be used with the TDC7200 to determine different cable lengths of determine the length of copper cables. After this goal has been achieved, it should also be possible to carry out optical measurements with the same demonstrator. The students have set themselves the goal of ensuring that the measurements with a wall as a target work up to a distance of 10 meters. Furthermore, a spatial resolution of about 30 centimeters should be achieved.

1.3 Structure

The following is a brief overview of the chapters in the rest of this paper:

- **Chapter 2 - Theory:** Deals with the theoretical foundations that were developed for the demonstrator.
- **Chapter 3 - Concept:** Explains the rough concepts and challenges that need to be overcome. In particular, the electronics to be developed and the time requirements for the TDC are recorded.
- **Chapter 4 - Simulations:** Contains information about the simulations that are carried out before the design but also during the commissioning of the demonstrator.
- **Chapter 5 - Realization:** Explains the realization. Circuits and the manufacturing process are presented here. It also explains what software development is necessary to realize such a demonstrator.
- **Chapter 6 - Results:** Documents all measurements carried out and their results. It shows to what extent the initial objective has been achieved.

- **Chapter 7 - Conclusion:** Captures the students' final thoughts and summarizes the work. It also provides an outlook on possible extensions and improvements.

2 Theory

The following subchapters develop the theoretical foundations necessary for understanding and designing a demonstrator for the ToF method.

2.1 Time of Flight

The measurement method known as "Time of Flight" involves measuring the transit time of a signal. The signal can be either optical or acoustic. One application of this is, for example, distance measurement.

In the optical principle discussed here, a transmitter emits light that is detected by a receiver. A time measurement can be made between the sending and receiving of the signal. Because light propagates at a defined speed, this time measurement can be used to determine the distance the signal has traveled within the measured time.

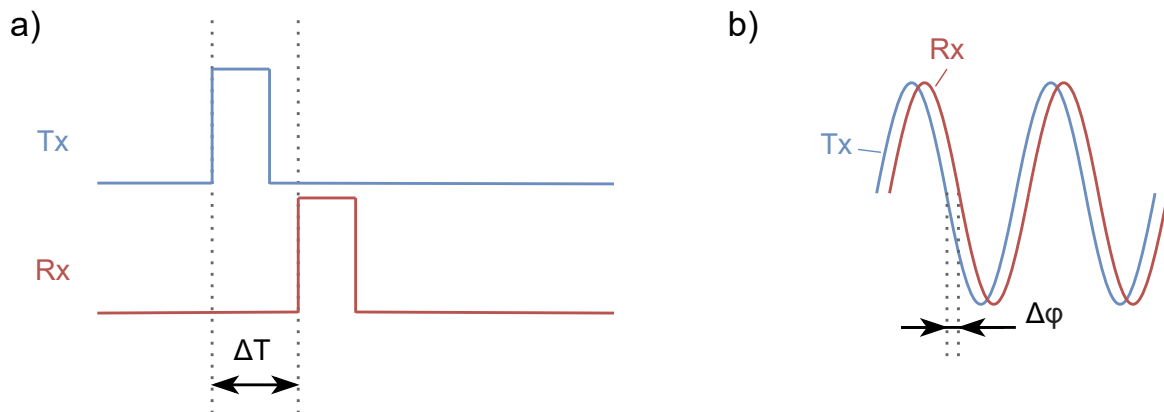


Abbildung 1: Visualization of a: direct ToF and b: indirect ToF

There are two types of ToF: direct ToF (dToF) and indirect ToF (iToF). With dToF only the time between the transmission and reception pulse is determined. In the figure 1, such pulses are shown at a).

In iToF, the phase shift of a modulated light signal is also measured (in some cases additionally) to the runtime measurement. This is shown in figure 1 in part b). This method allows higher resolutions to be achieved, but is much more complex in terms of evaluation and signal processing.

This work is limited to the direct ToF method.

In the following, the theoretical principles, especially with regard to designing a suitable optical system, are discussed.

2.2 Photocurrent

To calculate the theoretically expected photocurrent, a distance of 10 m from the wall is assumed.

In a simplified consideration, a laser beam with a divergence of 0° hits a wall at right angles and is scattered uniformly in a hemisphere. In reality, the laser will not hit the wall at 0° and the scattering will not be uniform, but rather more concentrated in the center.

The calculation of the received radiant power, the radiant intensity, the solid angle of a hemisphere, and the photocurrent are shown in formulas 1, 2, 3 and 4, respectively.

$$P_{in} = E_e \cdot A = \frac{I_e}{r^2} \cdot A \quad (1)$$

$$I_e = \frac{P_{out}}{\Omega} \quad (2)$$

$$\Omega = 4 \cdot \pi \cdot 0.5 \quad (3)$$

$$I_{ph} = S \cdot P_{in} \quad (4)$$

2.2.1 Calculation with RLD94PZJ5 and BPV23NF

Initial calculations are carried out with the RLD94PZJ5 laser diode (ROHM, 2020) and the BPV23NF photodiode (Vishay Semic., 2024).

The relevant values from the data sheets are listed in formula 5 and 6.

$$P_{out} = 285 \text{ mW} \quad (5)$$

$$\begin{aligned} A &= 4.4 \text{ mm}^2 \\ S &= 0.6 \frac{\text{A}}{\text{W}} \end{aligned} \quad (6)$$

These values inserted in formula 2, 1 and 4 yield the results in formula 7.

$$\begin{aligned} I_e &= \frac{P_{out}}{\Omega} = \frac{285 \text{ mW}}{4 \cdot \pi \cdot 0.5 \text{ sr}} = 45.4 \frac{\text{mW}}{\text{sr}} \\ P_{in} &= \frac{I_e}{r^2} \cdot A = \frac{45.4 \frac{\text{mW}}{\text{sr}}}{(10 \text{ m})^2} \cdot 4.4 \text{ mm}^2 = 2 \text{ nW} \\ I_{ph} &= S \cdot P_{in} = 0.6 \frac{\text{A}}{\text{W}} \cdot 2 \text{ nW} = 1.2 \text{ nA} \end{aligned} \quad (7)$$

2.2.2 Calculation with RLD65NZX1 and NJL6401R-3

The laser diode RLD94PZJ5 has two disadvantages with regard to this project work: on the one hand, it has such a high power that it can be harmful to the human eye. On the other hand, it emits in a wavelength range that is not visible to the human eye.

For these reasons, a second laser diode is being evaluated: RLD65NZX1 (ROHM, 2019). It is paired with the photodiode NJL6401R-3 (JRC, 2014). The following calculations are based on these two components.

The relevant values from the data sheets are listed in formula 8 and 9.

$$P_{out} = 10 \text{ mW} \quad (8)$$

$$\begin{aligned} A &= 0.7 \text{ mm} \cdot 0.7 \text{ mm} = 0.49 \text{ mm}^2 \\ S &= 0.42 \frac{A}{W} \end{aligned} \quad (9)$$

Inserted into the formulas 2, 1 and 4 the new results in formula 10.

$$\begin{aligned} I_e &= \frac{P_{out}}{\Omega} = \frac{10 \text{ mW}}{4 \cdot \pi \cdot 0.5 \text{ sr}} = 1.59 \frac{\text{mW}}{\text{sr}} \\ P_{in} &= \frac{I_e}{r^2} \cdot A = \frac{1.59 \frac{\text{mW}}{\text{sr}}}{(10 \text{ m})^2} \cdot 0.49 \text{ mm}^2 = 7.8 \text{ pW} \\ I_{ph} &= S \cdot P_{in} = 0.42 \frac{A}{W} \cdot 7.8 \text{ pW} = 3.28 \text{ pA} \end{aligned} \quad (10)$$

2.2.3 Calculation with receiving lens

At the suggestion of the lecturers, various optics from a modular system from QIOPTIQ are being tried out. A lens with a diameter of 17 mm and a focal length of 40 mm appears to be particularly promising. The lens is shown in Figure 2.



Abbildung 2: Lens from QIOPTIQ

This lens enlarges the effective area on which the photocurrent can be received. This results in higher reception power, i.e. a higher photocurrent. The formula 11 shows the increase in area to be expected when using such an optical system.

$$A' = \frac{A_L}{A_{PD}} = \frac{\left(\frac{17 \text{ mm}}{2}\right)^2 \cdot \pi}{0.49 \text{ mm}^2} = 463.2 \quad (11)$$

2.2.4 Expected photocurrent

Two scenarios are considered below: In the first, it is assumed, as before, that the light from the laser propagates in a focused beam and then reflects off the target in a hemispherical and diffuse manner.

In the second scenario, an attempt is made to model the radiation pattern of the laser diode. According to the data sheet, the diode has two different beam angles in the beam plane, which can be approximated with the solid angle of a rectangular pyramid. Scattering is prevented because the beam is reflected by a mirror.

In both scenarios, the radiated power of the laser diode is higher than in the first calculation. In pulsed operation, this can easily be increased above the maximum rating. Thermally, this does not pose a problem for the diode because the duty cycle is very small. The formula 12 shows the new expected radiant power of such a pulse. The efficiency of the laser diode is taken from the RLD65NZX1 data sheet (ROHM, 2019).

$$\begin{aligned} \eta &= \frac{P_{25 \text{ mA}}}{I_{ld}} = \frac{7 \text{ mW}}{25 \text{ mA}} = 0.28 \frac{\text{W}}{\text{A}} \\ I'_{ld} &= \frac{5 \text{ V} - V_{LD}}{R_v} = \frac{5 \text{ V} - 2.8 \text{ V}}{5 \Omega} = 440 \text{ mA} \\ P'_{out} &= \eta \cdot I'_{ld} = 0.28 \frac{\text{W}}{\text{A}} \cdot 440 \text{ mA} = 123 \text{ mW} \end{aligned} \quad (12)$$

For a target located at a distance of 2 meters, the following photocurrent can be expected according to formula 13:

$$P_{in} = \frac{P_{out}}{\Omega \cdot r^2} \cdot A_L = \frac{123 \text{ mW}}{4\pi \cdot 0.5 \text{ sr} \cdot (2 \text{ m})^2} \cdot 227 \text{ mm}^2 = 1.11 \text{ }\mu\text{W}$$

$$I_{ph} = S \cdot P_{in} = 0.42 \frac{\text{A}}{\text{W}} \cdot 1.11 \text{ }\mu\text{W} = 467 \text{ nA}$$
(13)

Without optics, the laser diode does not emit a collimated beam. In the second scenario, this radiation pattern is modeled using the solid angle of a pyramid, as defined in formula 14. The typical radiation angles θ are also taken from the laser diode data sheet. It should be noted that the full angle is given in the data sheet. The value in the formula is therefore halved.

$$\Omega_P = 4 \cdot \arcsin(\sin(\theta_x) \cdot \sin(\theta_y)) = 4 \cdot \arcsin\left(\sin\left(\frac{9^\circ}{2}\right) \cdot \sin\left(\frac{29^\circ}{2}\right)\right) = 4.2 \text{ sr}$$
(14)

The formula 15 is used again to calculate the expected received power and the corresponding photocurrent. However, the difference to the previous calculation is that the solid angle must also be taken into account before reflection. Therefore, the distance to the target is multiplied by two.

$$P_{in} = \frac{P_{out}}{\Omega \cdot (2 \cdot r)^2} \cdot A_L = \frac{123 \text{ mW}}{4.2 \text{ sr} \cdot (2 \cdot 2 \text{ m})^2} \cdot 227 \text{ mm}^2 = 416 \text{ nW}$$

$$I_{ph} = S \cdot P_{in} = 0.42 \frac{\text{A}}{\text{W}} \cdot 416 \text{ nW} = 175 \text{ nA}$$
(15)

2.3 Transimpedance amplifier

As the calculations in the previous chapters show, the photocurrents to be detected have very small amplitudes. Furthermore, the light pulses are typically only of very short duration, namely in the range of a few nanoseconds. As a result, the signals have correspondingly short rise times, which need to be detected.

The requirements indicate that an amplifier with a very high gain and a very high bandwidth must be used. The amplifier circuit used is a current-to-voltage converter. Such a circuit of the operational amplifier, as shown in Figure 3, is also called a transimpedance amplifier (TIA).

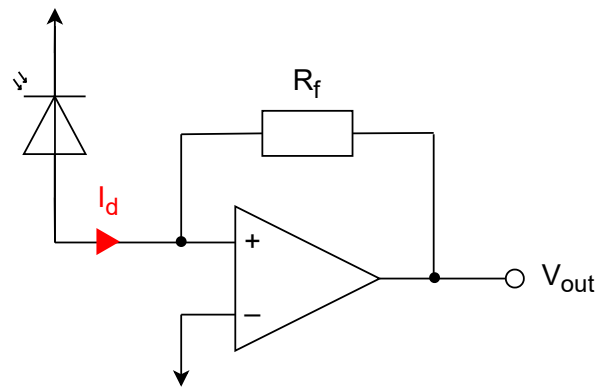


Abbildung 3: Operational amplifier circuit as a TIA

As the name suggests, the transimpedance converter converts the photocurrent I_d into a voltage V_{out} . The feedback resistor R_f determines the amplification factor, as the formula 16 shows:

$$V_{out} = -R_f \cdot I_d \quad (16)$$

3 Concept

The following subchapters explain the development work necessary for the implementation of the ToF demonstrator. In particular, they discuss how the electronics can be constructed. In addition, further challenges of such a system and how they can be overcome are shown.

3.1 Signal chain

Some explanations can be found below on how the ToF measurement is realized.

A block diagram of the overall project, consisting of the electrical and the optical part, is shown in Figure 4.

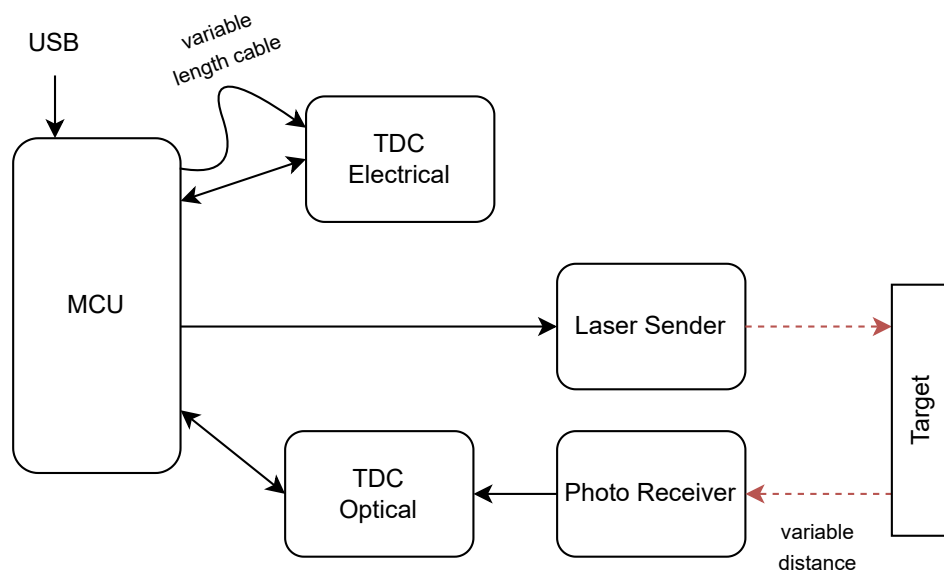


Abbildung 4: Block diagram

A more detailed sketch of the optical signal chain can be seen in the figure 5.

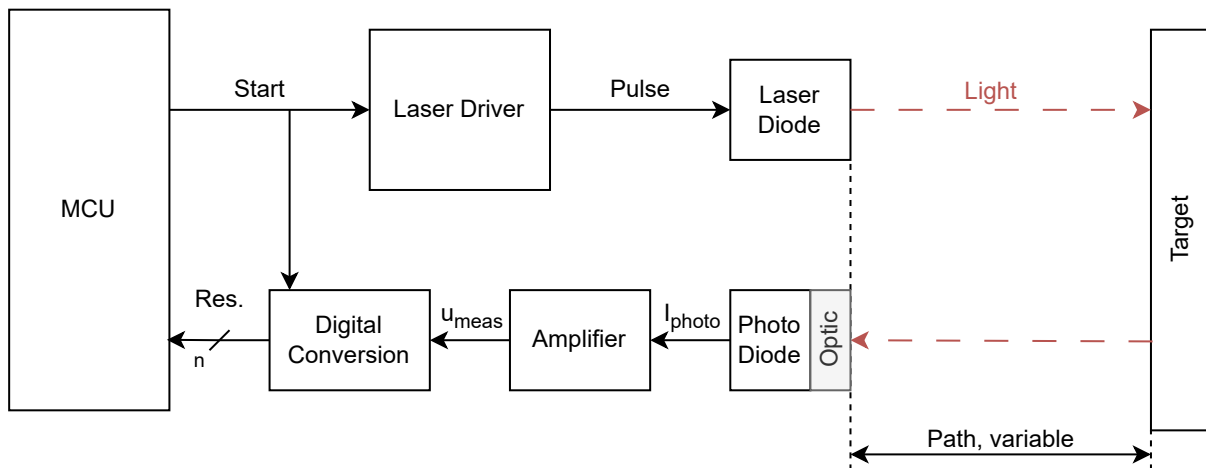


Abbildung 5: Conceptual signal path

An MCU takes care of starting and evaluating the measurement results. Generating a start signal a laser diode is triggered by a laser driver. This pulse, in the form of a light pulse after the laser diode, is then reflected by a target that is at a variable distance from the electronics.

The reflected light pulse is focused by an optical system and then converted into a photocurrent. This photocurrent is converted into a voltage by a transimpedance amplifier. Before the MCU can evaluate this voltage, it must first be digitized. This is done using a comparator. This can be understood as a 1-bit ADC, since only a distinction between the two states "light" and "no light" is necessary. The result is a received pulse.

If the time between the start signal and the received pulse is measured, it is possible to draw a conclusion about the variable distance between the electronics and the target. The time measurement is subject to the highest requirements. As symbolization: light travels a distance of 30 cm within a nanosecond. The next subchapter explains how such a time measurement can be realized.

3.2 Time measurement

In order for a direct ToF signal to be evaluated, it requires a high-resolution time measurement. The TDC7200 from Texas Instruments offers a good solution for this. According to the data sheet (TI, 2016b), it can resolve up to 55 ps with a standard deviation of 35 ps. If we now insert the speed of light and this minimum temporal resolution in a motion equation, the module should achieve the following spatial resolution according to the formula 17:

$$s_{min} = c \cdot t = 299.8 \cdot 10^6 \frac{m}{s} \cdot 55 \text{ ps} = 16.5 \text{ mm} \quad (17)$$

In principle, the ToF functionality should be put into operation in two steps. In the first part, the aim is purely to get to know the TDC7200 IC better.

Note: In reality, such a resolution will not be achievable. The challenges consist, for example, in the fact that various delays are present in the entire signal chain. For example, the switching on of the laser diode, the bandwidth of the amplifier and the switching time of the comparator

play a major role. Also, the jitter of the MCU when switching on and off of their outputs will be important at the minimum resolution. In the chapter 6.1, various measurements are used to show how close the TDC7200's specified limit is approached.

3.3 Approach

As already explained, it can be assumed that the entire signal path introduces an inaccuracy into the measurement that is far outside that of the TDC7200. It therefore makes sense to examine this in advance. For this purpose, a second TDC7200 is used on the electronics, which is responsible for measuring a purely electrical signal .

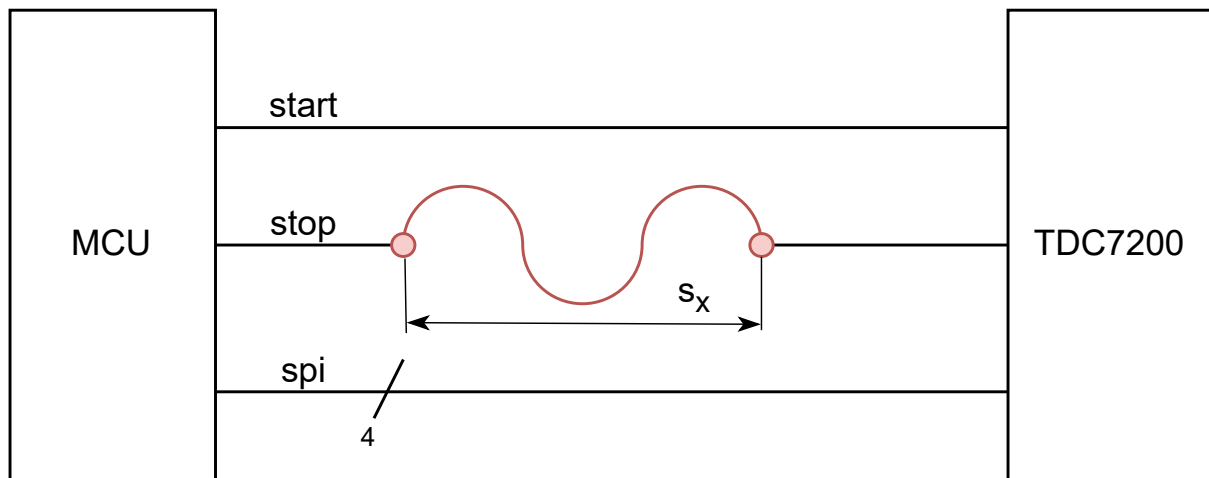


Abbildung 6: Concept operation TDC purely electric

Figure 6 shows a possible design for such a circuit. The wavy red line symbolizes a variable measurement path. Cables of different lengths can be connected here via a plug. This makes it possible to verify the measurement accuracy of the TDC7200 purely electrically.

The following points can be checked with such an extension:

- Measurement of jitter at the MCU outputs
- Configuration and operation of the TDC7200 (e.g. different measurement modes)
- Evaluation of the TDC7200 results

4 Simulations

In this chapter, the simulations carried out are documented.

The "Xpediton AMS" tool from Siemens is used for the simulation. As the name suggests, this tool supports Analog/Mixed Signal (AMS) analysis. (Siemens, 2025)

4.1 Laser Driver

Figure 7 shows the schematic for the laser driver simulation. This corresponds to the circuit from chapter 5.1.7.

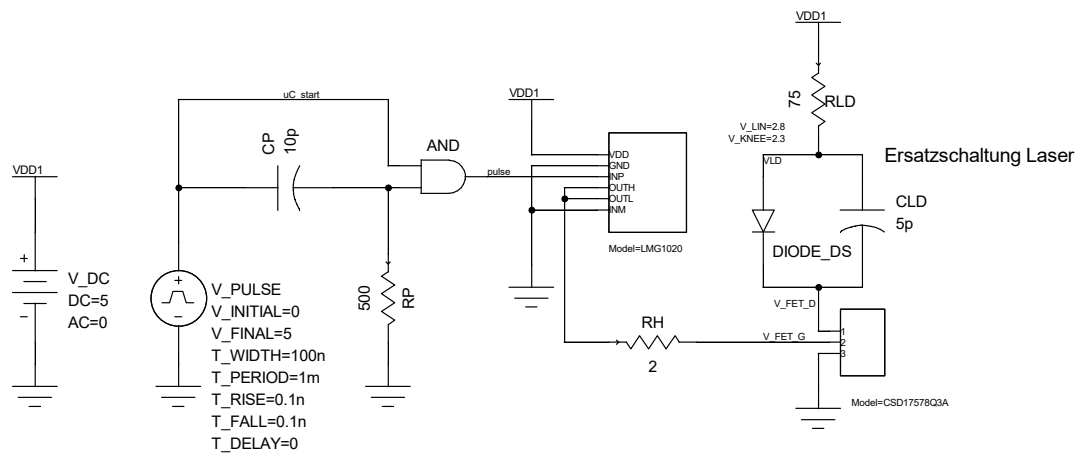


Abbildung 7: Simulation Laser Driver - Schematic

The SPICE model of the gate driver LMG1025-Q1 is the same as for LMG1020 and was downloaded from the TI website (TI, 2024b).

The SPICE model of the NexFET CSD17578Q3A was also downloaded from the TI website (TI, 2024a).

No SPICE model could be found for the laser diode RLD65NZX1, so an equivalent circuit consisting of a diode and a parallel parasitic capacitance was inserted.

The result of the simulation is shown in Figure 8.

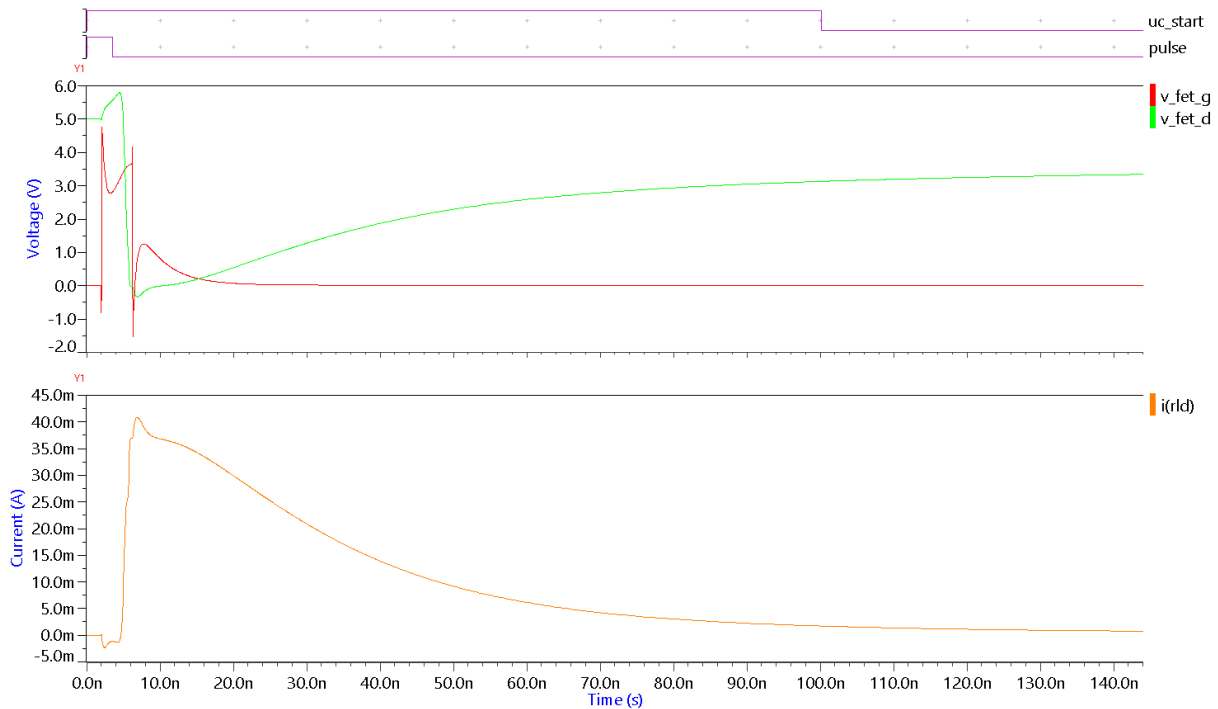


Abbildung 8: Simulation Laser-Driver - Plot

It can be seen that the laser can switch on very quickly (2 ... 3 ns). Switching off takes a little longer (about 100 ns). Since the ToF is measured up to the first edge, a longer switch-off time is not a problem for this application.

4.2 Transimpedance amplifier

Figure 9 shows the schematic for the simulation of the transimpedance amplifier. This corresponds to the circuit from chapter 5.1.8, without the comparator.

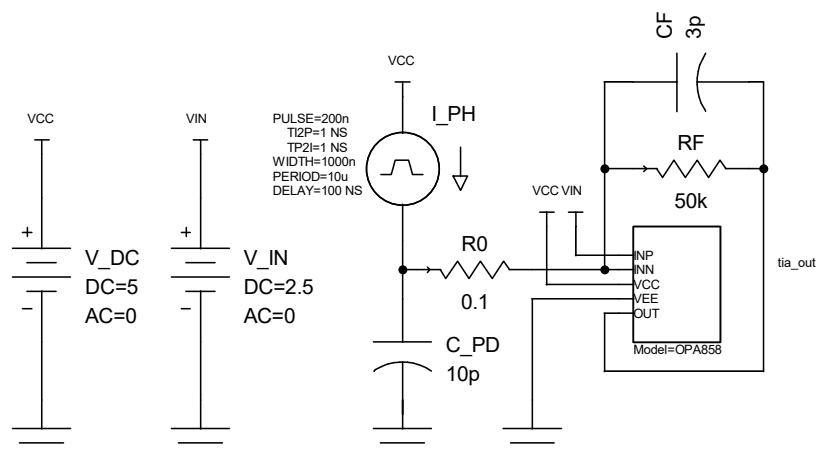


Abbildung 9: Simulation TIA - Schema

The SPICE model of the operational amplifier OPA858 was downloaded from the TI website (TI, 2024d).

No SPICE model could be found for the photo diode NJL6401R, so an equivalent circuit consisting of a current pulse source and a parallel parasitic capacitance was inserted.

The result of the simulation is shown in Figure 10.

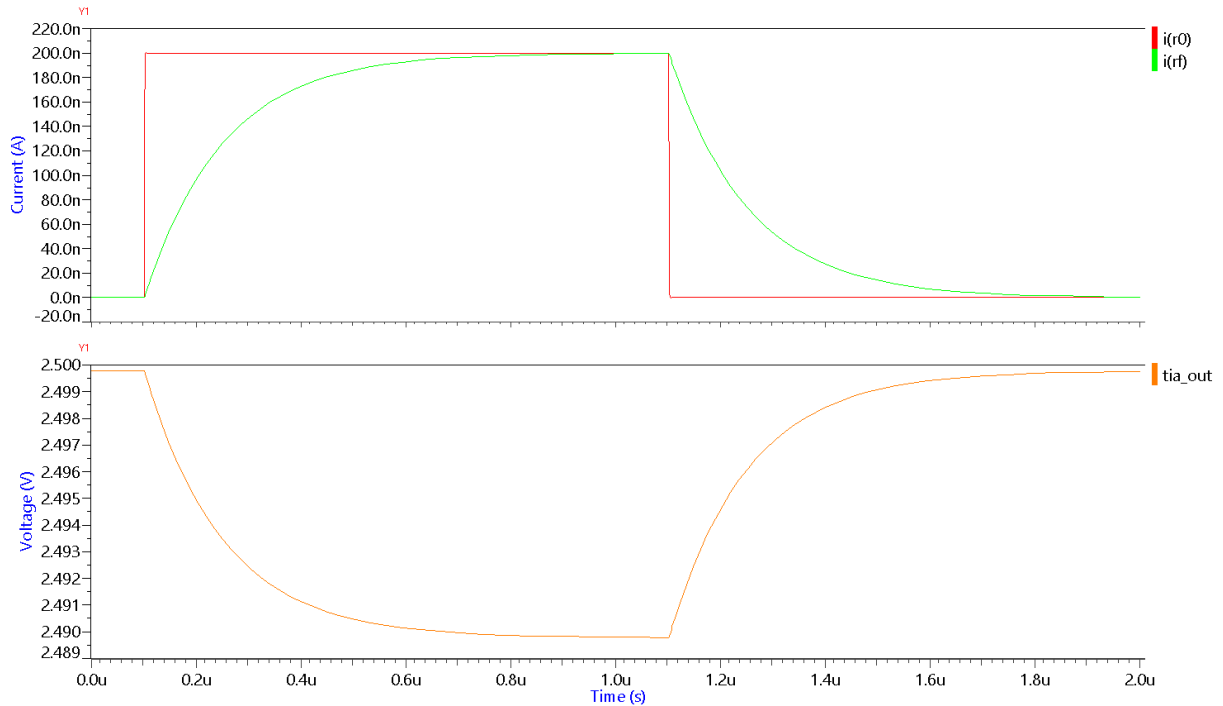


Abbildung 10: Simulation TIA - Plot

It can be seen that the time constant of the current through the feedback resistor R_F according to formula 18 can be determined.

$$\tau = R_F \cdot C_F = 50 \text{ k}\Omega \cdot 3 \text{ pF} = 150 \text{ ns} \quad (18)$$

However, the feedback capacitor C_F cannot be chosen arbitrarily small, otherwise the circuit will oscillate. Figure 11 shows the simulation result without the feedback capacitor C_F .

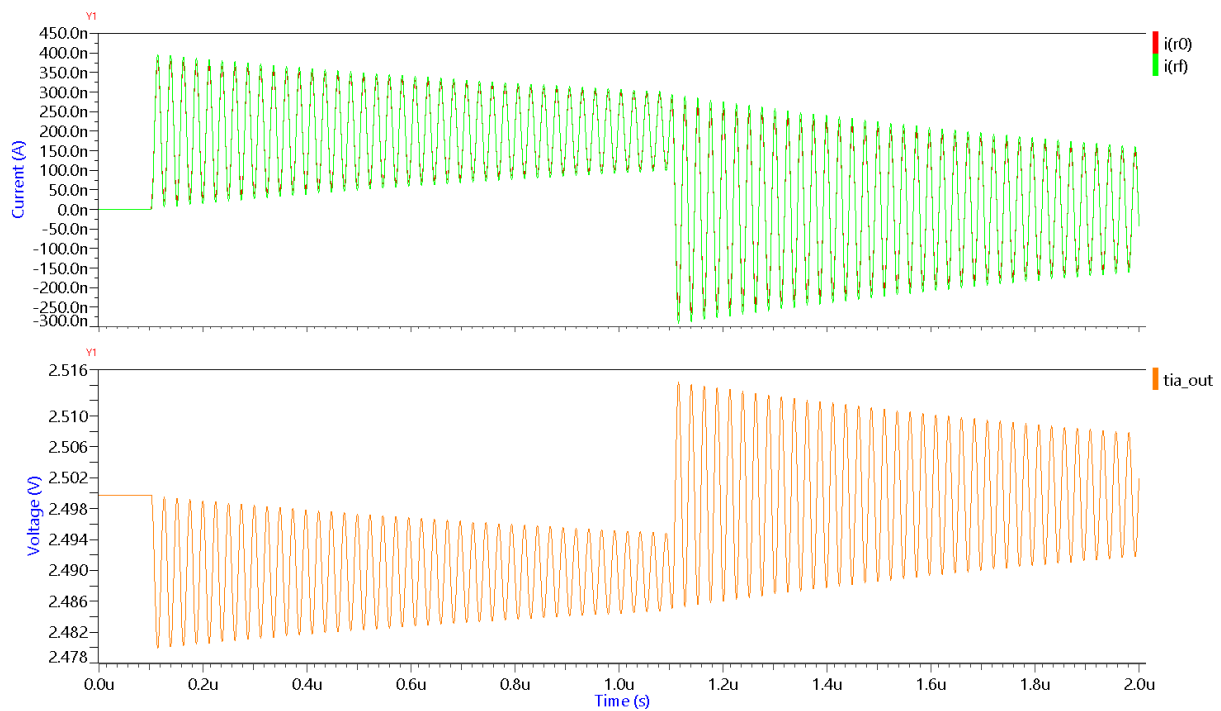


Abbildung 11: Simulation TIA - Plot without c_f

The exact dimensioning of c_f will be determined by tests with the PCB. Possibly, depending on parasitic capacities on the PCB, c_f will not even have to be populated.

5 Realization

This chapter documents the implementation of the hardware and firmware of the ToF demonstrator.

All technical documents and some photos of the fabricated demonstrator are stored in the appendix 8.4 to and including 8.9.

5.1 Circuits

In the following, all the sub-circuits that are necessary for the developed ToF evaluation module are discussed. A complete schematic can be found in the appendix 8.4. The complete project files are available in the electronic appendix of this project.

The schematic is drawn using the open-source tool "KiCad EDA 8.0" (KiCad, 2025).

5.1.1 Selective Input Voltage

Figure 12 shows the circuit for selecting the power supply.

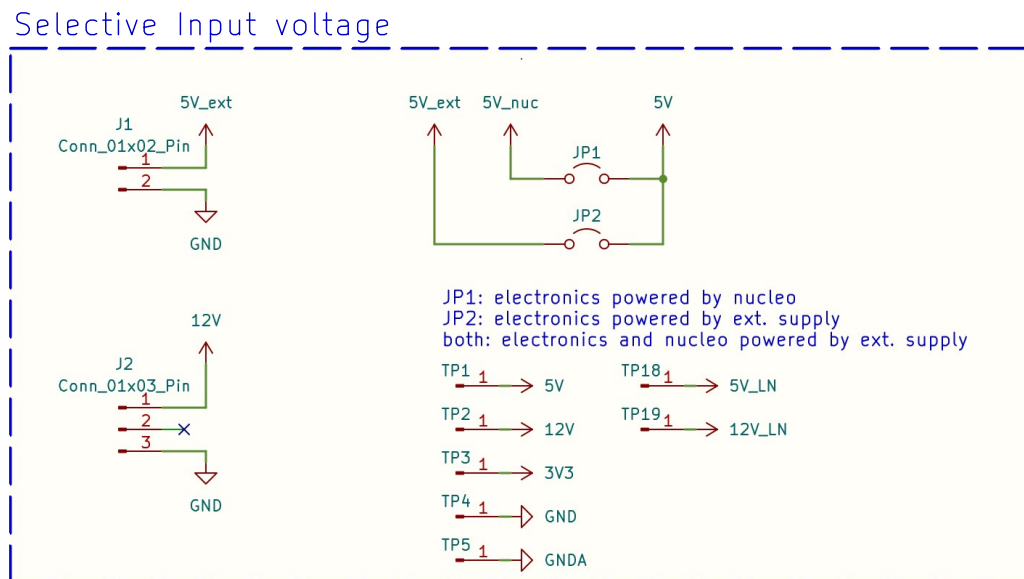


Abbildung 12: Selective Input Voltage

The Nucleo board can be powered in the following ways:

- 5 V from USB port
- 5 V from external power supply (JP1 + JP2)
- 12 V from external power supply (JP3)

See also chapter 5.1.2. The following options are available for supplying 5 V to the electronics:

- 5 V from Nucleo board (JP1)

- 5 V from external power supply (JP2)
- 12 V from external power supply via Nucleo board (JP1 + JP3)

The photodiode can be supplied with the following options:

- 5 V from 5 V electronics (sw2 position 3)
- 12 V from an external power supply (sw2 Position 1)

See also chapter 5.1.8.

5.1.2 Nucleo Board

The circuitry of the NUCLEO-F042K6 board (ST, 2024) is shown in Figure 13.

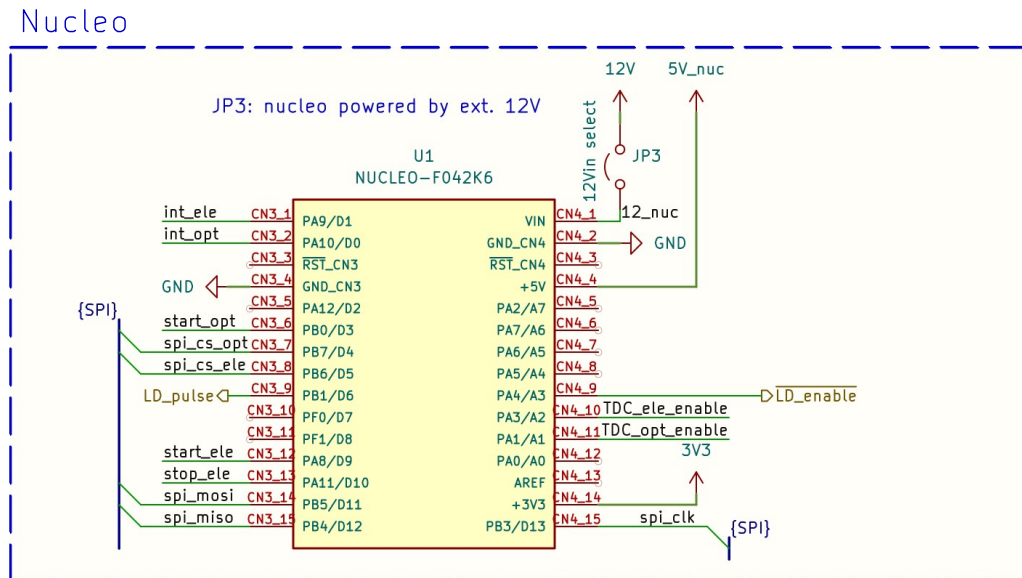


Abbildung 13: Nucleo Board

The NUCLEO board is a so-called "development kit", which contains an STM32F042K6 MCU. Various pins of the Microcontroller Unit (MCU) are made available via pin headers at the edge of the board. This makes the integration into your own electronics. The MCU is flashed via an USB interface.

In this design, the development board is also needed to operate the TDC7200 ICs. For this purpose, a Serial Peripheral Interface (SPI) is used to configure and read the measurements (summarized as SPI bus in the figure). With the signals `start_ele` or `start_opt` measurements can be started on the TDCs. For the TDC that is responsible for the electrical signals, `stop_ele` can also be used to generate a stop pulse. Finally, the NUCLEO is responsible for controlling the laser diode, which is done with the signals `LD_pulse` and `LD_enable`

5.1.3 TDC Electrical Signal

The electrical circuit of the TDC7200 (TI, 2016b) for the electrical part is shown in Figure 14.

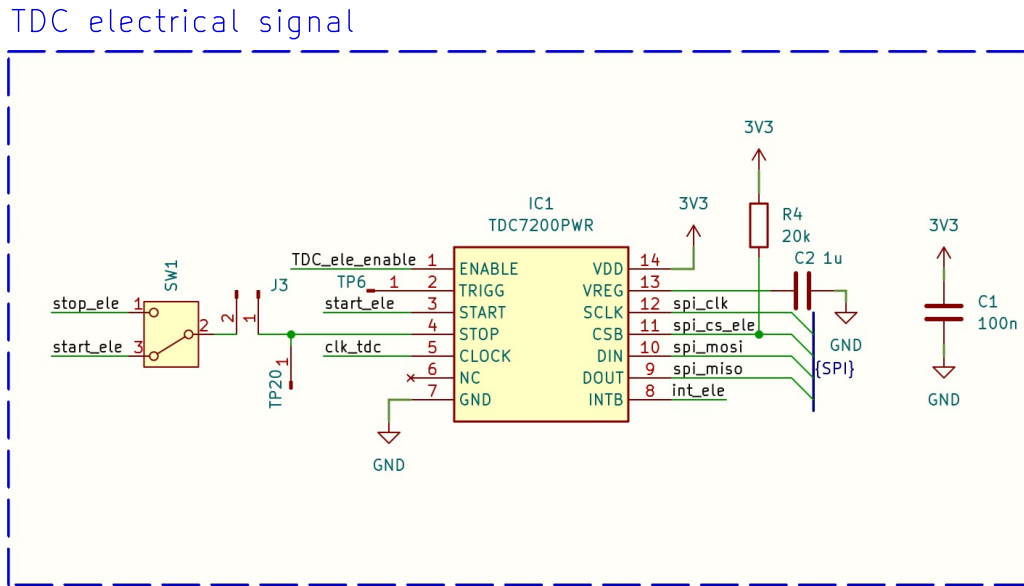


Abbildung 14: TDC Electrical Signal

The TDC7200 is connected to the MCU via the start and stop lines, an enable signal and the SPI lines.

As already mentioned in chapter 3.3, the electrical TDC is used to to be put into operation for the first time and to familiarize yourself with it. At the J3 connection, a cable of any length can be connected in a next step. This makes it possible to read the first measurement results, purely electrically of course, from the TDC7200.

Using the switch SW1, the STOP signal can be generated either via stop_ele or start_ele. This offers the possibility, on the one hand, to measure the time between the switching of two GPIOs. On the other hand, the same GPIO pin can be used to generate the START and STOP signals, which allows the delay time to be used to directly determine the cable length at J3.

5.1.4 TDC Optical Signal

The circuitry of the TDC7200 (TI, 2016b) for the optical part is shown in Figure 15.

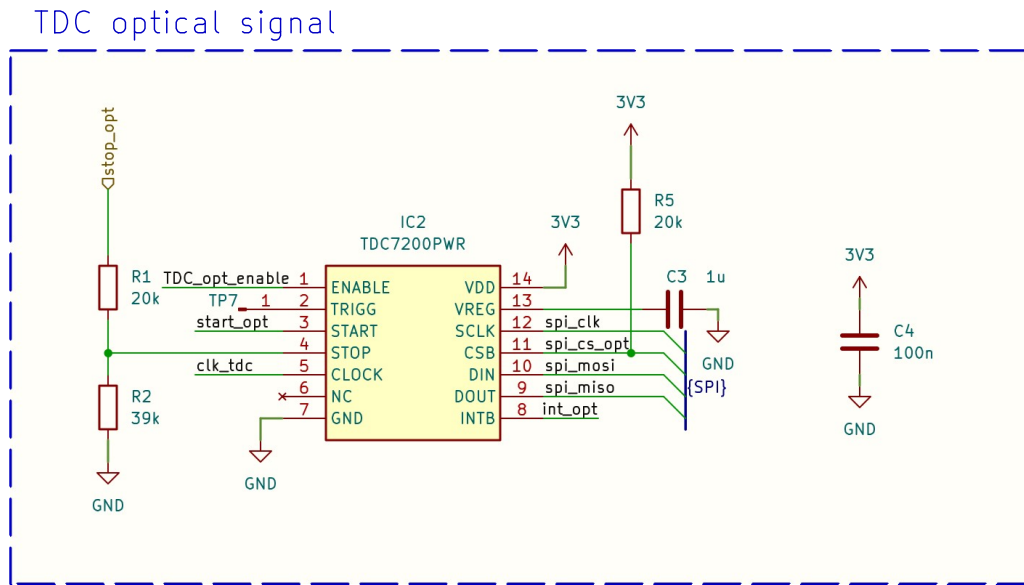


Abbildung 15: TDC Optical Signal

The TDC is connected in practically the same way for the optical measurement as for the electrical counterpart. The main difference here, however, is that the `STOP` signal is not generated by the MCU itself, but by a comparator that is placed at the end of the optical measurement path. Since the comparator works with a 5 V level, the voltage divider $R1 / R2$ is needed to divide the 5 V pulse to the appropriate 3.3 V.

5.1.5 Oscillator For TDCs

The oscillator circuit for the two TDC is shown in Figure 16. It is a normal quartz oscillator with an integrated oscillating circuit. Conveniently, this means that no further circuitry is required.

Oscillator for TDCs

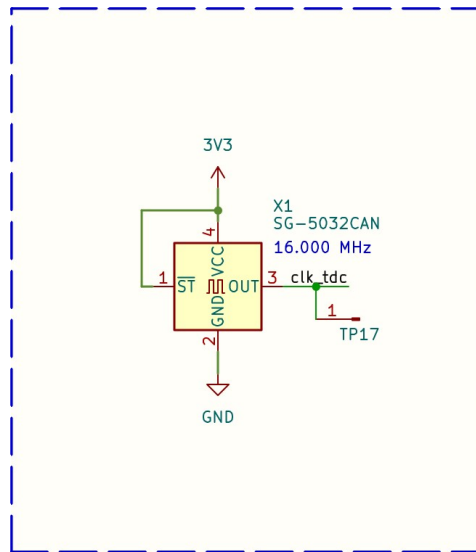


Abbildung 16: Oscillator for TDCs

5.1.6 Power Supply Separation

For the circuitry of the photodiode, including the TIA and comparator, it makes sense to have a power supply with as little noise as possible.

For this purpose, the separation was carried out, which is shown in Figure 17.

Powersupply separation

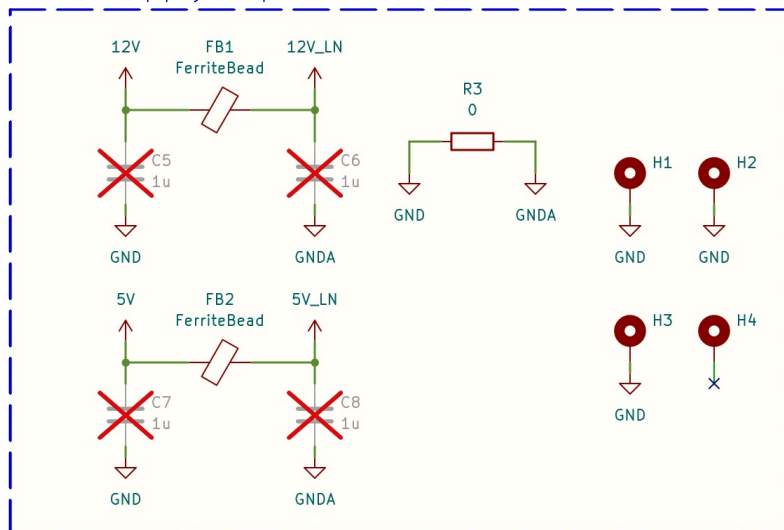


Abbildung 17: Power Supply Separation

In principle, the supplies should be somewhat suppressed by a ferrite bead. Of course, the physical course of the supply path on the layout is also decisive here. More about this in the corresponding chapter 5.2. Alternatively, in addition to the ferrite beads, there is the option of

further decoupling the supply using capacitors. However, assumed that this is not necessary in a first step.

5.1.7 Laser Driver

The laser diode RLD65NZX1 (ROHM, 2019) is driven by a laser driver LMG1025-Q1 (TI, 2024c) and NexFET (TI, 2016a). The AND gate IC21 (Diodes Inc., 2020) together with the high-pass filter (C21, R21 and RV21) implements a pulse shortener. This path can be used to generate pulses in the range of 0.5 ... 100 ns for the laser. See figure 18.

The resistor R25 forms the series resistor for the laser diode, which is calculated according to the formula 19.

$$R_v = \frac{V_{cc} - V_{fld}}{I_{ld}} = \frac{5\text{ V} - 2\text{ V}}{40\text{ mA}} = 75\ \Omega \quad (19)$$

A placeholder value is currently shown in the diagram. During commissioning, the resistance is. If it is reduced, the current through the laser diode increases and thus the emitted light power.

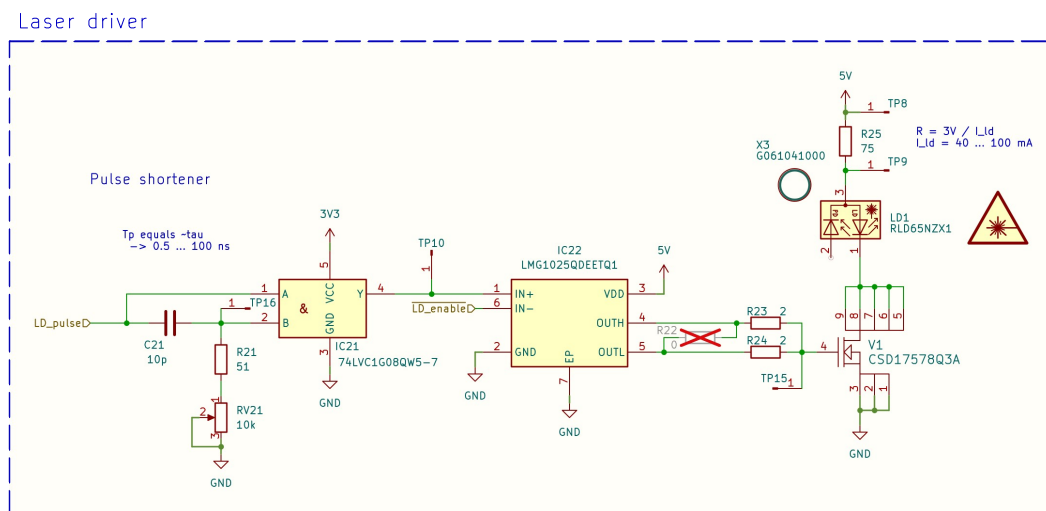


Abbildung 18: Laser Driver

5.1.8 Photo Receiver

To amplify the photocurrent of the photodiode NJL6401R (JRC, 2014) and convert it into a voltage, a transimpedance amplifier is built with the operational amplifier OPA858 (TI, 2018). The output of the transimpedance amplifier goes to the comparator TLV3501 (TI, 2016c) to generate the STOP signal for the TDC. See figure 19.

The feedback resistor R26 can be changed as needed. Increasing it also increases the negative pulse at the output of the TIA. With C20 a small capacitance can be added if needed. This is to suppress oscillation of the amplifier, should it have a tendency to oscillate.

The offset of the OPA858 and the switching threshold of the TLV3501 can be adjusted using the two trimmers RV22 and RV23. This helps during commissioning, since it is not yet clear in

advance where the output of the operational amplifier will be due to the converted dark current of the photodiode.

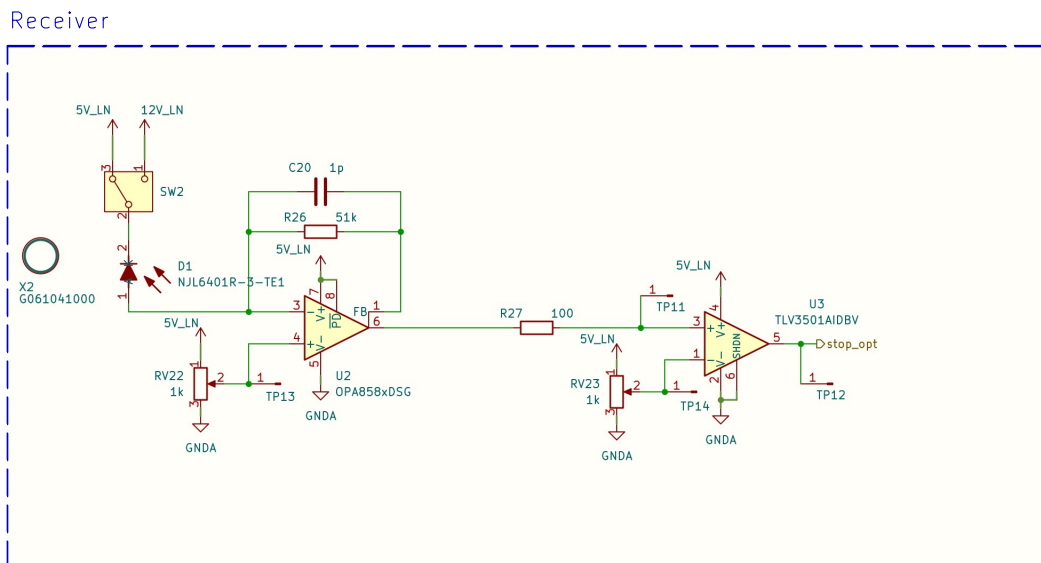


Abbildung 19: Photo Receiver

5.1.9 Decoupling Capacitors

The circuitry of the decoupling capacitors is shown in figure 20. The capacitors for the operational amplifier OPA858 and for the comparator TLV3501 are taken from the suggestion in the data sheet. In addition, the gate driver at the laser driver is supported by an additional, larger capacity, which should have a positive effect on the stability of the supply.

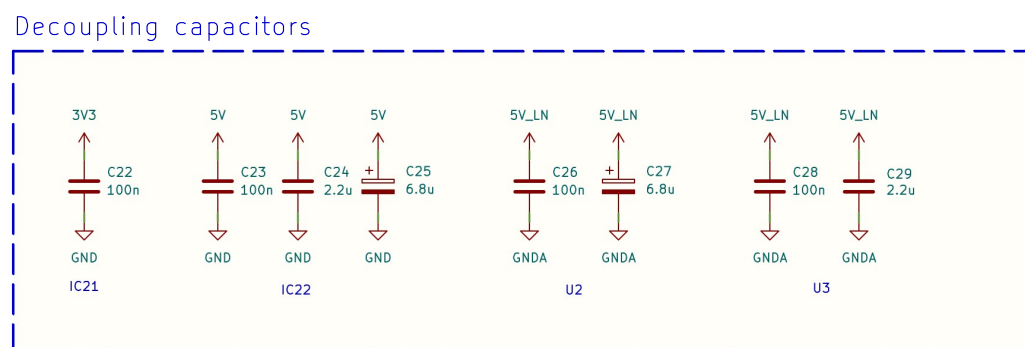


Abbildung 20: Decoupling Capacitors

5.2 Layout

When laying out the circuit board, care is taken to ensure that the various circuits on the PCB do not interfere with each other. To this end, a clean power supply concept is used that defines the ground and power planes by layer. In principle, the four layers of the circuit board are separated as follows:

- Top layer: signal and ground
- Inner layer 1: power supply layer
- Inner layer 2: signal and ground
- Bottom layer: signal and ground

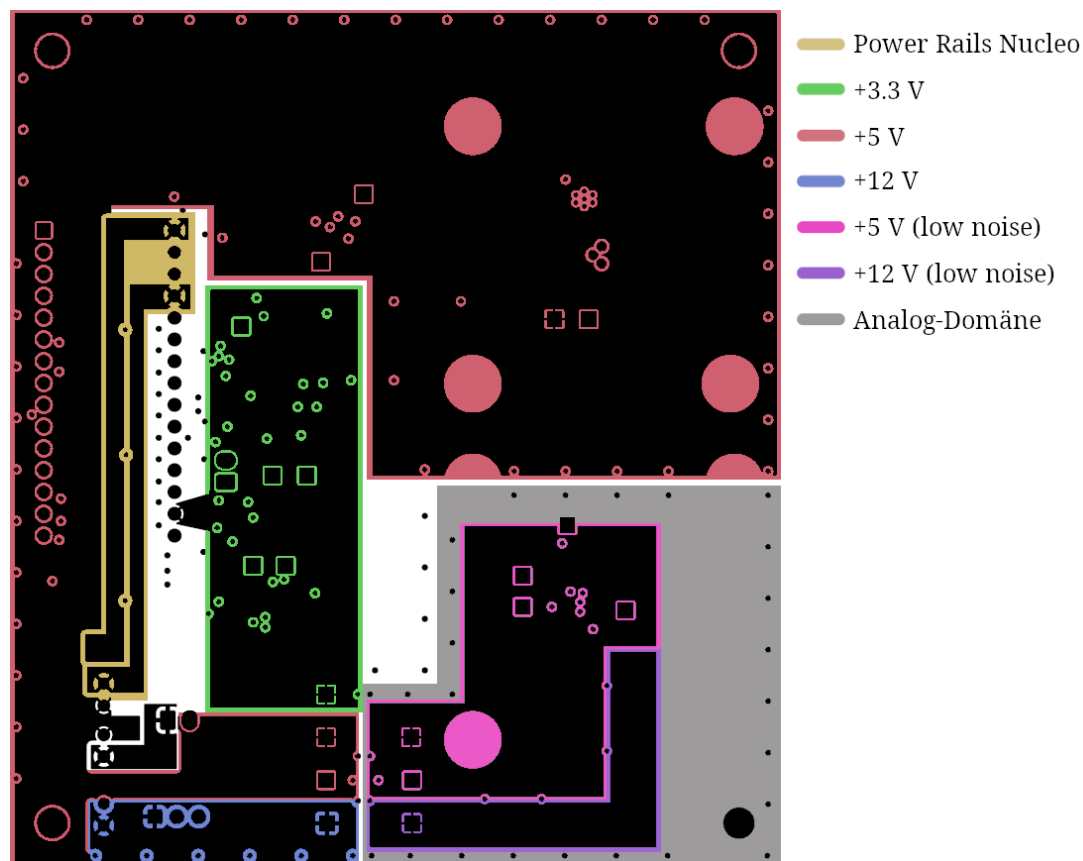


Abbildung 21: Inner Layer 1 with notes on the supply concept

Figure 21 shows the implementation of the supply concept. In the lower right corner contains the photodiode, the TIA and the comparator. This section of the PCB has its own ground planes and suppressed power supplies. With the help of a few vias at the outer boundary of this section, the analog domain should also be less susceptible to externally induced interference fields.

Such so-called "stitching vias" are also placed around the whole PCB for shielding.

The complete layout can be found in the appendix 8.6.

5.3 3D View

A 3D view can be helpful when designing the layout and especially when placing components. This is because it shows where mechanical and electronic components could get in each other's way. The figure 22 shows a 3D rendering of the top view of the ToF demonstrator.

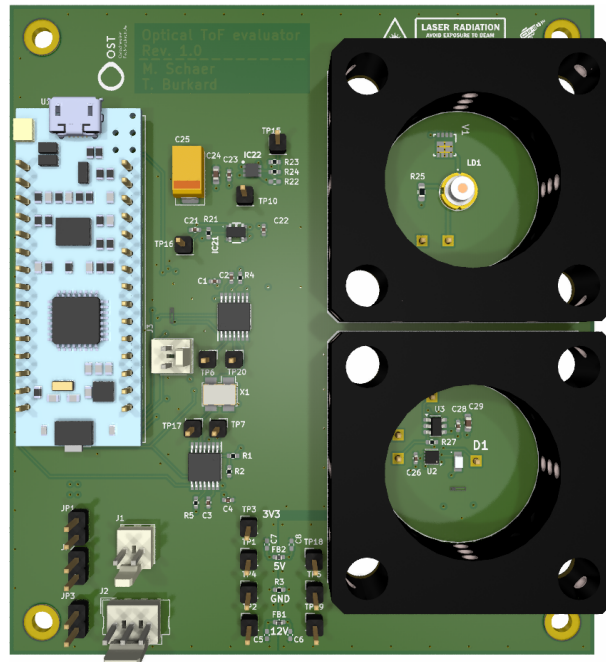


Abbildung 22: 3D View Top

The lens mounts are integrated into the view using a step file provided by QIOPTIQ (Qioptiq, 2024b). In the figure 22, these are shown as black cuboids with cylindrical cutouts. There must be no components directly under these holders, as otherwise the holders cannot be mounted.

When placing the components, care is also taken to ensure that all test points, trimmers and switches for configuration are easily accessible. If these components are close to the optoelectronic components, they are placed on the underside. This means that they are also available with the optics mounted.

5.4 Fabrication

KiCad 8.0 allows you to create various manufacturing files for the different layers from the layout design. These so-called Gerber and NC-Drill files can be used by a PCB manufacturer to produce a printed circuit board. Optionally, a printed circuit board can also be assembled directly, but this takes additional time in production.

Since most components can be mounted relatively easily on the PCB, automated assembly is avoided and the components are mounted by hand.

The components are ordered from Mouser Electronics, while the printed circuit boards are (JLC, 2025) obtained from the Chinese manufacturer JLCPCB. Mouser meets the expected delivery time of a few days. The order from JLCPCB also arrives on time, exactly seven days later.

JLCPCB's engineers do not need to ask any questions about the design, which probably has a positive effect on the delivery time.

The finished printed circuit board can be seen in the figure 23. Further photographs of the ToF demonstrator can be found in the appendix 8.9.

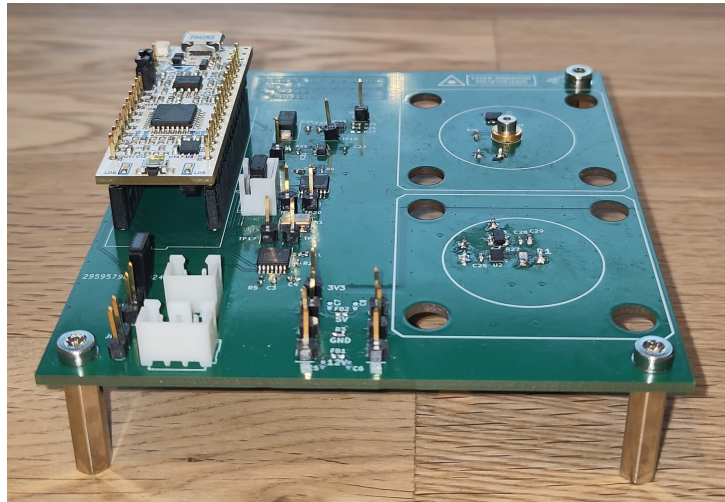


Abbildung 23: Demonstrator from the front without lens

5.5 Firmware

When it comes to the firmware, care is taken to ensure that there is a clean separation between the application, the driver and the peripherals. This ensures that the design is as modular as possible. The FW hierarchy is Figure 24 comprehensible.

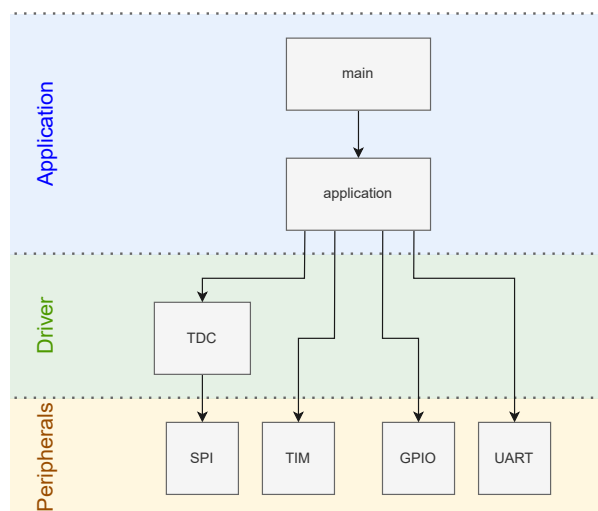


Abbildung 24: Structure of the firmware project

The application is responsible for the timing of the measurements. The code for triggering is expanded step by step during the measurement phase, starting with the use of simple GPIOs, and ending with optimal timing behavior using various hardware timers to generate the trigger

pulses for the TDCs. The individual expansion steps are explained in more detail in chapter 6.1. explains the individual expansion steps in more detail.

The TDC driver provides functions for initializing and enabling the TDC. See code 1 for an example.

```
1 TDC_init(&tdc_opt, &hspi1, SPI_CS_OPT_GPIO_Port, SPI_CS_OPT_Pin,  
    TDC_OPT_ENABLE_GPIO_Port, TDC_OPT_ENABLE_Pin);  
2  
3 TDC_enable(&tdc_opt);
```

Code 1: TDC Init and Enable with Driver

The TDC driver contains the complete register definition of the TDC7200. The driver also provides simple read and write routines for communication via SPI. This makes it easy to reconfigure, set up and read out the measurement IC. An example is shown in Code 2.

```
1 TDC_read(&tdc_ele, TDC_ADR_CONFIG1, data);  
2  
3 data[0] |= TDC_CONFIG1_MEAS_MODE_2;  
4  
5 TDC_write(&tdc_ele, TDC_ADR_CONFIG1, data);
```

Code 2: TDC Read/Write with Driver

Furthermore, the driver can directly convert the measurement results using the formulas given in the data sheet formulas. The code 3 shows an example of code for a single measurement.

```
1 uint64_t tof_fs = 0;  
2  
3 TDC_start(&tdc_opt);  
4  
5 /* ... generate start and stop signals */  
6  
7 if (TDC_read_result(&tdc_opt, &tof_fs) != TDC_OK) {  
8     for (;;) {} // Error handling...  
9 }  
10  
11 /* tof_fs now holds the measurement result, in [fs] */
```

Code 3: TDC Measurement with driver

The self-developed firmware driver for the TDC is in the appendix 8.2.

With the development environment STM32CubeIDE / CubeMX (ST, 2025), it is possible to MCU can be configured graphically. With the help of a code generator, the initialization routines for all peripherals can be generated automatically. This allows the focus to be placed on the TDC driver and the actual application during development.

The pin configuration can be seen in the figure 25.

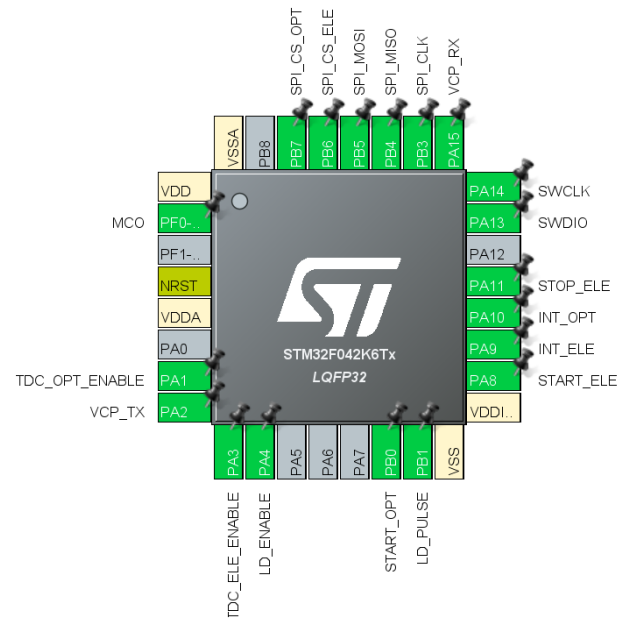


Abbildung 25: Pin configuration of the STM32F042K6

6 Results

The results are documented in this chapter.

The Python scripts used to calculate the statistical values and to plot the diagrams can be found in the Appendix 8.3.

6.1 Electrical Measurements

This subchapter documents the measurement results that were recorded purely electrically (i.e. without the optical part).

The Digital Storage Oscilloscope (DSO) used is a Rohde & Schwarz RTB2004 1.25 GSa/s.

The time measurements are carried out by IC1 (see figure 14) and triggered and read by the firmware, which is triggered and read on the Nucleo board u1 (see figure 13).

6.1.1 GPIO Toggle with HAL

First, the time it takes for the CPU of the MCU to switch two GPIO pins using the Hardware Abstraction Layer (HAL) library (ST, 2020) is measured.

The firmware implementation for this is shown in Code 4.

```
1 // Configure TDC
2
3 TDC_init(&tdc_ele, &hspi1, SPI_CS_ELE_GPIO_Port, SPI_CS_ELE_Pin,
4         TDC_ELE_ENABLE_GPIO_Port, TDC_ELE_ENABLE_Pin);
5
6 TDC_enable(&tdc_ele);
7
8 TDC_read(&tdc_ele, TDC_ADR_CONFIG1, data);
9 data[0] |= TDC_CONFIG1_MEAS_MODE_2;
10
11 TDC_write(&tdc_ele, TDC_ADR_CONFIG1, data);
12
13 while (1) {
14     TDC_start(&tdc_ele);
15
16     // Set Pins to High with HAL
17     HAL_GPIO_WritePin(START_ELE_GPIO_Port, START_ELE_Pin, GPIO_PIN_SET);
18     HAL_GPIO_WritePin(STOP_ELE_GPIO_Port, STOP_ELE_Pin, GPIO_PIN_SET);
19
20     TDC_read_result(&tdc_ele, &tof_fs);
21
22     sprintf(string, "ToF = %lu [ps]\n", tof_fs / 1000);
23     HAL_UART_Transmit(&huart2, (uint8_t *)string, strlen(string), 10000);
24
25     // Set Pins to Low with HAL (preparation for next iteration)
26     HAL_GPIO_WritePin(START_ELE_GPIO_Port, START_ELE_Pin, GPIO_PIN_RESET);
27     HAL_GPIO_WritePin(STOP_ELE_GPIO_Port, STOP_ELE_Pin, GPIO_PIN_RESET);
28
29     HAL_Delay(10); // 10 ms
30 }
```

Code 4: GPIO Toggle with HAL

The TDC measures the time between lines 15 and 16 in Code 4. To do this, the STOP pin of the TDC is connected to `stop_ele` via SW1. The cable connection J3 is shorted. See figure 14.

Via UART, 2000 measurement values were recorded. A section of this is shown in Code 5. The remaining data can be found in the electronic appendix.

```

1 ToF = 6375779 [ps]
2 ToF = 6374666 [ps]
3 ToF = 6377003 [ps]
4 ToF = 6375333 [ps]
5 ToF = 6377061 [ps]
6 ToF = 6374721 [ps]
7 ToF = 6377504 [ps]
8 ToF = 6374888 [ps]
9 ToF = 6376725 [ps]
10 ToF = 6377005 [ps]
11 ...

```

Code 5: GPIO Toggle with HAL

The arithmetic mean and standard deviation are listed in formula 20.

$$\begin{aligned}\overline{ToF} &= 6'375'888 \text{ ps} \\ \sigma &= 1'059 \text{ ps}\end{aligned}\tag{20}$$

Since the CPU is running at 8 MHz, it can be concluded that a pin toggle with HAL takes about 50 CPU cycles. This seems plausible.

The histogram and boxplot are shown in Figure 26 and 27 shown.

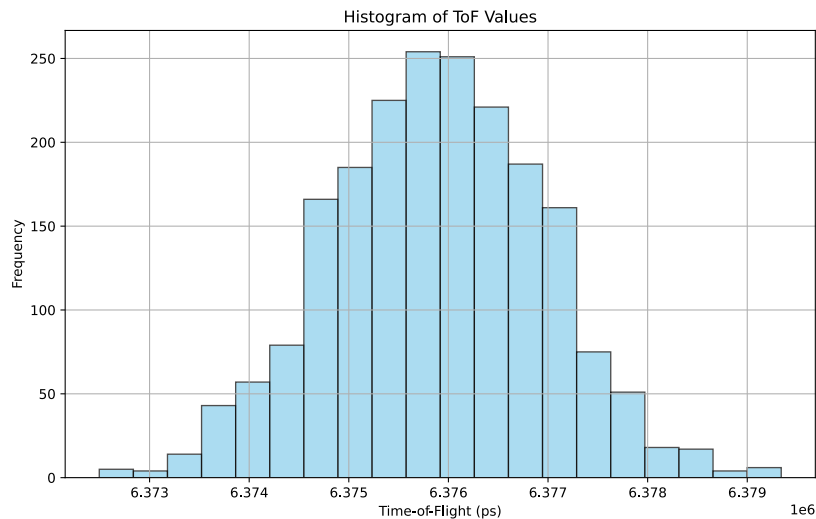


Abbildung 26: GPIO Toggle with HAL - Histogram

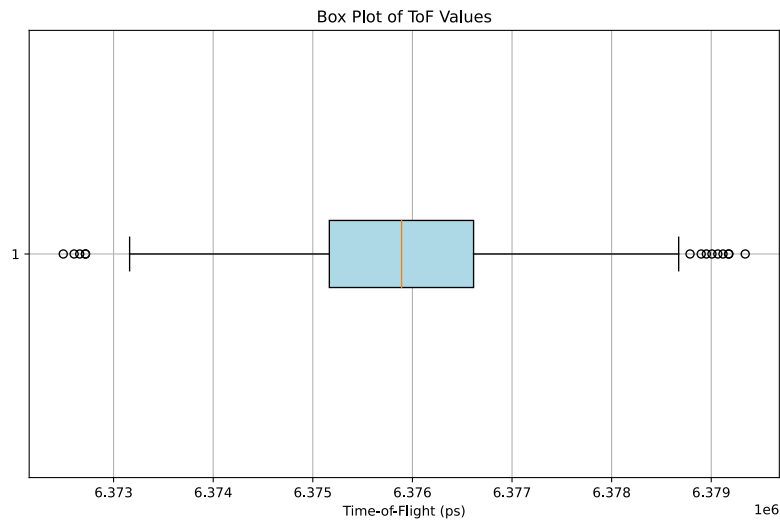


Abbildung 27: GPIO Toggle with HAL - Boxplot

To validate the results of the TDC, the same measurement was also carried out using Digital Storage Oscilloscope (DSO). The measurements are shown in Figure 28 and 29.



Abbildung 28: GPIO Toggle with HAL - DSO



Abbildung 29: GPIO Toggle with HAL - DSO (Zoom)

6.1.2 GPIO Toggle without HAL

Next, we measure how long it takes for the CPU of the MCU to switch two GPIO pins using direct register access (via pointer; without HAL library).

The firmware implementation for this is shown in Code 6.

```

1 // Configure TDC
2
3 TDC_init(&tdc_ele, &hspi1, SPI_CS_ELE_GPIO_Port, SPI_CS_ELE_Pin,
4         TDC_ELE_ENABLE_GPIO_Port, TDC_ELE_ENABLE_Pin);
5 TDC_enable(&tdc_ele);
6
7 TDC_read(&tdc_ele, TDC_ADR_CONFIG1, data);
8 data[0] |= TDC_CONFIG1_MEAS_MODE_2;
9
10 TDC_write(&tdc_ele, TDC_ADR_CONFIG1, data);
11
12 while (1) {
13     TDC_start(&tdc_ele);
14
15     // Set Pins to High with direct register access
16     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) |= (1 << 8); // START_ELE
17     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) |= (1 << 11); // STOP_ELE
18
19     TDC_read_result(&tdc_ele, &tof_fs);
20
21     sprintf(string, "ToF = %lu [ps]\n", tof_fs / 1000);
22     HAL_UART_Transmit(&huart2, (uint8_t *)string, strlen(string), 10000);
23
24     // Set Pins to Low with direct register access (preparation for next
25     // iteration)
26     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) &= ~(1 << 8); // START_ELE
27     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) &= ~(1 << 11); // STOP_ELE
28
29     HAL_Delay(10); // 10 ms

```

28 }

Code 6: GPIO Toggle without HAL

The TDC thus measures the time between lines 15 and 16 in Code 6. To do this, as in chapter 6.1.1, the STOP pin of the TDC is connected to SW1 via SW1 stop_ele. The cable connection J3 is shorted. See Figure 14.

Via UART, 2000 measurement values were recorded. A section of this is shown in Code 7 shown. The remaining data can be found in the electronic appendix.

```

1 ToF = 1377894 [ps]
2 ToF = 1378450 [ps]
3 ToF = 1377615 [ps]
4 ToF = 1377840 [ps]
5 ToF = 1377729 [ps]
6 ToF = 1377615 [ps]
7 ToF = 1377337 [ps]
8 ToF = 1378063 [ps]
9 ToF = 1377949 [ps]
10 ToF = 1377615 [ps]
11 ...

```

Code 7: GPIO Toggle ohne HAL

The arithmetic mean and standard deviation are listed in formula 21.

$$\begin{aligned}\overline{ToF} &= 1'377'773 \text{ ps} \\ \sigma &= 402 \text{ ps}\end{aligned}\tag{21}$$

Since the CPU is running at 8 MHz, it can be concluded that a pin toggle without HAL takes about 10 CPU cycles. This seems plausible.

The histogram and boxplot are shown in figures 30 and 31, respectively.

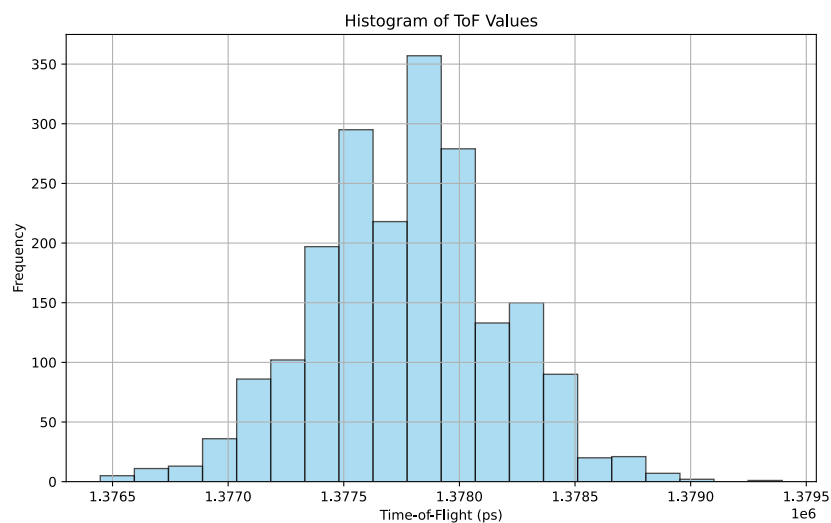


Abbildung 30: GPIO Toggle without HAL - Histogram

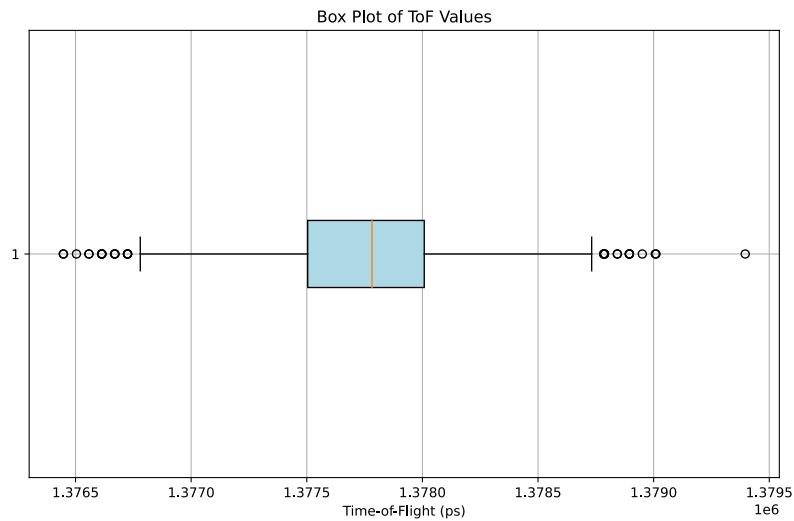


Abbildung 31: GPIO Toggle without HAL - Boxplot

To validate the results of the TDC, the same measurement was also carried out using Digital Storage Oscilloscope (DSO). The measurements are shown in Figure 32 and 33.

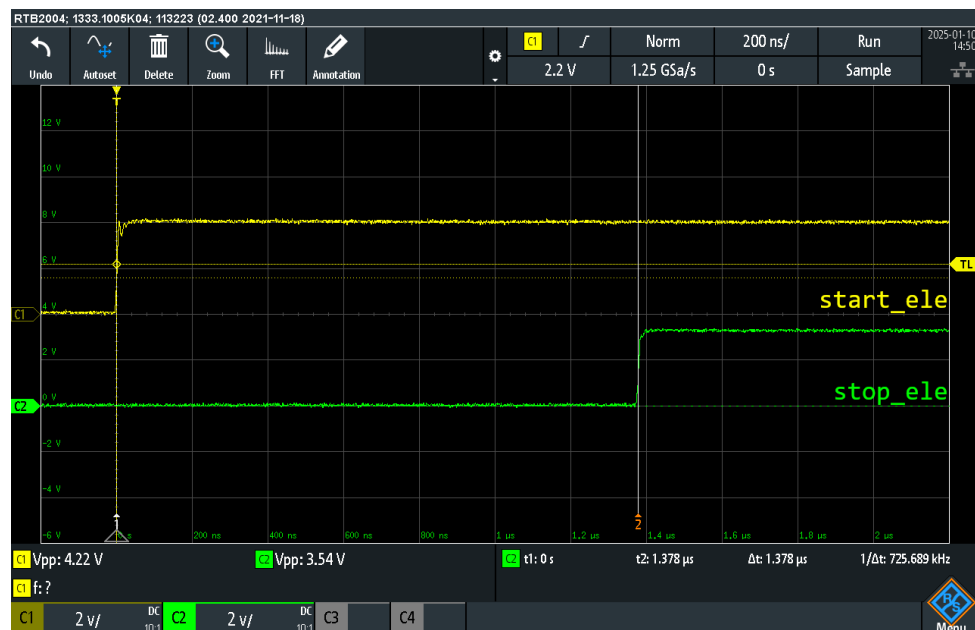


Abbildung 32: GPIO Toggle without HAL - DSO



Abbildung 33: GPIO Toggle without HAL - DSO (Zoom)

6.1.3 Different cable lengths

For this measurement, the same setup as in chapter 6.1.2 is used.

Instead of short-circuiting J3 (see figure 14), different cable lengths are now connected.

A single insulated copper wire ($\varnothing_{cu} = 0.8 \text{ mm}$, $\varnothing_{ges} = 1.6 \text{ mm}$) is used as the cable.

It has been found that a circular arrangement of the cable is important. This is because when the the two cable ends overlap, short times are measured. This has to do with the capacitive coupling between the conductors.

The results are shown in Figure 34. The list with the data points can be found in the electronic appendix.

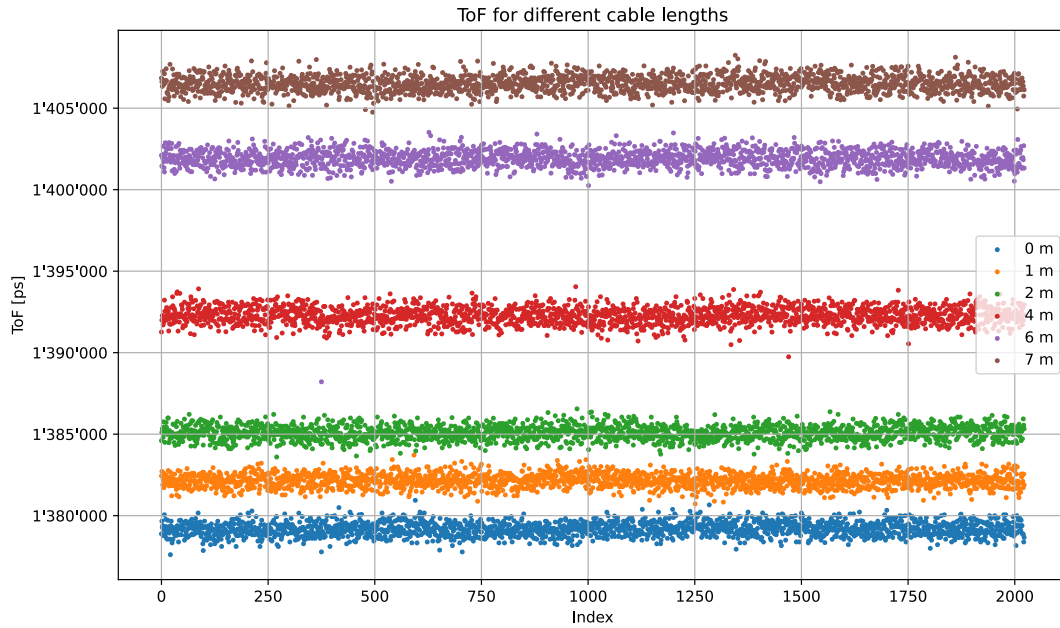


Abbildung 34: Different cable lengths

The arithmetic mean values and standard deviations are listed in Table 1.

Length	Mean value	Standard deviation
0 m	1'379'188 ps	421 ps
1 m	1'382'152 ps	413 ps
2 m	1'385'074 ps	437 ps
4 m	1'392'275 ps	519 ps
6 m	1'401'903 ps	577 ps
7 m	1'406'512 ps	491 ps

Tabelle 1: Different cable lengths

The signal propagation speed in copper is approximately $\frac{2}{3}$ of the speed of light (firewall.xc, 2025). To validate the results in Table 1, we calculate back to the cable length, as shown in formula 22. The runtime at 0 m is subtracted to compensate for the delay caused by switching the GPIOs.

$$\begin{aligned}
 c_{cu} &\approx \frac{2}{3} \cdot c_0 = \frac{2}{3} \cdot 299'792'458 \frac{m}{s} \approx 200'000'000 \frac{m}{s} \\
 ToF_n &= ToF_{n_{abs}} - ToF_0 \\
 l_n &= ToF_n \cdot c_{cu}
 \end{aligned} \tag{22}$$

The results are shown in Table 2.

Actual length	ToF _n	Calculated length
0 m	0 ps	
1 m	2'964 ps	0.6 m
2 m	5'886 ps	1.2 m
4 m	13'087 ps	2.6 m
6 m	22'715 ps	4.5 m
7 m	27'324 ps	5.5 m

Tabelle 2: Calculated cable lengths

It is noticeable that the results do not match exactly. However, a clear correlation can be seen. The difference could have various causes. One possible explanation would be that the propagation speed of the cable used is closer to the speed of light than assumed. Further measurement results and evaluations in chapter 6.1.7.

6.1.4 Mode 1 vs. Mode 2

The TDC7200 supports two modes with different measurement ranges (TI, 2016b):

- Mode 1: 12 ns to 500 ns
- Mode 2: 250 ns to 8 ms

In mode 1, only the internal ring oscillator of the TDC is used. See figure 35.

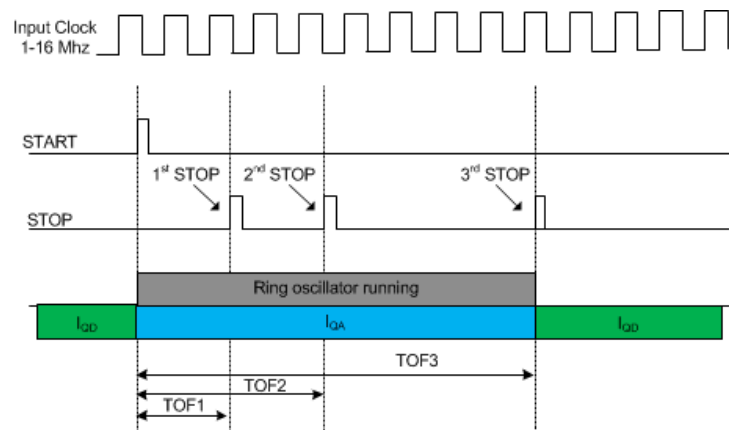


Abbildung 35: TDC Mode 1 (TI, 2016b)

In mode 2, to be able to measure longer times, the external clock of the TDC is also used. See Figure 36.

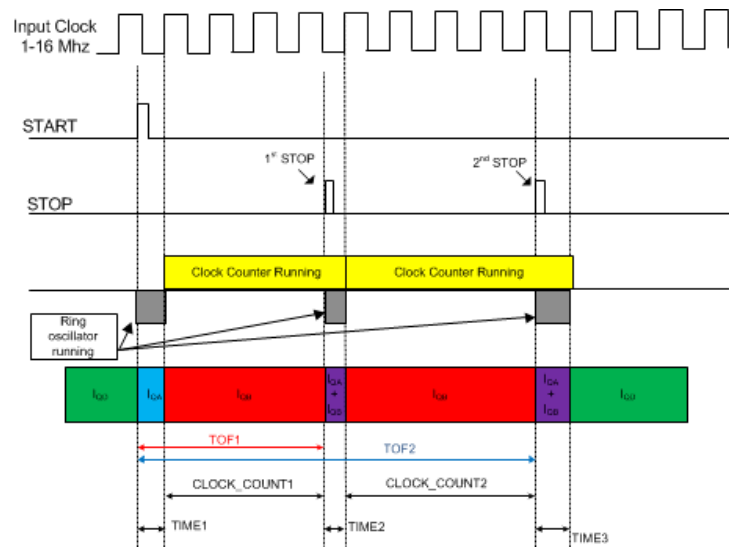


Abbildung 36: TDC Mode 2 (TI, 2016b)

The previous measurements (chapter 6.1.1, 6.1.2 and 6.1.3) were performed in mode 2 because switching the GPIOs with the CPU took more than 500 ns.

In future measurements, the GPIOs are to be switched by a hardware timer of the MCU. This will enable switching times of 125 ns at 8 MHz and 20.8 ns at 48 MHz. This chapter will therefore compare the measurement results of the two modes.

Three measurements were made for this purpose:

1. GPIO Toggle without HAL in mode 2 with cable length = 0 m (as in chapter 6.1.2 and 6.1.3)
2. GPIO Toggle without HAL in mode 2 with cable length 6 m (as in chapter 6.1.3)
3. Without GPIO Toggle in mode 1 with cable length 6 m

For the measurement in mode 1, the delay caused by switching the GPIOs should be avoided. To do this, the `START` and `STOP` signals are generated by the same GPIO pin, `start_ele`. To do this, `SW1` is connected to `start_ele` (see figure 14).

The firmware implementation for a measurement in mode 1 is shown in code 8.

```

1 // Configure TDC
2
3 TDC_init(&tdc_ele, &hspi1, SPI_CS_ELE_GPIO_Port, SPI_CS_ELE_Pin,
4         TDC_ELE_ENABLE_GPIO_Port, TDC_ELE_ENABLE_Pin);
5 TDC_enable(&tdc_ele);
6
7 TDC_read(&tdc_ele, TDC_ADR_CONFIG1, data);
8 data[0] |= TDC_CONFIG1_MEAS_MODE_1;
9
10 TDC_write(&tdc_ele, TDC_ADR_CONFIG1, data);
11
12 while (1) {
13     TDC_start(&tdc_ele);
14
15     // Set Pin to High
16     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) |= (1 << 8); // START_ELE

```

```
16     TDC_read_result(&tdc_ele, &tof_fs);
17
18
19     sprintf(string, "ToF = %lu [ps]\n", tof_fs / 1000);
20     HAL_UART_Transmit(&huart2, (uint8_t *)string, strlen(string), 10000);
21
22     // Set Pin to Low (preparation for next iteration)
23     *((volatile uint32_t*)(GPIOA_BASE + 0x14)) &= ~(1 << 8); // START_ELE
24
25     HAL_Delay(10); // 10 ms
26 }
```

Code 8: Mode 1

The differences compared to the measurements in chapter 6.1.2 and 6.1.3 are:

- Line 7: TDC is configured in mode 1 (instead of mode 2)
- Line 15 and 23: Only the `start_ele` pin is toggled (instead of the `start_ele` and `stop_ele` pins)

The expectation is that the difference between measurement 1 and measurements 2 roughly corresponds to the result of measurement 3.

The results of these three measurements are listed in Table 3. The remaining data can be found in the electronic appendix.

Measurement	Mean value	Standard deviation
1	1'378'222 ps	407 ps
2	1'403'224 ps	541 ps
3	25'145 ps	331 ps

Tabelle 3: Mode 1 vs. Mode 2

The calculation in formula 23 shows that the measurement results are close.

$$\begin{aligned}\Delta \text{Mode 2} &= 1'403'224 \text{ ps} - 1'378'222 \text{ ps} = 25'001 \text{ ps} \\ \text{Mode 1} &= 25'145 \text{ ps}\end{aligned}\tag{23}$$

It can be assumed that Mode 1 and Mode 2 have been correctly implemented in the firmware driver.

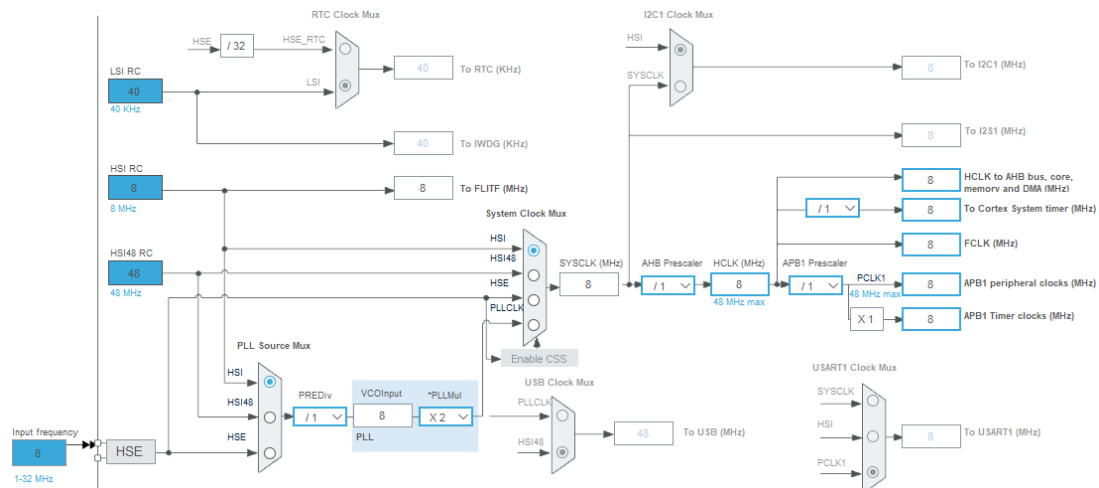
Note: In the firmware driver (see appendix 8.2), the same function can be used for both modes `TDC_read_result()`. For mode 1, the calculation is simplified because `TIME2 = 0` and `CLOCK_COUNT1 = 0`.

6.1.5 Timer Output

In chapters 6.1.1 and 6.1.2 it was shown that it takes several CPU cycles to toggle GPIOs using software, i.e. CPU instructions.

A faster method is to have the GPIOs toggled by the MCU's hardware peripherals. Hardware timers are suitable for this purpose.

Figure 37 shows the default clock configuration of the Nucleo board.



By default, the High Speed Internal (HSI) clock is used at 8 MHz. This should make it possible to use the hardware timer to switch two GPIOs with a delay according to the formula 24.

$$t = \frac{1}{8 \text{ MHz}} = 125 \text{ ns} \quad (24)$$

For this purpose, channels CH1 and CH4 of the hardware timer TIM1 are used in Output Compare Match mode. The associated GPIO pins are `start_ele` and `stop_ele`, respectively.

When the 16-bit timer reaches 32'000, `start_ele` is switched to high. When the value 32'001 is reached, `stop_ele` is switched to high.

The TDC is configured in mode 1. The STOP pin of the TDC is connected to SW1 via `stop_ele`. The cable connection J3 is shorted. See Figure 14.

The firmware implementation for this is shown in Code 9.

```

1 // Configure TDC
2
3 TDC_init(&tdc_ele, &hspi1, SPI_CS_ELE_GPIO_Port, SPI_CS_ELE_Pin,
4         TDC_ELE_ENABLE_GPIO_Port, TDC_ELE_ENABLE_Pin);
5
6 TDC_enable(&tdc_ele);
7
8 TDC_read(&tdc_ele, TDC_ADR_CONFIG1, data);
9 data[0] |= TDC_CONFIG1_MEAS_MODE_1;
10
11 TDC_write(&tdc_ele, TDC_ADR_CONFIG1, data);
12
13 while (1) {
14     TDC_start(&tdc_ele);
15
16     // Start Timer to toggle Pins from Low to High
17     MX_TIM1_Init();
18     HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_1); // START_ELE
19     HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_4); // STOP_ELE
20
21     HAL_Delay(10); // 10 ms
22 }

```

```
21 TDC_read_result(&tdc_ele, &tof_fs);
22
23 sprintf(string, "ToF = %lu [ps]\n", (uint32_t)(tof_fs / 1000));
24 HAL_UART_Transmit(&huart2, (uint8_t*)string, strlen(string), 10000);
25
26 // Start Timer one more time to toggle Pins from High to Low (preparation
27 // for next iteration)
28 MX_TIM1_Init();
29 HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_1); // START_ELE
30 HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_4); // STOP_ELE
31
32 HAL_Delay(10); // 10 ms
33 }
```

Code 9: GPIO Toggle with timer output

The measured arithmetic mean and standard deviation are listed in formula 25. The list of data points can be found in the electronic appendix.

$$\begin{aligned}\overline{ToF} &= 128'108 \text{ ps} \\ \sigma &= 81 \text{ ps}\end{aligned}\tag{25}$$

As expected, switching the GPIOs takes about 125 ns. The standard deviation is lower than in the previous chapters due to the smaller number of clock cycles.

6.1.6 Different Clock Configurations

In this subsection, measurements are performed with the same setup as in chapter 6.1.5.

The arithmetic mean values and standard deviations of different clock sources of the Nucleo board are compared. Three measurements are made:

1. HSI: 8 MHz (as in chapter 6.1.5)
2. HSI with PLL: 16 MHz
3. HSI48: 48 MHz

The clock configurations for measurements 2 and 3 are shown in Figure 38 and 39, respectively.

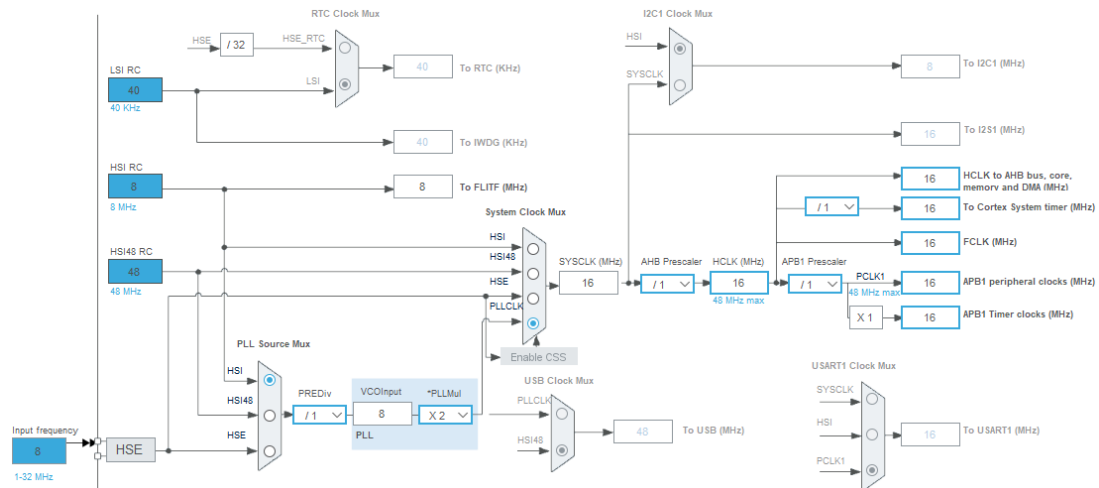


Abbildung 38: Clock configuration HSI with PLL 16 MHz

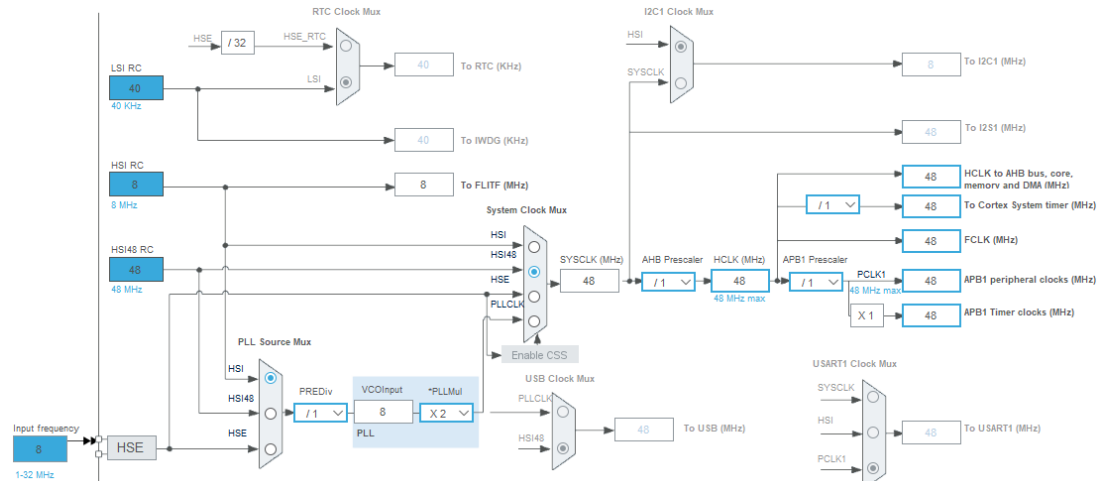


Abbildung 39: Clock configuration HSI48 48 MHz

The arithmetic mean values and standard deviations are shown in Table 4. The list of data points can be found in the electronic appendix.

Measurement	Mean value	Standard deviation
1	128'108 ps	81 ps
2	65'694 ps	83 ps
3	24'275 ps	42 ps

Tabelle 4: Different clock sources

The results roughly match the expected values from formula 26.

$$\begin{aligned}
 t_1 &= \frac{1}{8 \text{ MHz}} = 125 \text{ ns} \\
 t_2 &= \frac{1}{16 \text{ MHz}} = 62.5 \text{ ns} \\
 t_3 &= \frac{1}{48 \text{ MHz}} = 20.8 \text{ ns}
 \end{aligned}
 \tag{26}$$

The data sheet contains information about the PLL jitter, see figure 40.

Table 42. PLL characteristics

Symbol	Parameter	Value			Unit
		Min	Typ	Max	
$f_{\text{PLL_IN}}$	PLL input clock ⁽¹⁾	1 ⁽²⁾	8.0	24 ⁽²⁾	MHz
	PLL input clock duty cycle	40 ⁽²⁾	-	60 ⁽²⁾	%
$f_{\text{PLL_OUT}}$	PLL multiplier output clock	16 ⁽²⁾	-	48	MHz
t_{LOCK}	PLL lock time	-	-	200 ⁽²⁾	μs
Jitter _{PLL}	Cycle-to-cycle jitter	-	-	300 ⁽²⁾	ps

1. Take care to use the appropriate multiplier factors to obtain PLL input clock values compatible with the range defined by $f_{\text{PLL_OUT}}$.

2. Guaranteed by design, not tested in production.

Abbildung 40: PLL-Jitter (ST, 2017)

This information is approximately in the same order of magnitude as the measurement results.

6.1.7 Different cable lengths with DSO

In this subchapter, the same cable lengths as in chapter 6.1.3 are measured again.

The following firmware configuration is used to reduce the jitter:

- TDC: Mode 1
- Clock: 48 MHz
- Switching the GPIOs: Timer Output

In addition, the different cable lengths are also measured simultaneously via DSO (Rohde & Schwarz RTB2004 2.5 GSa/s.) in order to compare the results of the TDC with them.

Figure 41 shows the recorded measurement results of the TDC. While the measurement values were being recorded, the DSO was connected to the pins. The list of data points can be found in the electronic appendix.

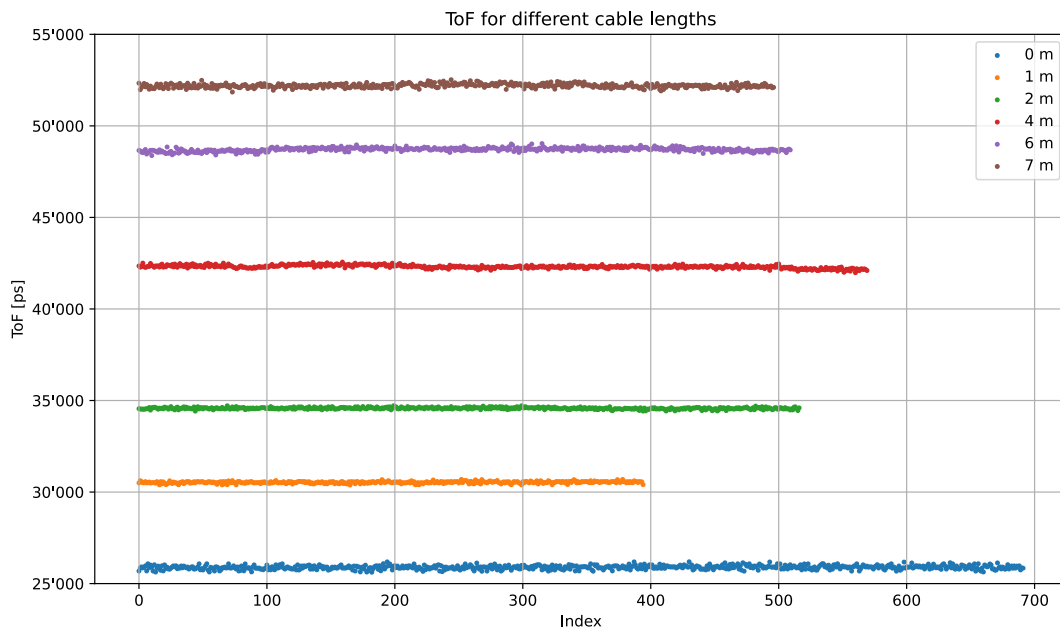


Abbildung 41: Different cable lengths - measurement values TDC

Table 5 lists the mean values and standard deviations of the measured results of the TDC.

Length	Mean value	Standard deviation	Δ at 0 m
0 m	25'890 ps	104 ps	0 ns
1 m	30'531 ps	53 ps	4.6 ns
2 m	34'575 ps	50 ps	8.7 ns
4 m	42'308 ps	88 ps	16.4 ns
6 m	48'710 ps	98 ps	22.8 ns
7 m	52'194 ps	104 ps	26.3 ns

Tabelle 5: Different cable lengths - results TDC

The measurement results recorded by the DSO are listed in Table 6. The last column shows the difference between DSO and TDC. It can be seen that the difference is very small.

Length	Value	Δ at 0 m	Δ at TDC
0 m	1.5 ns	0 ns	0.0 ns
1 m	6.2 ns	4.7 ns	+0.1 ns
2 m	10.2 ns	8.7 ns	0.0 ns
4 m	18.1 ns	16.6 ns	+0.2 ns
6 m	24.7 ns	23.2 ns	+0.4 ns
7 m	28.1 ns	26.6 ns	+0.3 ns

Tabelle 6: Different cable lengths - results DSO

Table 7 shows the theoretically expected propagation times for the different assumptions for the signal propagation velocities of $\frac{2}{3}c$ and c .

Length	Theory $\frac{2}{3} c$	Theory c
0 m	0.0 ns	0 ns
1 m	5.0 ns	3.3 ns
2 m	10.0 ns	6.7 ns
4 m	20.0 ns	13.3 ns
6 m	30.0 ns	20.0 ns
7 m	35.0 ns	23.3 ns

Tabelle 7: Different cable lengths - theoretical expectation

The differences between the TDC results and the theoretical expectations are shown in the tables 8 and 9. First, the difference in runtime is shown, then the recalculated cable length.

Length	Δ TDC to theory $\frac{2}{3} c$	Δ TDC theory to c
0 m	0.0 ns	0.0 ns
1 m	-0.4 ns	+1.3 ns
2 m	-1.3 ns	+2.0 ns
4 m	-3.6 ns	+3.1 ns
6 m	-7.2 ns	+2.8 ns
7 m	-8.7 ns	+3.0 ns

Tabelle 8: Different cable lengths - TDCns difference compared to theory

Length	TDC acc. to theory $\frac{2}{3} c$	TDC acc. to theory c
0 m	0.0 m	0.0 m
1 m	0.9 m	1.4 m
2 m	1.7 m	2.6 m
4 m	3.3 m	4.9 m
6 m	4.6 m	6.8 m
7 m	5.3 m	7.9 m

Tabelle 9: Different cable lengths - TDC [m] according to theory

Table 9 shows that neither of the two theoretical values for c lead to a perfect TDC result for the recalculated distance [m].

In Table 8, we see that the difference between the TDC results compared to the theoretically expected runtime at c is slightly smaller than at $\frac{2}{3} c$. It seems that c is a slightly better approximation.

If the measured difference can actually be explained by an incorrect assumption of the propagation speed, then the propagation speed in the selected cable would have to be approximately $0.9 c$. The recalculated results are listed in Table 10.

Length	TDC acc. to theory 0.9 c
0 m	0.0 m
1 m	1.2 m
2 m	2.3 m
4 m	4.4 m
6 m	6.2 m
7 m	7.1 m

Tabelle 10: Different cable lengths - TDC [m] according to the 0.9 c theory

The figures 42 and 43 show the DSO records for cable lengths of 1 m and 6 m are shown. The remaining DSO records can be found in the electronic appendix.



Abbildung 42: DSO recording with a cable length of 1 m



Abbildung 43: DSO recording with a cable length of 6 m

It can be seen that the slope of the STOP signal flattens out a little with longer cable lengths. This has an influence on the runtime measurement. Steeper edges would help to minimize this influence. More about this in chapter 7.1.

6.2 Optical Measurements

This subsection documents the measurement results that relate to the optical path of the demonstrator.

The DSO used is a Rohde & Schwarz RTB2004 2.5 GSa/s.

The following firmware configurations are used for the measurements in this chapter:

- TDC: Mode 1
- Clock: 48 MHz
- Switching the GPIOs: Timer Output

6.2.1 Signal Path Transmitter

In this subsection, the signal path of the optical transmitter is measured.

The delay times between the MCU signals `start_op` and `LD_pulse` as well as the potential at the gate of the NexFET (v1 pin 4 in Figure 18) are shown in Figure 44



Abbildung 44: Measurement of the signal path sender: `start_op`, `LD_pulse`, FET gate

The measured delay time of 20.4 ns (20.8 ns corresponds to one clock cycle at 48 MHz) between `start_op` and `LD_pulse` was deliberately inserted to ensure that the minimum measurement duration of 12 ns (TI, 2016b) in mode 1 is always guaranteed.

The measured delay time of 8.4 ns between LD_pulse and the FET gate (this corresponds to the output of the gate driver IC22) is small and shows a steep edge.

Figure 45 shows the current through the laser diode (LD1 in Figure 18).



Abbildung 45: Measurement signal path sender: Current through laser diode

The current was calculated using the formula 27.

$$I_{LD} = \frac{V(\text{TP8}) - V(\text{TP9})}{R_V} = \frac{V(\text{TP8}) - V(\text{TP9})}{5 \Omega} \quad (27)$$

To make the delay time visible, the signal LD_pulse and the potential at the FET gate were also recorded, as in Figure 44.

The measured current during transmission is approximately 174 mA.

This is considerably less than the expected current of 440 mA from formula 12. This is due to the increased diode voltage. During transmission, the voltage across the laser diode increases to approximately 4 V, while the voltage across the drain-source of the FET is negligible. This results in the current through the diode according to the formula 28.

$$I_{LD} = \frac{5 \text{ V} - V_{LD} - V_{DS}}{R_V} \approx \frac{5 \text{ V} - 4 \text{ V}}{5 \Omega} \approx 200 \text{ mA} \quad (28)$$

6.2.2 ToF measurements via mirror

To ensure that the photodiode receives the strongest possible signal, a mirror was used instead of a wall for these measurements. The setup is shown in Figure 46.

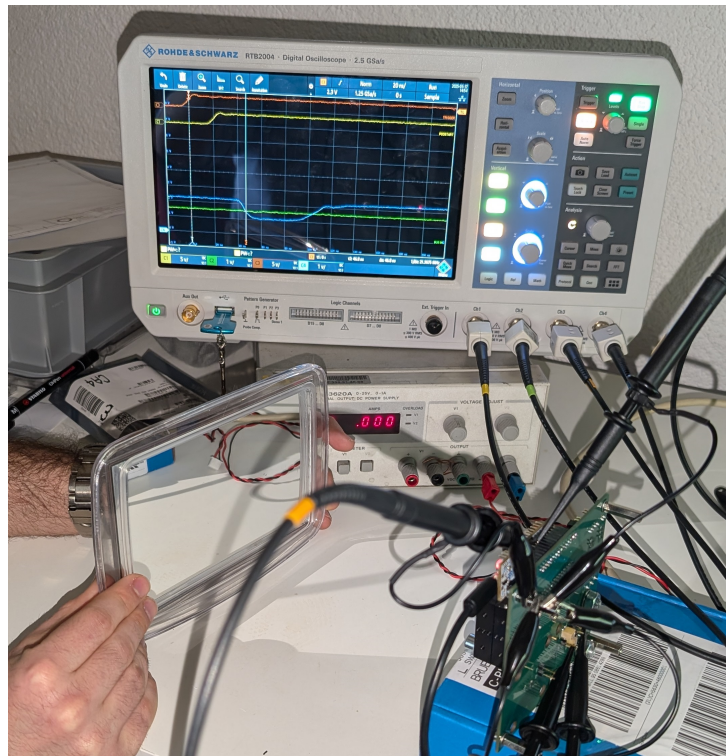


Abbildung 46: Measurement setup with mirror

According to the data sheet, the laser diode has a non-ideal point-shaped radiation pattern, see Figure 47.

Beam divergence	$\theta_{//}$	$P_o=7\text{mW}$	7	9	13	deg.
	$\theta_{\perp}$		20	27	35	deg.

Abbildung 47: Laser beam pattern (ROHM, 2019)

It has been noticed that with the chosen PCB layout, the orientation of the laser diode for a mirror measurement. The wider beam angle is perpendicular to the line between the laser and photodiode. See Figure 48.

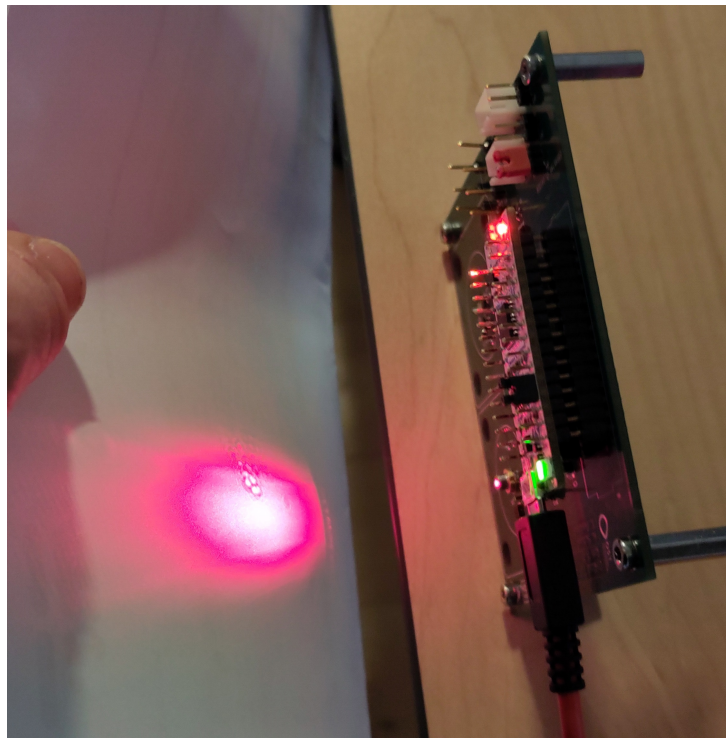


Abbildung 48: Laser alignment

The laser diode was therefore manually rotated by 90°, as shown in Figure 49.

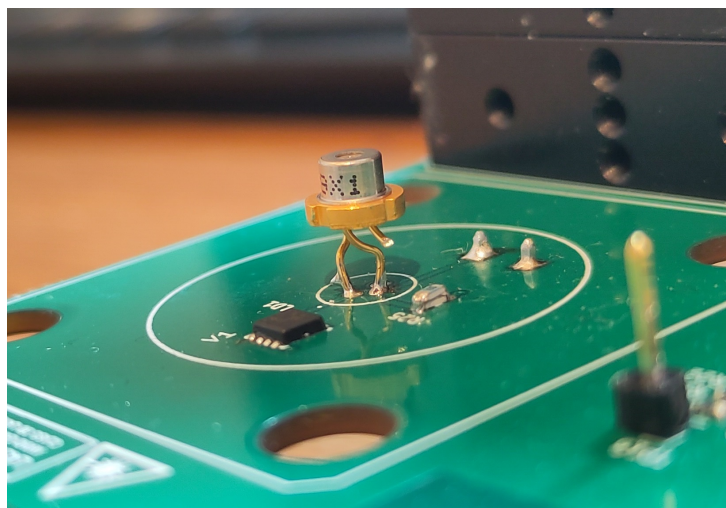


Abbildung 49: Laser alignment adjusted

The DSO diagram of an initial mirror measurement at a distance of approx. 20 cm from the mirror is shown in Figure 50. It shows the output of the transimpedance amplifier. In addition, to see the delay time, the MCU signals `start_op` and `LD_pulse` as well as the voltage potential at the gate of the FET.



Abbildung 50: Initial mirror measurement with DSO

The demonstrator was used to record measurements for three different distances: 19 cm, 24 cm and 28.5 cm.

The data points recorded with the optical TDC are shown in Figure 51. Approximately 300 measurements were recorded for each distance, the list with the data points can be found in the electronic appendix.

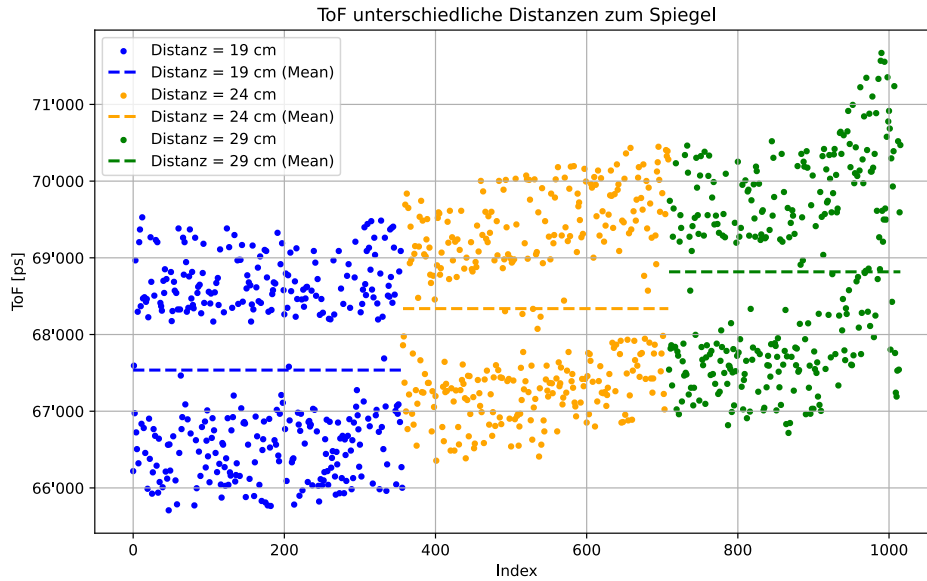


Abbildung 51: Different distances to the mirror - plot

The arithmetic mean values and the standard deviations are listed in Table 11.

Distance to mirror	Mean value	Standard deviation
19 cm	67'536 ps	1'172 ps
24 cm	68'338 ps	1'191 ps
29 cm	68'817 ps	1'273 ps

Tabelle 11: Different distances to the mirror - statistical quantities

It is noticeable that the standard deviations are relatively large. This has to do in particular with the two clusters which can be seen in Figure 51. The cause of these clusters is unclear. One hypothesis is that they are related to the DSO. It was found that connecting the has a significant influence on the offset of the TDC runtime values (approx. 10 ns). Possibly the DSO has a time-dependent influence on the measurement results due to the sample and hold – depending on whether the light pulse hits the photodiode during the sample or hold phase. The cause of these clusters should be analyzed in more detail in a possible follow-up work.

A change in the mirror distance of 5 cm corresponds to a length change of 10 cm.

The measurements show an average change of about 0.6 ns for this length change of 10 cm.

In formula 29, the theoretically expected runtime difference is calculated.

$$\Delta ToF = \frac{\Delta L}{c} = \frac{10 \text{ cm}}{3 \cdot 10^8 \text{ m/s}} = 0.33 \text{ ns} \quad (29)$$

This difference (approx. factor of 2) could have several reasons:

- 1.) One possible explanation is that the edge becomes less steep with increasing distance, which causes a slightly longer runtime to be measured.

2.) Another thing that stands out in Figure 51 is a time drift during the recording of the measurement data at 24 cm and 29 cm. Figure 52 is intended to clarify this.

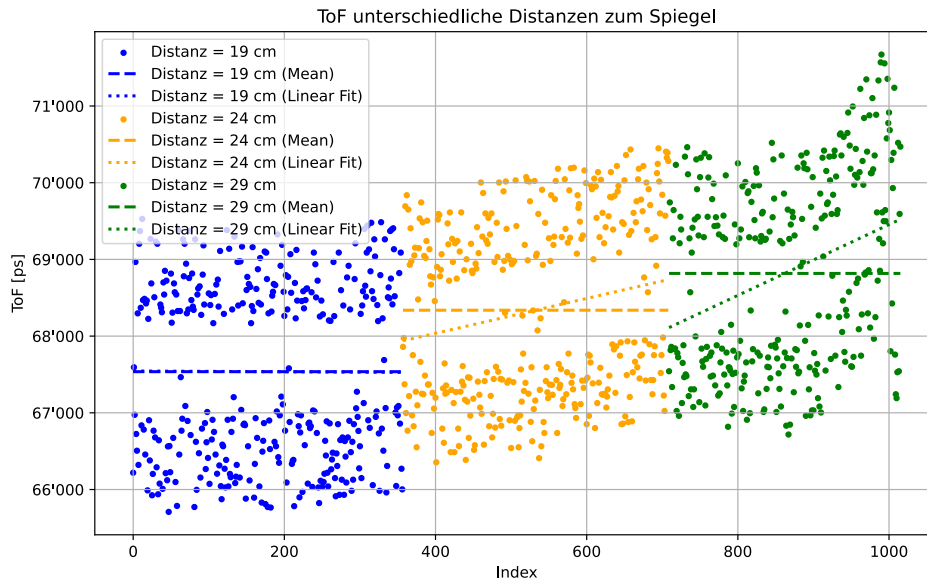


Abbildung 52: Different distances to the mirror - plot with linear interpolation

However, this time drift cannot conclusively explain the measured difference.

Note: The mirror had to be aligned very precisely and then kept very stable in order to obtain results with relatively small standard deviation (about 1 ns).

At distances of more than 30 cm, the TDC signal becomes unusable.

This is because the photodiode then receives only a small amount of light and, accordingly, generates a small photocurrent. This leads to a smaller modulation at the output of the transimpedance amplifier. Thus, the threshold of the comparator is no longer or only just reached.

Figures 53 and 54 are intended to illustrate this.



Abbildung 53: TIA output and comparator threshold at 12 cm



Abbildung 54: TIA output and comparator switching threshold at 30 cm

The figures show, in addition to the MCU signals `start_op` and `LD_pulse`, the output of the transimpedance amplifier and the comparator switching threshold.

Possible improvements are listed in chapter 7.1.

6.2.3 ToF measurements via wall

Instead of the mirror used in chapter ??, this chapter shows measurement results with a white wall as a target. Distances of about 10 to 30 cm were tried.

No useful measurement results could be obtained with this setup in the short time available.

Possible improvements are listed in chapter 7.1.

7 Conclusion

This chapter collects final thoughts. It is intended to provide an outlook for possible further work. It also lists some points that went well, but also those where potential for improvement was identified.

7.1 Outlook

In principle, it can be confirmed that a ToF system, as imagined by the students, can be developed and operated. In the following, a few possible improvements and further measurements are listed.

1. The optics should be better aligned. On the one hand, this would allow measurements to be taken over longer distances measurements over longer distances. Furthermore, it should also be possible to detect targets with non-reflective or weakly reflective surface properties.
2. For measurements with the mirror, it would make sense to try a lens with a longer focal length (or the same lens at a slightly shorter distance to the PCB). This would have the advantage that the mirror would not have to be aligned quite as precisely. The disadvantage would be a slightly lower photocurrent with parallel incident light.
3. There is room for improvement in the control of the TDCs. For example, a faster clocked MCU could minimize the jitter at `START` and `STOP` signals, which would lead to measurement results with even less dispersion.
4. Another way to minimize dispersion would be to use an external clock source.
5. It may be possible to do without the artificially inserted delay of one clock cycle. This would be the case if there is already enough delay in the measurement path to exceed the minimum measurement duration of 12 ns. If not, the artificial delay could also be implemented by means of a separate circuit (with the lowest possible jitter) instead of the MCU separate circuit (with the lowest possible jitter).
6. The Nucleo used offers the option of configuring the rise time of the GPIO. With a faster rise time, it may be possible to improve the measurement results.
7. Optical receiving path: Measurements should be carried out with different gains of the TIA to increase the distance with a mirror and to enable measurements against the wall or other targets. In this context, the switching threshold of the comparator should be tested more accurately and fine-tuned.
8. Optical transmission path: It is worth experimenting with higher transmission power. Due to the 5 V supply of the laser diode, we are almost at the limit with the 5 Ω series resistor. In a next PCB version, it would be worthwhile to provide the option of a 12 V supply with a switch, analogous to how this is already implemented for the photodiode.
9. Optical measurements: It is worth analyzing why the measurement results are distributed in two clusters (approx. 2 ns apart).
10. For further measurements with a mirror, it is worth constructing a precise mechanical test setup. This would allow the mirror to be better fixed and adjusted. It is difficult to achieve reproducible results by hand.

In conclusion, it is possible to perform a time-of-flight measurement with a TDC7200, whereby the temporal resolution of the measurement itself is the limiting factor.

7.2 Lessons Learned

As the measurement results and the outlook already suggest, a little more time could have been invested primarily in designing the optoelectronics. The initial rough estimates and calculations regarding the desired limitations of the system should have been scrutinized more critically. Initially, the students' goal was to take measurements at distances of up to 10 cm with a resolution of about 10 cm. However, it turned out in the end that it was not the temporal resolution but rather the distance and the strength of the measurement signal that was problematic.

The student team is satisfied with the fabrication of the ToF demonstrator. When using such small SMD components, such as in the TIA, it is worth considering whether machine assembly is preferable to manual assembly. Manual assembly probably saved hardly any time, since it also took several hours, spread over several evenings.

7.3 Acknowledgements

We would like to take this opportunity to express our sincere thanks to the two lecturers, Prof. Guido Keel and Michael Lehmann. During the project, they were always on hand to help us with advice and support and followed the project with great interest. We would also like to thank them for the optical equipment loaned to us. This enabled us to achieve a partial success in the optical measurement.

We would also like to thank Robin Burkard, who carried out a layout review for us. Thanks to his valuable tips, the PCB could be put into operation with as few glitches as possible.

8 Appendix

8.1 Outline

Projektarbeit: Distanzmessung mit TDC

Outline

Modul: CAS Sensorik und Sensor Signal Conditioning
Institution: OST – Ostschweizer Fachhochschule
Autoren: Matthias Schär, Timon Burkard

Wir möchten in einer ersten Phase den TDC7200 kennen lernen und austesten. In einer zweiten Phase soll ein optischer Pfad dazu kommen, Ziel ist es eine Distanzmessung mittels Time of Flight (ToF) durchzuführen.

Teil 1: Elektrisch

Um den TDC7200 elektrisch kennen zu lernen, sollen die folgenden Punkte getestet werden:

1a: IO ein- und ausschalten

Ersten Pin schalten zum START, dann einen zweiten Pin zum STOP. Mit verschiedenen Varianten; z.B. mit CPU oder Timer – Jitter ausmessen und beste Variante finden.

1b: Elektrische Laufzeitmessung

z.B. mit unterschiedlich langen Leitungen auf PCB und externe Anschlüsse für längere Kabel.

Ziel: Länge eines angeschlossenen Kabels bestimmen. Wie nahe kommen wir an das theoretische Minimum von 15mm?

Teil 2: Optisch

Schaltung wird mit optischen Komponenten ergänzt: Laserdiode und Photodiode/-transistor. Dazu braucht es Verstärkerschaltungen mit OP.

Wie können wir die Laufzeit des Verstärkersignal bestimmen? – z.B. optischen Teil überbrücken und vom Ende des Sende-Verstärkers direkt zum Anfang des Empfangsverstärkers.

Kalibrierung mit fixer Distanz. Wie gross ist die Standardabweichung?

Was passiert bei unterschiedlichen Oberflächen? – Kann man die Stärke des Messsignals kompensieren? – Evtl. führt z.B. ein stärkeres Messsignal zu einer steileren Flanke bei Signal der Photodiode?

Wie nahe kommen wir nun an das theoretische Minimum von 15mm?

PCB

Für Teil 1 möchten wir ein eigenentwickeltes PCB verwenden. Um später nicht ein zweites PCB bestellen zu müssen, sollte bereits dann klar sein, wie das PCB aufgebaut sein muss. um auch den Teil 2 damit erledigen zu können.

Ziel ist es das PCB vor Jahresende zu bestellen, sodass dieses Anfangs Januar geliefert wird.

Dazu möchten wir gerne in KW51 vom Dozenten ein Feedback zum PCB einholen.

Abbildung 55: Outline

8.2 TDC Driver

The code in 10 and 11 shows the firmware driver developed in-house for the TDC.

```
1  /*
2   * TDC.h
3   *
4   * Driver for TI TDC7200 -- Header file
5   *
6   * (C) T. Burkard | M. Schaer
7   *
8   */
9
10 #ifndef TDC_H_
11 #define TDC_H_
12
13 #include <stdint.h>
14 #include "stm32f0xx_hal.h"
15
16 #define TDC_CLOCK_PERIOD_FS 62500000 // 16 MHz in [fs]
17
18 /**
19  * Convert 24bit buffer received from SPI to integer
20  */
21 #define TDC_24BIT_BUF_TO_INT(buf) ((buf[0] << 16) + (buf[1] << 8) + buf[2])
22
23 /**
24  * @brief Register addresses of the TDC
25  *
26  */
27 typedef enum {
28     TDC_ADR_CONFIG1           = 0x00, // Configuration Register 1
29     TDC_ADR_CONFIG2           = 0x01, // Configuration Register 2
30     TDC_ADR_INT_STATUS        = 0x02, // Interrupt Status Register
31     TDC_ADR_INT_MASK          = 0x03, // Interrupt Mask Register
32     TDC_ADR_COARSE_CNTR_OVF_H = 0x04, // Coarse Counter Overflow Value High
33     TDC_ADR_COARSE_CNTR_OVF_L = 0x05, // Coarse Counter Overflow Value Low
34     TDC_ADR_CLOCK_CNTR_OVF_H  = 0x06, // CLOCK Counter Overflow Value High
35     TDC_ADR_CLOCK_CNTR_OVF_L  = 0x07, // CLOCK Counter Overflow Value Low
36     TDC_ADR_CLOCK_CNTR_STOP_MASK_H = 0x08, // CLOCK Counter STOP Mask High
37     TDC_ADR_CLOCK_CNTR_STOP_MASK_L = 0x09, // CLOCK Counter STOP Mask Low
38     TDC_ADR_TIME1              = 0x10, // Measured Time 1
39     TDC_ADR_CLOCK_COUNT1       = 0x11, // CLOCK Counter Value
40     TDC_ADR_TIME2              = 0x12, // Measured Time 2
41     TDC_ADR_CLOCK_COUNT2       = 0x13, // CLOCK Counter Value
42     TDC_ADR_TIME3              = 0x14, // Measured Time 3
43     TDC_ADR_CLOCK_COUNT3       = 0x15, // CLOCK Counter Value
44     TDC_ADR_TIME4              = 0x16, // Measured Time 4
45     TDC_ADR_CLOCK_COUNT4       = 0x17, // CLOCK Counter Value
46     TDC_ADR_TIME5              = 0x18, // Measured Time 5
47     TDC_ADR_CLOCK_COUNT5       = 0x19, // CLOCK Counter Value
48     TDC_ADR_TIME6              = 0x1A, // Measured Time 6
49     TDC_ADR_CALIBRATION1       = 0x1B, // Calibration 1, 1 CLOCK Period
50     TDC_ADR_CALIBRATION2       = 0x1C, // Calibration 2, 2/10/20/40 CLOCK
51     Periods
52     TDC_ADR_AMOUNT // Amount of registers
53 } TDC_adr_t;
54
55 /**
56  * @brief Register sizes of the TDC
57  *
```

```
57 * Index = Register address (TDC_adr_t)
58 * Value = Register size (in bytes)
59 *
60 */
61 extern const uint8_t TDC_REG_SIZE[TDC_ADR_AMOUNT];
62
63 /**
64 * @brief Bit description of the TDC CONFIG1 register
65 *
66 */
67 typedef enum {
68     TDC_CONFIG1_FORCE_CAL_OFF    = (0b0 << 7), // Calibration is not performed
        after interrupted measurement (for example, due to counter overflow or
        missing STOP signal)
69     TDC_CONFIG1_FORCE_CAL_ON     = (0b1 << 7), // Calibration is always
        performed at the end (for example, after a counter overflow)
70     TDC_CONFIG1_PARITY_EN_OFF    = (0b0 << 6), // Parity bit for Measurement
        Result Registers disabled (Parity Bit always 0)
71     TDC_CONFIG1_PARITY_EN_ON     = (0b1 << 6), // Parity bit for Measurement
        Result Registers enabled (Even Parity)
72     TDC_CONFIG1_TRIGG_EDGE_RISE  = (0b0 << 5), // TRIGG is output as a Rising
        edge signal
73     TDC_CONFIG1_TRIGG_EDGE_FALL  = (0b1 << 5), // TRIGG is output as a Falling
        edge signal
74     TDC_CONFIG1_STOP_EDGE_RISE   = (0b0 << 4), // Measurement is stopped on
        Rising edge of STOP signal
75     TDC_CONFIG1_STOP_EDGE_FALL   = (0b1 << 4), // Measurement is stopped on
        Falling edge of STOP signal
76     TDC_CONFIG1_START_EDGE_RISE  = (0b0 << 3), // Measurement is started on
        Rising edge of START signal
77     TDC_CONFIG1_START_EDGE_FALL  = (0b1 << 3), // Measurement is started on
        Falling edge of START signal
78     TDC_CONFIG1_MEAS_MODE_1      = (0b00 << 1), // Measurement Mode 1 (for
        expected time-of-flight < 500 ns).
79     TDC_CONFIG1_MEAS_MODE_2      = (0b01 << 1), // Measurement Mode 2
        (recommended)
80     TDC_CONFIG1_START_MEAS       = (0b1 << 0), // Start New Measurement.
        Writing a 1 will clear all bits in the Interrupt Status Register and Start
        the measurement (by generating an TRIGG signal) and will reset the content
        of all Measurement Results registers
81 } TDC_config1_t;
82
83 /**
84 * @brief Bit description of the TDC CONFIG2 register
85 *
86 */
87 typedef enum {
88     TDC_CONFIG2_CALIBRATION2_PERIODS_2 = (0b00 << 6), // Measuring 2 CLOCK
        periods
89     TDC_CONFIG2_CALIBRATION2_PERIODS_10 = (0b01 << 6), // Measuring 10 CLOCK
        periods
90     TDC_CONFIG2_CALIBRATION2_PERIODS_20 = (0b10 << 6), // Measuring 20 CLOCK
        periods
91     TDC_CONFIG2_CALIBRATION2_PERIODS_40 = (0b11 << 6), // Measuring 40 CLOCK
        periods
92     TDC_CONFIG2_AVG_CYCLES_1          = (0b000 << 3), // 1 Measurement Cycle
        only (no Multi-Cycle Averaging Mode)
93     TDC_CONFIG2_AVG_CYCLES_2          = (0b001 << 3), // 2 Measurement Cycles
94     TDC_CONFIG2_AVG_CYCLES_4          = (0b010 << 3), // 4 Measurement Cycles
95     TDC_CONFIG2_AVG_CYCLES_8          = (0b011 << 3), // 8 Measurement Cycles
96     TDC_CONFIG2_AVG_CYCLES_16         = (0b100 << 3), // 16 Measurement Cycles
```

```

97     TDC_CONFIG2_AVG_CYCLES_32      = (0b101 << 3), // 32 Measurement Cycles
98     TDC_CONFIG2_AVG_CYCLES_64      = (0b110 << 3), // 64 Measurement Cycles
99     TDC_CONFIG2_AVG_CYCLES_128     = (0b111 << 3), // 128 Measurement
    Cycles
100     TDC_CONFIG2_NUM_STOP_1         = (0b000 << 0), // Single Stop
101     TDC_CONFIG2_NUM_STOP_2         = (0b001 << 0), // Two Stops
102     TDC_CONFIG2_NUM_STOP_3         = (0b010 << 0), // Three Stops
103     TDC_CONFIG2_NUM_STOP_4         = (0b011 << 0), // Four Stops
104     TDC_CONFIG2_NUM_STOP_5         = (0b100 << 0), // Five Stops
105 } TDC_config2_t;
106
107 /**
108  * @brief Bit description of the TDC INT_STATUS register
109  *
110  */
111 typedef enum {
112     TDC_STATUS_MEAS_COMPLETE_FLAG = (1 << 4), // Measurement has completed
113     TDC_STATUS_MEAS_STARTED_FLAG  = (1 << 3), // Measurement has started
    (START signal received)
114     TDC_STATUS_CLOCK_CNTR_OVF_INT = (1 << 2), // Clock overflow detected,
    running measurement will be stopped immediately
115     TDC_STATUS_COARSE_CNTR_OVF_INT = (1 << 1), // Coarse overflow detected,
    running measurement will be stopped immediately
116     TDC_STATUS_NEW_MEAS_INT       = (1 << 0), // Interrupt detected - New
    Measurement has been completed
117 } TDC_status_t;
118
119 /**
120  * @brief Bit description of the TDC INT_MASK register
121  *
122  */
123 typedef enum {
124     TDC_MASK_CLOCK_CNTR_OVF_MASK_OFF = (0 << 2), // CLOCK Counter Overflow
    Interrupt disabled
125     TDC_MASK_CLOCK_CNTR_OVF_MASK_ON  = (1 << 2), // CLOCK Counter Overflow
    Interrupt enabled
126     TDC_MASK_COARSE_CNTR_OVF_MASK_OFF = (0 << 1), // Coarse Counter Overflow
    Interrupt disabled
127     TDC_MASK_COARSE_CNTR_OVF_MASK_ON  = (1 << 1), // Coarse Counter Overflow
    Interrupt enabled
128     TDC_MASK_NEW_MEAS_MASK_OFF       = (0 << 0), // New Measurement Interrupt
    disabled
129     TDC_MASK_NEW_MEAS_MASK_ON        = (1 << 0), // New Measurement Interrupt
    enabled
130 } TDC_mask_t;
131
132 /**
133  * @brief Error codes of the TDC module
134  *
135  */
136 typedef enum {
137     TDC_OK = 0,
138     TDC_ERROR,
139     TDC_WRONG_ADDRESS,
140     TDC_WRONG_SIZE,
141     TDC_COM_ERROR,
142 } TDC_error_t;
143
144 /**
145  * @brief Handle for a TDC instance
146  */

```

```
147 */
148 typedef struct {
149     SPI_HandleTypeDef* spi;
150     GPIO_TypeDef* cs_port;
151     uint16_t cs_pin;
152     GPIO_TypeDef* en_port;
153     uint16_t en_pin;
154 } TDC_t;
155
156 /**
157  * @brief Initialize a TDC instance
158  *
159  * Note, SPI and GPIOs need to be initialized manually before calling this
160  * function.
161  *
162  * @param[out] tdc -- Pointer to TDC handle
163  * @param[in] spi -- Pointer to SPI handle
164  * @param[in] cs_port -- GPIO port of Chip Select
165  * @param[in] cs_pin -- GPIO pin of Chip Select
166  * @param[in] en_port -- GPIO port of Enable
167  * @param[in] en_pin -- GPIO pin of Enable
168  *
169  * @return TDC_error -- Error code
170  */
171 TDC_error_t TDC_init(TDC_t* tdc, SPI_HandleTypeDef* spi, GPIO_TypeDef* cs_port,
172     uint16_t cs_pin, GPIO_TypeDef* en_port, uint16_t en_pin);
173
174 /**
175  * @brief Enable the TDC
176  *
177  * @param[in] tdc -- Pointer to TDC handle
178  *
179  * @return TDC_error_t -- Error code
180  */
181 TDC_error_t TDC_enable(TDC_t* tdc);
182
183 /**
184  * @brief Disable the TDC
185  *
186  * @param[in] tdc -- Pointer to TDC handle
187  *
188  * @return TDC_error_t -- Error code
189  */
190 TDC_error_t TDC_disable(TDC_t* tdc);
191
192 /**
193  * @brief Read register data from TDC
194  *
195  * @param[in] tdc -- Pointer to TDC handle
196  * @param[in] address -- Address of the register(s) to read
197  * @param[out] rx_data -- Pointer to data
198  *
199  * @attention @p rx_data needs to provide space for at least TDC_REG_SIZE[ @p
200  * address ] bytes.
201  *
202  * @return TDC_error_t -- Error code
203  */
204 TDC_error_t TDC_read(TDC_t* tdc, TDC_adr_t address, uint8_t* rx_data);
205
206 /**
207  * @brief Write register data to TDC
208  *
209  * @param[in] tdc -- Pointer to TDC handle
210  * @param[in] address -- Address of the register(s) to write
211  * @param[in] tx_data -- Pointer to data
212  *
213  * @return TDC_error_t -- Error code
214  */
215 TDC_error_t TDC_write(TDC_t* tdc, TDC_adr_t address, uint8_t* tx_data);
```

```

205 *
206 * @param[in] tdc      -- Pointer to TDC handle
207 * @param[in] address -- Address of the register(s) to write
208 * @param[in] tx_data  -- Pointer to data
209 *
210 * @attention @p tx_data needs to provide space for at least TDC_REG_SIZE[ @p
211 *           address ] bytes.
212 *
213 * @return TDC_error_t -- Error code
214 */
215 TDC_error_t TDC_write(TDC_t* tdc, TDC_adr_t address, uint8_t* tx_data);
216
217 /**
218 * @brief Start measurement of TDC
219 *
220 * @param[in] tdc -- Pointer to TDC handle
221 *
222 * @return TDC_error_t -- Error code
223 */
224 TDC_error_t TDC_start(TDC_t* tdc);
225
226 /**
227 * @brief Read ToF measurement result from TDC
228 *
229 * @param[in] tdc    -- Pointer to TDC handle
230 * @param[out] tof_fs -- ToF Measurement result [fs]
231 *
232 * @return TDC_error_t -- Error code
233 */
234 TDC_error_t TDC_read_result(TDC_t* tdc, uint64_t* tof_fs);
235 #endif /* TDC_H_ */

```

Code 10: TDC Driver (Header)

```

1 /*
2  * TDC.c
3  *
4  * Driver for TI TDC7200 -- Source file
5  *
6  * (C) T. Burkard | M. Schaer
7  *
8  */
9
10 #include "TDC/TDC.h"
11 #include <stdbool.h>
12
13 #define TDC_AUTO_INC_OFF (0 << 7)
14 #define TDC_AUTO_INC_ON  (1 << 7)
15
16 #define TDC_RW_READ  (0 << 6)
17 #define TDC_RW_WRITE (1 << 6)
18
19 #define TDC_CMD_READ  (TDC_AUTO_INC_ON | TDC_RW_READ)
20 #define TDC_CMD_WRITE (TDC_AUTO_INC_ON | TDC_RW_WRITE)
21
22 const uint8_t TDC_REG_SIZE[TDC_ADR_AMOUNT] = {
23     // Register Address      Register Size [bytes]
24     [TDC_ADR_CONFIG1]      = 1,
25     [TDC_ADR_CONFIG2]      = 1,
26     [TDC_ADR_INT_STATUS]    = 1,

```

```

27     [TDC_ADR_INT_MASK]           = 1,
28     [TDC_ADR_COARSE_CNTR_OVF_H]  = 1,
29     [TDC_ADR_COARSE_CNTR_OVF_L]  = 1,
30     [TDC_ADR_CLOCK_CNTR_OVF_H]   = 1,
31     [TDC_ADR_CLOCK_CNTR_OVF_L]   = 1,
32     [TDC_ADR_CLOCK_CNTR_STOP_MASK_H] = 1,
33     [TDC_ADR_CLOCK_CNTR_STOP_MASK_L] = 1,
34     [TDC_ADR_TIME1]              = 3,
35     [TDC_ADR_CLOCK_COUNT1]       = 3,
36     [TDC_ADR_TIME2]              = 3,
37     [TDC_ADR_CLOCK_COUNT2]       = 3,
38     [TDC_ADR_TIME3]              = 3,
39     [TDC_ADR_CLOCK_COUNT3]       = 3,
40     [TDC_ADR_TIME4]              = 3,
41     [TDC_ADR_CLOCK_COUNT4]       = 3,
42     [TDC_ADR_TIME5]              = 3,
43     [TDC_ADR_CLOCK_COUNT5]       = 3,
44     [TDC_ADR_TIME6]              = 3,
45     [TDC_ADR_CALIBRATION1]       = 3,
46     [TDC_ADR_CALIBRATION2]       = 3,
47 };
48
49 static TDC_error_t send(TDC_t* tdc, TDC_adr_t address, uint8_t* data, bool
    write);
50
51 TDC_error_t TDC_init(TDC_t* tdc, SPI_HandleTypeDef* spi, GPIO_TypeDef* cs_port,
    uint16_t cs_pin, GPIO_TypeDef* en_port, uint16_t en_pin)
52 {
53     if ((tdc == NULL) || (spi == NULL) || (cs_port == NULL) || (en_port ==
    NULL)) {
54         return TDC_ERROR;
55     }
56
57     tdc->spi      = spi;
58     tdc->cs_port   = cs_port;
59     tdc->cs_pin    = cs_pin;
60     tdc->en_port   = en_port;
61     tdc->en_pin    = en_pin;
62
63     return TDC_OK;
64 }
65
66 TDC_error_t TDC_enable(TDC_t* tdc)
67 {
68     if (tdc == NULL) {
69         return TDC_ERROR;
70     }
71
72     HAL_GPIO_WritePin(tdc->en_port, tdc->en_pin, GPIO_PIN_SET);
73
74     return TDC_OK;
75 }
76
77 TDC_error_t TDC_disable(TDC_t* tdc)
78 {
79     if (tdc == NULL) {
80         return TDC_ERROR;
81     }
82
83     HAL_GPIO_WritePin(tdc->en_port, tdc->en_pin, GPIO_PIN_RESET);
84

```

```

85     return TDC_OK;
86 }
87
88 TDC_error_t TDC_write(TDC_t* tdc, TDC_adr_t address, uint8_t* tx_data)
89 {
90     return send(tdc, address, tx_data, true);
91 }
92
93 TDC_error_t TDC_read(TDC_t* tdc, TDC_adr_t address, uint8_t* rx_data)
94 {
95     return send(tdc, address, rx_data, false);
96 }
97
98 TDC_error_t TDC_start(TDC_t* tdc)
99 {
100     uint8_t data[TDC_REG_SIZE[TDC_ADR_CONFIG1]];
101     TDC_error_t error_code = TDC_OK;
102
103     if ((error_code = TDC_read(tdc, TDC_ADR_CONFIG1, data)) != TDC_OK) {
104         return error_code;
105     }
106
107     data[0] |= TDC_CONFIG1_START_MEAS;
108
109     return TDC_write(tdc, TDC_ADR_CONFIG1, data);
110 }
111
112 TDC_error_t TDC_read_result(TDC_t* tdc, uint64_t* tof_fs)
113 {
114     uint8_t time1_buf[TDC_REG_SIZE[TDC_ADR_TIME1]];
115     uint8_t time2_buf[TDC_REG_SIZE[TDC_ADR_TIME2]];
116     uint8_t clock_count1_buf[TDC_REG_SIZE[TDC_ADR_CLOCK_COUNT1]];
117     uint8_t calibration1_buf[TDC_REG_SIZE[TDC_ADR_CALIBRATION1]];
118     uint8_t calibration2_buf[TDC_REG_SIZE[TDC_ADR_CALIBRATION2]];
119     uint8_t config2_buf[TDC_REG_SIZE[TDC_ADR_CONFIG2]];
120
121     uint32_t time1;
122     uint32_t time2;
123     uint32_t clock_count1;
124     uint32_t calibration1;
125     uint32_t calibration2;
126     uint32_t calibration2_periods;
127
128     TDC_error_t error_code = TDC_OK;
129
130     // Read registers
131
132     if ((error_code = TDC_read(tdc, TDC_ADR_TIME1, time1_buf)) != TDC_OK) {
133         return error_code;
134     }
135
136     time1 = TDC_24BIT_BUF_TO_INT(time1_buf);
137
138     if ((error_code = TDC_read(tdc, TDC_ADR_TIME2, time2_buf)) != TDC_OK) {
139         return error_code;
140     }
141
142     time2 = TDC_24BIT_BUF_TO_INT(time2_buf);
143
144     if ((error_code = TDC_read(tdc, TDC_ADR_CLOCK_COUNT1, clock_count1_buf)) !=
TDC_OK) {

```

```
145     return error_code;
146 }
147
148 clock_count1 = TDC_24BIT_BUF_TO_INT(clock_count1_buf);
149
150 if ((error_code = TDC_read(tdc, TDC_ADR_CALIBRATION1, calibration1_buf)) !=
TDC_OK) {
151     return error_code;
152 }
153
154 calibration1 = TDC_24BIT_BUF_TO_INT(calibration1_buf);
155
156 if ((error_code = TDC_read(tdc, TDC_ADR_CALIBRATION2, calibration2_buf)) !=
TDC_OK) {
157     return error_code;
158 }
159
160 calibration2 = TDC_24BIT_BUF_TO_INT(calibration2_buf);
161
162 if ((error_code = TDC_read(tdc, TDC_ADR_CONFIG2, config2_buf)) != TDC_OK) {
163     return error_code;
164 }
165
166 if (config2_buf[0] & TDC_CONFIG2_CALIBRATION2_PERIODS_10) {
167     calibration2_periods = 10;
168 } else if (config2_buf[0] & TDC_CONFIG2_CALIBRATION2_PERIODS_20) {
169     calibration2_periods = 20;
170 } else if (config2_buf[0] & TDC_CONFIG2_CALIBRATION2_PERIODS_40) {
171     calibration2_periods = 40;
172 } else { // TDC_CONFIG2_CALIBRATION2_PERIODS_2
173     calibration2_periods = 2;
174 }
175
176 // Calculations
177
178 double cal_count = (double)(calibration2 - calibration1) /
(calibration2_periods - 1);
179 uint64_t norm_lsb_fs = (uint64_t)(TDC_CLOCK_PERIOD_FS / cal_count);
180
181 *tof_fs = (int64_t)norm_lsb_fs * ((int64_t)time1 - (int64_t)time2) +
((int64_t)clock_count1 * TDC_CLOCK_PERIOD_FS);
182
183 return TDC_OK;
184 }
185
186 /**
187  * @brief Read/write register data from/to TDC
188  *
189  * Function for both reading and writing register data, depending on @p write
parameter.
190  *
191  * @param[in]    tdc      -- Pointer to TDC handle
192  * @param[in]    address  -- Address of the register(s) to read/write
193  * @param[in,out] data    -- Pointer to data
194  * @param[in]    write    -- True = write, false = read
195  *
196  * @attention @p data needs to provide space for at least TDC_REG_SIZE[ @p
address ] bytes.
197  *
198  * @return TDC_error_t -- Error code
199  */
```



```
200 static TDC_error_t send(TDC_t* tdc, TDC_adr_t address, uint8_t* data, bool
    write)
201 {
202     uint8_t size; // number of data bytes
203     uint8_t cmd = address | (write ? TDC_CMD_WRITE : TDC_CMD_READ);
204
205     if ((tdc == NULL) || (data == NULL)) {
206         return TDC_ERROR;
207     }
208
209     if (address >= TDC_ADR_AMOUNT) {
210         return TDC_WRONG_ADDRESS;
211     }
212
213     size = TDC_REG_SIZE[address];
214
215     HAL_GPIO_WritePin(tdc->cs_port, tdc->cs_pin, GPIO_PIN_RESET);
216     HAL_Delay(1); // 1 ms
217
218     if (HAL_SPI_Transmit(tdc->spi, &cmd, 1, HAL_MAX_DELAY) != HAL_OK) {
219         return TDC_COM_ERROR;
220     }
221
222     if (write) {
223         if (HAL_SPI_Transmit(tdc->spi, data, size, HAL_MAX_DELAY) != HAL_OK) {
224             return TDC_COM_ERROR;
225         }
226     } else {
227         if (HAL_SPI_Receive(tdc->spi, data, size, HAL_MAX_DELAY) != HAL_OK) {
228             return TDC_COM_ERROR;
229         }
230     }
231
232     HAL_GPIO_WritePin(tdc->cs_port, tdc->cs_pin, GPIO_PIN_SET);
233     HAL_Delay(1); // 1 ms
234
235     return TDC_OK;
236 }
```

Code 11: TDC Driver (Source)

8.3 Python Scripts

The Python script for calculating the arithmetic mean and the standard deviation, as well as for plotting the histogram and the box plot, is shown in code 12.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Read the data from the log.txt file
5 with open('log.txt', 'r') as file:
6     tof_values = []
7     for line in file:
8         # Extract the ToF value using the pattern 'ToF = <value> [ps]'
9         if 'ToF' in line:
10             tof_value = int(line.split('=')[1].split(' ')[0].strip())
11             tof_values.append(tof_value)
12
13 # Convert the list to a numpy array for easier calculation
14 tof_array = np.array(tof_values)
```

```
15
16 # Calculate the arithmetic mean and standard deviation
17 mean_tof = np.mean(tof_array)
18 std_tof = np.std(tof_array)
19
20 # Print the results
21 print(f'Arithmetic Mean of ToF: {mean_tof}')
22 print(f'Standard Deviation of ToF: {std_tof}')
23
24 # Plot a histogram
25 plt.figure(figsize=(10, 6))
26 plt.hist(tof_array, bins=20, color='skyblue', edgecolor='black', alpha=0.7)
27 plt.title('Histogram of ToF Values')
28 plt.xlabel('Time-of-Flight (ps)')
29 plt.ylabel('Frequency')
30 plt.grid(True)
31 plt.show()
32
33 # Plot a box plot
34 plt.figure(figsize=(10, 6))
35 plt.boxplot(tof_array, vert=False, patch_artist=True,
36             boxprops=dict(facecolor='lightblue', color='black'))
37 plt.title('Box Plot of ToF Values')
38 plt.xlabel('Time-of-Flight (ps)')
39 plt.grid(True)
40 plt.show()
```

Code 12: Python Analysis

The Python script for calculating the arithmetic mean and standard deviation and for plotting the values for several ToF data sets is shown in code 13.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.ticker as mticker
4
5 # Load data from logs.txt, handling empty values
6 filename = 'logs.txt'
7 data = np.genfromtxt(filename, delimiter=';', dtype=float,
8                       filling_values=np.nan)
9
10 # Calculate arithmetic mean and standard deviation for each column, ignoring
11 # NaNs
12 means = np.nanmean(data, axis=0)
13 std_devs = np.nanstd(data, axis=0)
14
15 # Print results
16 for i, (mean, std_dev) in enumerate(zip(means, std_devs), start=1):
17     print(f"Column {i}: Mean = {mean:.2f}, Standard Deviation = {std_dev:.2f}")
18
19 # Plot all columns with points
20 plt.figure(figsize=(10, 6))
21 labels = ["0 m", "1 m", "2 m", "4 m", "6 m", "7 m"]
22 for i in range(data.shape[1]):
23     plt.scatter(range(len(data[:, i])), data[:, i], label=labels[i], s=5)
24
25 plt.title('ToF for different cable lengths')
26 plt.xlabel('Index')
27 plt.ylabel('ToF [ps]')
28 plt.gca().yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _ :
29     f"{x:,.0f}".replace(",", "")))
```

```
27 plt.legend()
28 plt.grid()
29 plt.tight_layout()
30 plt.show()
```

Code 13: Python Analysis (Multi ToFs)

The Python script for plotting the values from several log files is shown in code 14.

```
1 import matplotlib.pyplot as plt
2 import matplotlib.ticker as mticker
3 import numpy as np
4
5 # Function to read ToF values from a log file
6 def read_tof_values(filename):
7     tof_values = []
8     with open(filename, 'r') as file:
9         for line in file:
10             if "ToF =" in line:
11                 try:
12                     value = int(line.split('=')[1].split()[0])
13                     tof_values.append(value)
14                 except (IndexError, ValueError):
15                     print(f"Skipping malformed line in {filename}: {line.strip()}")
16     return tof_values
17
18 # Read ToF values from the three log files
19 log_files = ["log1.txt", "log2.txt", "log3.txt"]
20 datasets = []
21
22 for log_file in log_files:
23     datasets.append(read_tof_values(log_file))
24
25 # Combine all data for plotting and calculate indices
26 data_indices = []
27 offset = 0
28
29 for data in datasets:
30     indices = list(range(offset, offset + len(data)))
31     data_indices.append(indices)
32     offset += len(data)
33
34 # Plot the data
35 plt.figure(figsize=(10, 6))
36 colors = ['blue', 'orange', 'green'] # Colors for the datasets
37 labels = ['19 cm', '24 cm', '29 cm'] # Dataset labels
38
39 for i, (indices, data) in enumerate(zip(data_indices, datasets)):
40     # Scatter plot for the original data points
41     plt.scatter(indices, data, label=f'Distanz = {labels[i]}', color=colors[i],
42                 s=10)
43
44     # Add mean value as a horizontal line within the range of the dataset
45     mean_value = np.mean(data)
46     plt.plot(indices, [mean_value] * len(indices), color=colors[i],
47               linestyle='--', linewidth=2, label=f'Distanz = {labels[i]} (Mean)')
48
49     # Linear fit for the dataset (polynomial of degree 1, i.e., a straight line)
50     slope, intercept = np.polyfit(indices, data, 1) # Fit a line to the data
51     fit_line = np.polyval([slope, intercept], indices) # Calculate the fitted values
```

```
50 plt.plot(indices, fit_line, color=colors[i], linestyle=':', linewidth=2,
51          label=f'Distanz = {labels[i]} (Linear Fit)')
52 # Customize the plot
53 plt.title('ToF unterschiedliche Distanzen zum Spiegel')
54 plt.xlabel('Index')
55 plt.ylabel('ToF [ps]')
56 plt.gca().yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
57     f"{x:,.0f}".replace(",", " ")))
58 plt.legend()
59 plt.grid(True)
60 # Show the plot
61 plt.show()
```

Code 14: Python Analysis (Multi Logs)

The Python script for live plotting of ToF values is shown in code 15. This will be used during the presentation for demonstration purposes.

```
1 import serial
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4 import matplotlib.ticker as mticker
5 from collections import deque
6 import re
7
8 # Configuration
9 COM_PORT = 'COMX'
10 BAUD_RATE = 38400
11 MAX_SAMPLES = 20000
12
13 # Calibration (zero point)
14 TOF_OM_PS = 25800 # ToF at cable length 0m in [ps]
15
16 # Constants
17 TOF_MIN_PS = 0 # ToF min. value (sanity check) [ps]
18 TOF_MAX_PS = 30000 + TOF_OM_PS # ToF max. value (sanity check) [ps]
19 TOF_PS_TO_LEN_CM = 0.9 * 3e8 / 1e12 * 100 # from [ps] to [cm] via c
20
21 def parse_tof(line):
22     """Extract the ToF value from a line of text."""
23     match = re.search(r'ToF\s*=\s*(\d+)\s*\[ps\]', line)
24     if match:
25         return int(match.group(1))
26     return None
27
28 def update(frame):
29     """Update the plot with new data."""
30     global data, ser
31
32     try:
33         if ser.in_waiting > 0:
34             lines = ser.read(ser.in_waiting).decode('utf-8').splitlines()
35             for line_content in lines:
36                 tof_value = parse_tof(line_content)
37                 if tof_value is not None and tof_value > TOF_MIN_PS and
38                 tof_value < TOF_MAX_PS: # Skip invalid values
39                     tof_value -= TOF_OM_PS # Calibration
40                     data.append(tof_value)
```

```
41         # Print latest value
42         tof_str_ps = f"{tof_value:,.0f}".replace(",", "'") #
1000th separator
43         tof_cm      = round(tof_value * TOF_PS_TO_LEN_CM)      #
No decimal places
44         line_text.set_text(f"Latest ToF: {tof_str_ps} ps | {tof_cm}
cm")
45
46         # Update the Line2D object
47         line.set_data(range(len(data)), data)
48         ax.relim()
49         ax.autoscale_view()
50     except Exception as e:
51         print(f"Error reading from serial port: {e}")
52
53 # Initialize serial port
54 try:
55     ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
56     print(f"Connected to {COM_PORT} at {BAUD_RATE} baud.")
57 except Exception as e:
58     print(f"Failed to connect to serial port: {e}")
59     exit()
60
61 # Initialize plot
62 fig, ax = plt.subplots()
63 data = deque(maxlen=MAX_SAMPLES) # Increase buffer to hold up to 2000 samples
64 line, = ax.plot([], [], label="ToF [ps]") # Create a Line2D object
65 ax.set_ylabel("Time of Flight [ps]")
66 ax.set_xlabel("Index")
67 plt.gca().yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f"{x:,.0f}".replace(",", "'")))
68
69 ax_m = ax.twinx() # Create secondary y-axis
70 ax_m.set_ylabel("Cable Length [cm]") # Label for secondary y-axis
71
72 line_text = plt.text(0.5, 0.95, "", transform=plt.gca().transAxes, ha="center")
73
74 def update_secondary_yaxis_limits(*args):
75     """Update the secondary y-axis limits whenever the primary y-axis
changes."""
76     primary_limits = ax.get_ylim()
77     ax_m.set_ylim(primary_limits[0] * TOF_PS_TO_LEN_CM, primary_limits[1] *
TOF_PS_TO_LEN_CM)
78
79 # Connect the limit update to the primary y-axis changes
80 ax.callbacks.connect("ylim_changed", update_secondary_yaxis_limits)
81
82 # Initial call to synchronize the limits
83 update_secondary_yaxis_limits()
84
85 ani = animation.FuncAnimation(fig, update, interval=50)
86
87 # Show plot
88 try:
89     plt.show()
90 except KeyboardInterrupt:
91     print("Exiting...")
92 finally:
93     ser.close()
```

Code 15: Python Live Plot

8.4 Schema

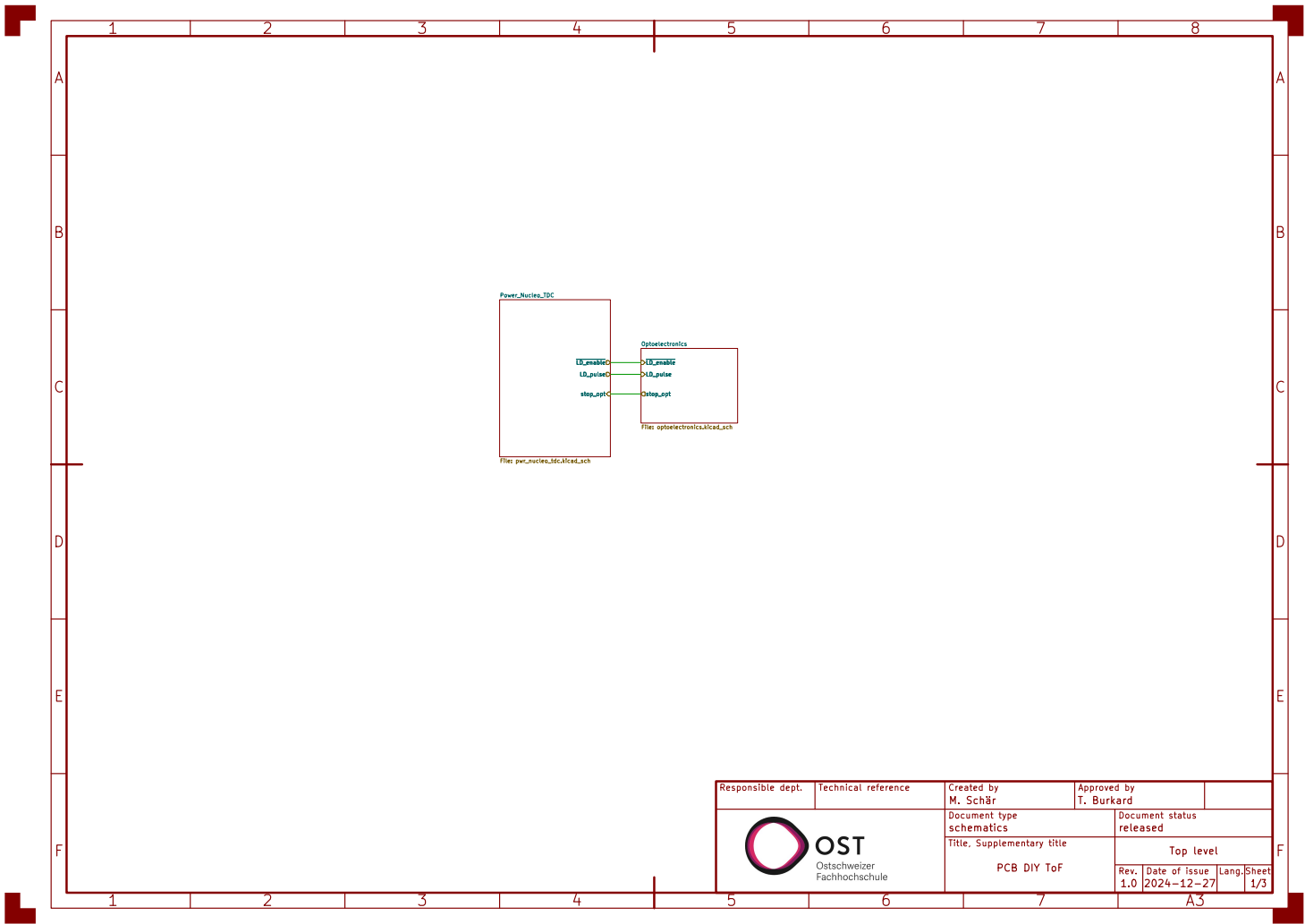


Abbildung 56: Schema S.1/3

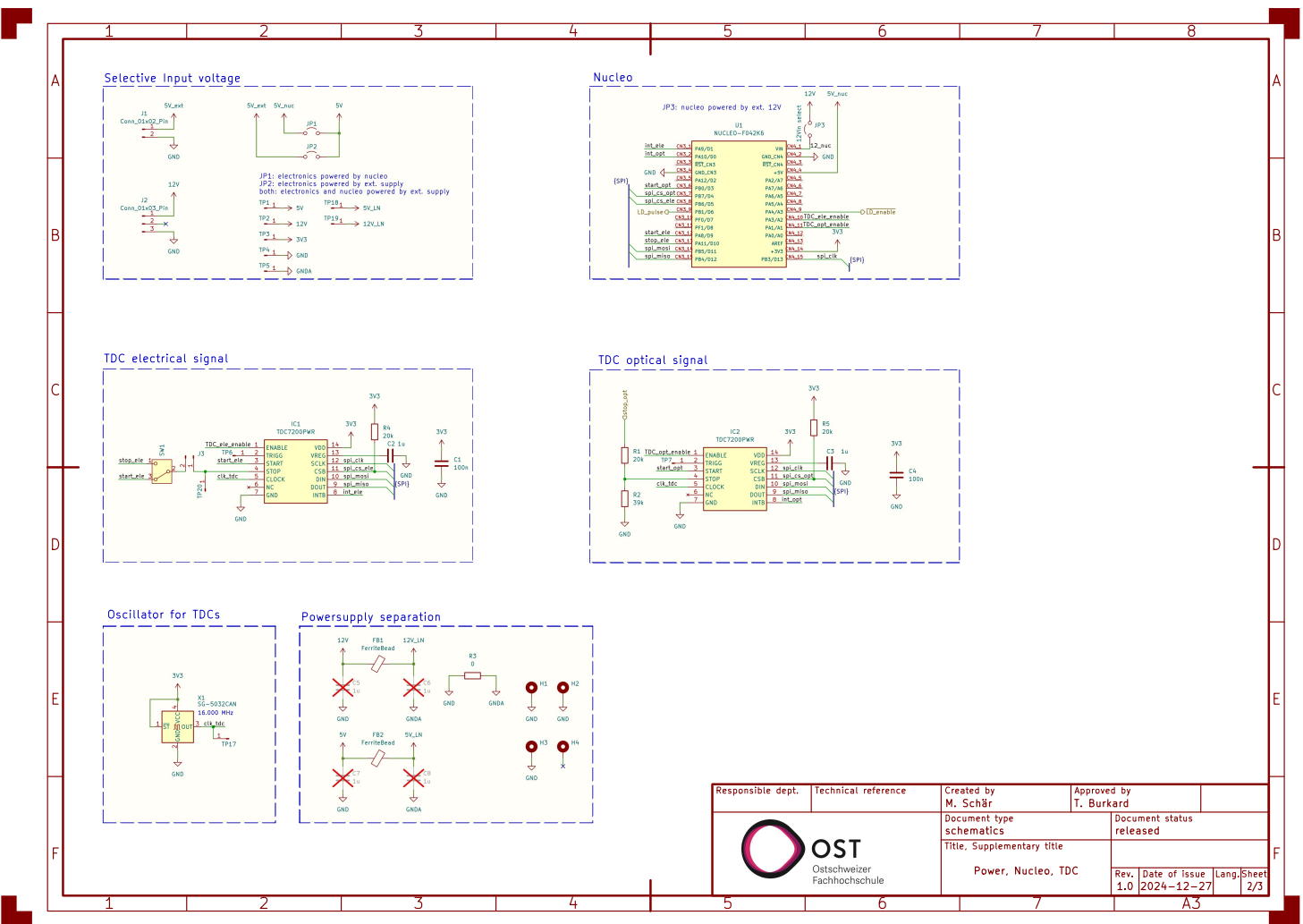


Abbildung 57: Schema S.2/3

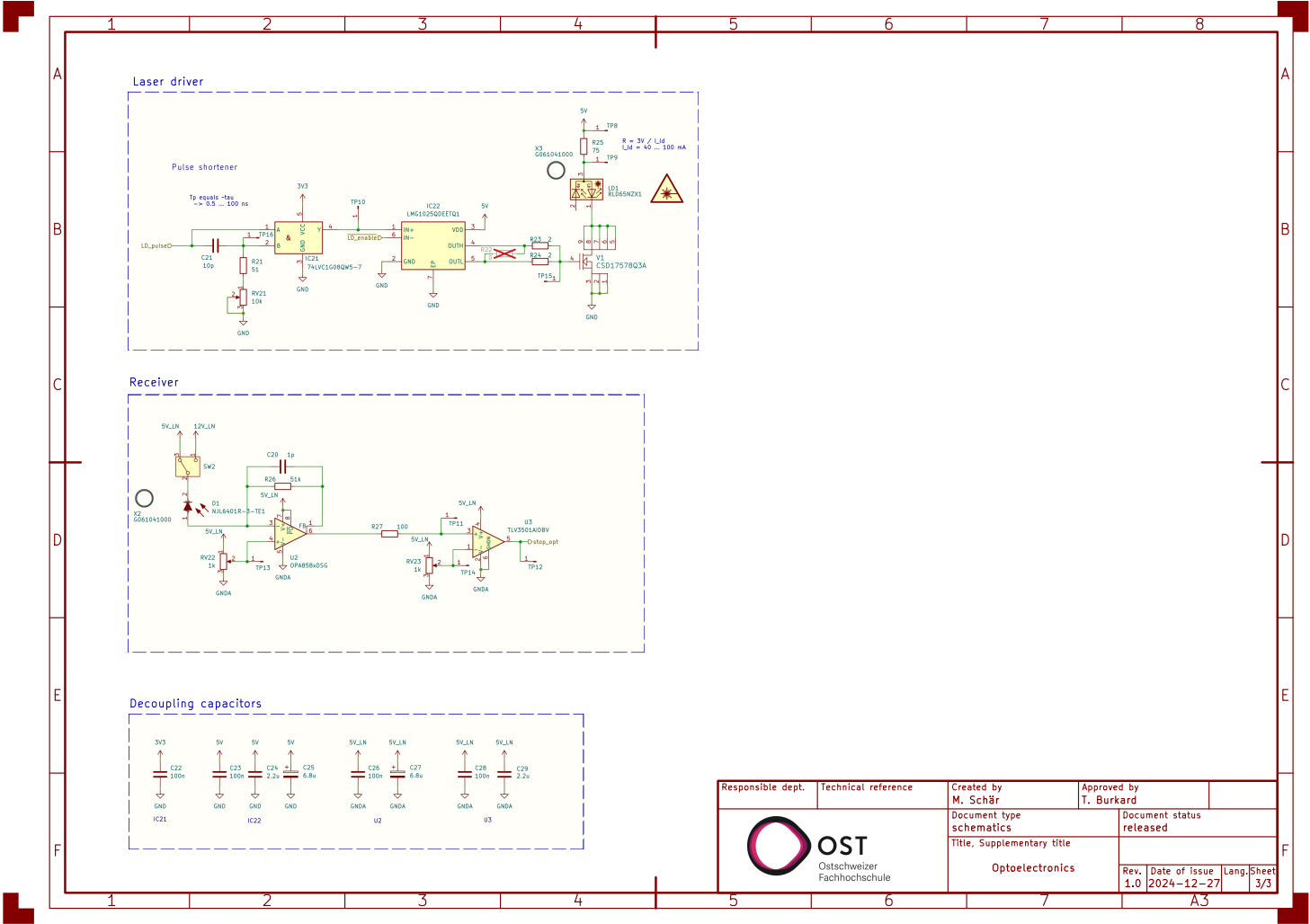


Abbildung 58: Schema S.3/3

8.5 List of Components

Reference	Value	Datasheet	Footprint	Qty	DNP
C1,C4,C22,C23,C26,C28	100n	(JRC, 2014)	Capacitor_SMD:C_0402_1005Metric_Pad0.74x0.62mm	6	DNP
C2,C3	1u		Capacitor_SMD:C_0402_1005Metric_Pad0.74x0.62mm	2	
C5,C6,C7,C8	1u		Capacitor_SMD:C_0402_1005Metric_Pad0.74x0.62mm	4	
C20	1p		Capacitor_SMD:C_0402_1005Metric_Pad0.74x0.62mm	1	
C21	10p		Capacitor_SMD:C_0402_1005Metric_Pad0.74x0.62mm	1	
C24,C29	2.2u		Capacitor_SMD:C_0603_1608Metric_Pad1.08x0.95mm	2	
C25,C27	6.8u		Capacitor_Tantal_SMD:CP_EIA-7343-43_Kemet-X_Pad2.25x2.55mm	2	
D1	NJL6401R-3-TE1		NJL6401R3TE1	1	
FB1,FB2	FerriteBead		Inductor_SMD:L_0402_1005Metric_Pad0.77x0.64mm	2	
IC1,IC2	TDC7200PWR		SOP65P640X120-14N	2	
IC21	74LVC1G08QW5-7		74LVC1G08QW57	1	
IC22	LMG1025QDEETQ1		SON65P200X200X80-7N	1	
J1,J3	Conn_01x02_Pin		Connector:Molex_KK-254_AE-6410-02A_1x02_P2.54mm_Vertical	2	
J2	Conn_01x03_Pin		Connector:Molex_SL_171971-0003_1x03_P2.54mm_Vertical	1	
JP1,JP2	Jumper_2_Open		TestPoint:TestPoint_2Pads_Pitch2.54mm_Drill0.8mm	2	
JP3	12Vin select	(ROHM, 2019)	TestPoint:TestPoint_2Pads_Pitch2.54mm_Drill0.8mm	1	DNP
LD1	RLD65NZX1-00A		RLD65NZX1-00A:RLD85PZJ400A	1	
R1,R4,R5	20k		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	3	
R2	39k		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
R3	0		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
R21	51		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
R22	0		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
R23,R24	2		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	2	
R25	75		Resistor_SMD:R_0603_1608Metric_Pad0.98x0.95mm	1	
R26	51k		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
R27	100		Resistor_SMD:R_0402_1005Metric_Pad0.72x0.64mm	1	
RV21	10k		Potentiometer_SMD:Potentiometer_Bourns.3224W_Vertical	1	
RV22,RV23	1k		Potentiometer_SMD:Potentiometer_Bourns.3224W_Vertical	2	
SW1,SW2	SW_Nidec_CAS-120A1		Button_Switch_SMD:Nidec.Copal_CAS-120A	2	
TP1..TP20	Conn_01x01_Pin	(Nidec Comp., 2024)	Connector_2.54mm:PinHeader_1x01_P2.54mm_Vertical	20	
U1	NUCLEO-F042K6	(ST, 2019)	NUCLEO_F042K6:MODULE_NUCLEO-F042K6	1	
U2	OPA858xDSG	(TI, 2018)	Package_SON:WSO8-8-1EP_2x2mm_P0.5mm_EP0.9x1.6mm	1	
U3	TLV3501AIDBV	(TI, 2016c)	Package_TO_SOT_SMD:SOT-23-6	1	
V1	CSD17578Q3A	(TI, 2016a)	DNH0008A	1	
X1	SG-5032CAN	(Epson Timing, 2024)	Oscillator_Oscillator_SMD_SeikoEpson_SG8002LB-4Pin_5.0x3.2mm	1	
X2,X3	G061041000	(Qioptiq, 2024a)	user:Qioptiq_Mount	2	

Tabelle 12: Bill of Material

8.6 Layout

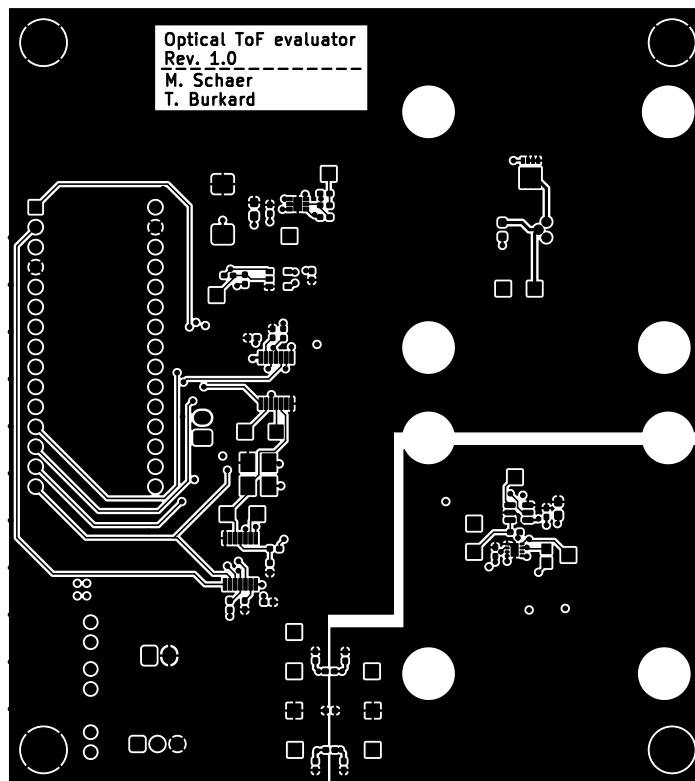


Abbildung 59: PCB Layout Top

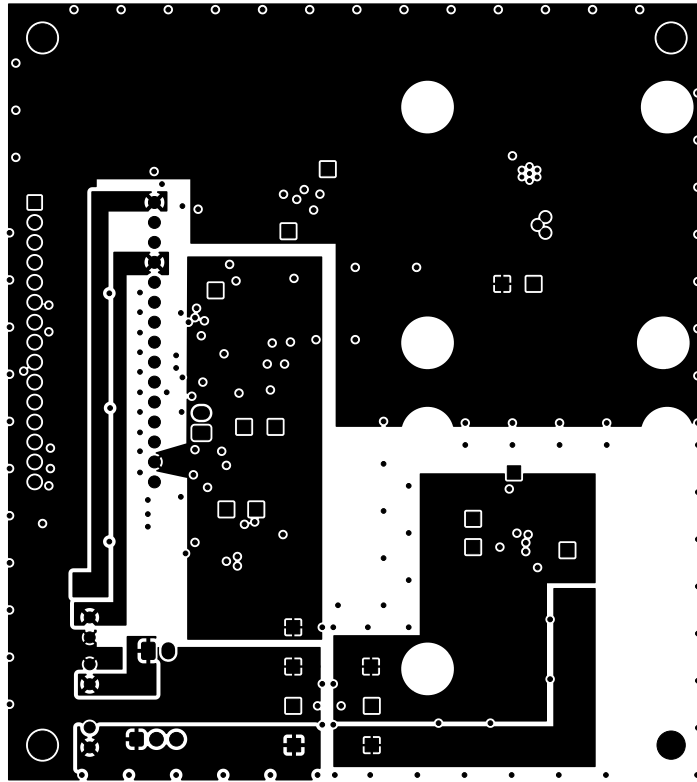


Abbildung 60: PCB Layout Inner 1

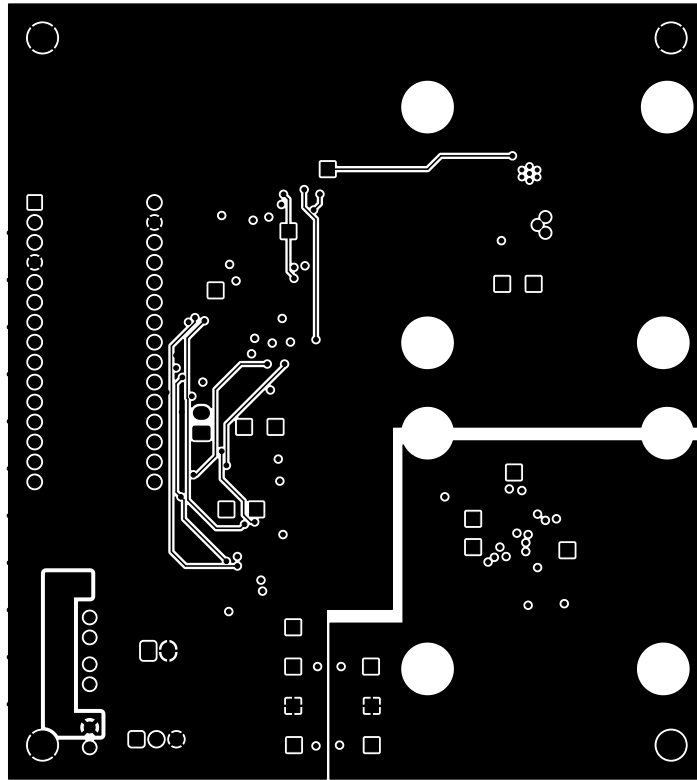


Abbildung 61: PCB Layout Inner 2

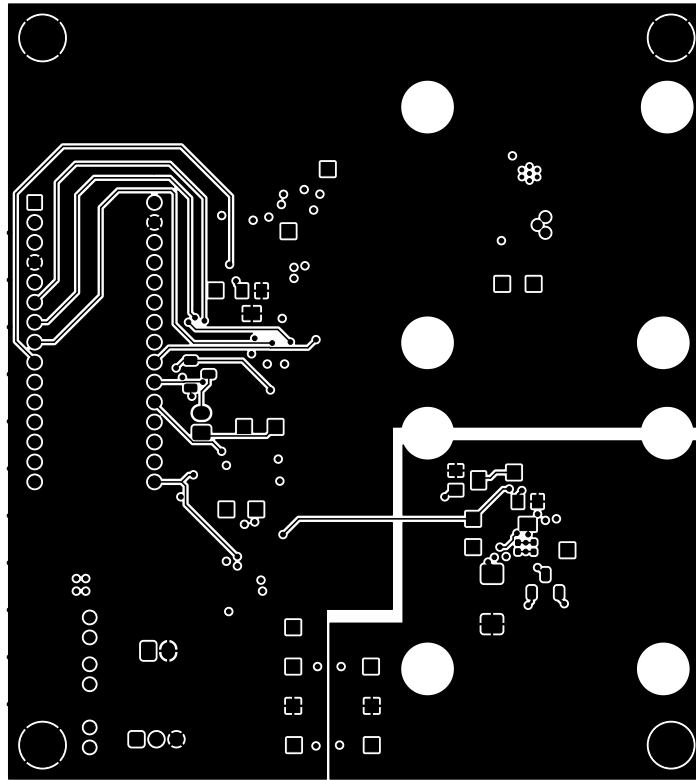


Abbildung 62: PCB Layout Bottom

8.7 Component Placement

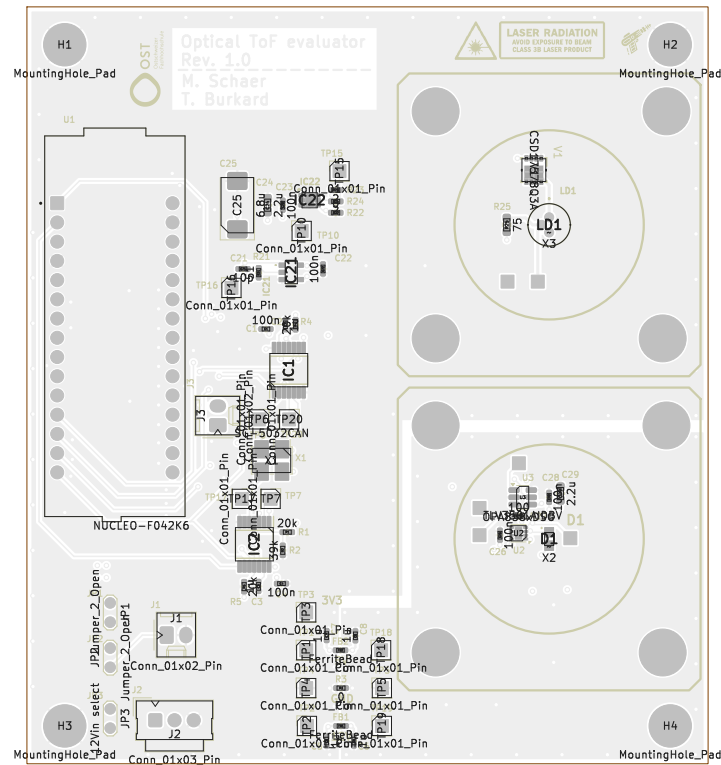


Abbildung 63: PCB component placement top

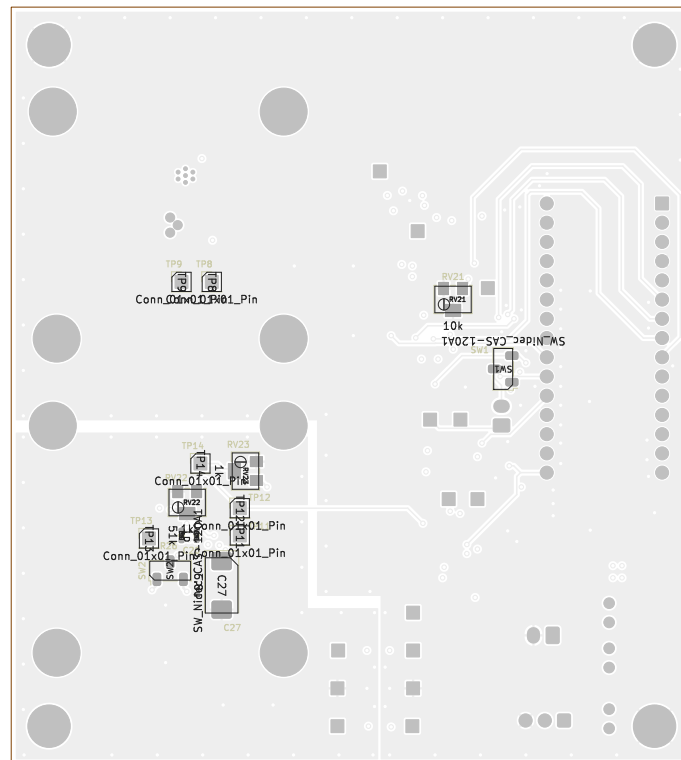


Abbildung 64: PCB component placement bottom

8.8 3D View

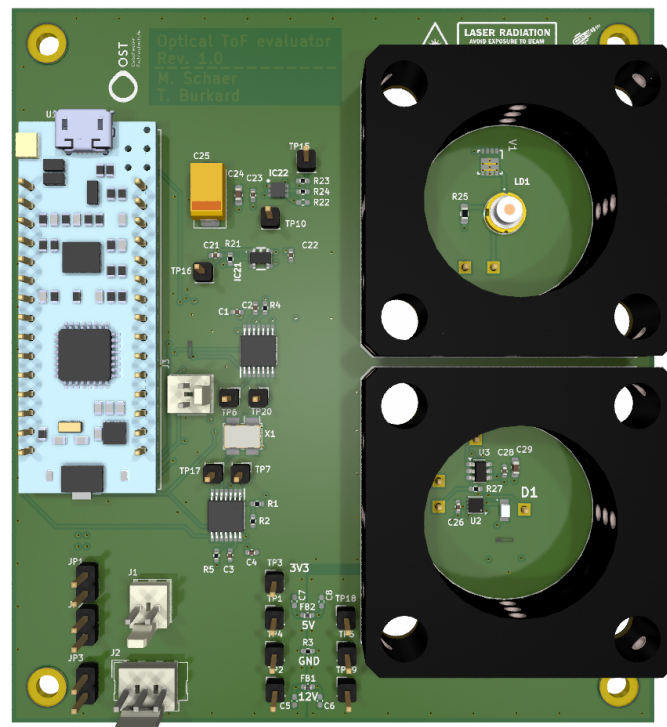


Abbildung 65: 3D View Top

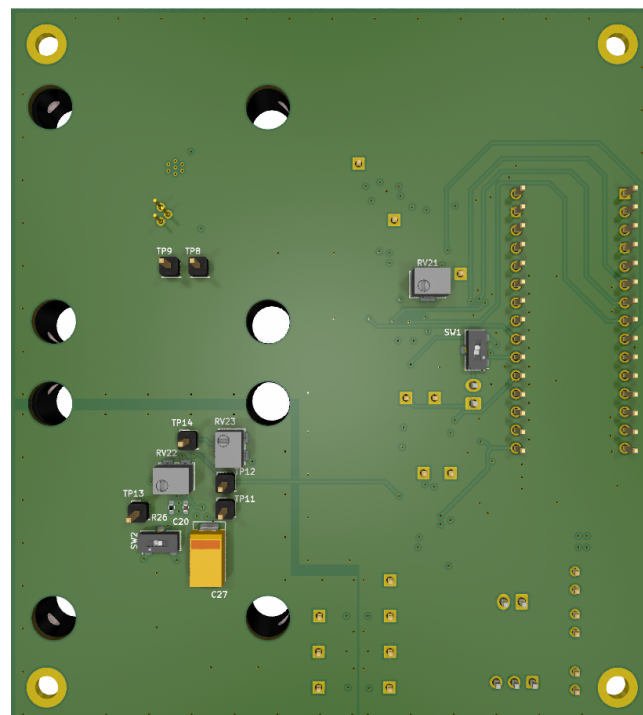


Abbildung 66: 3D View Bottom

8.9 Photos of the demonstrator

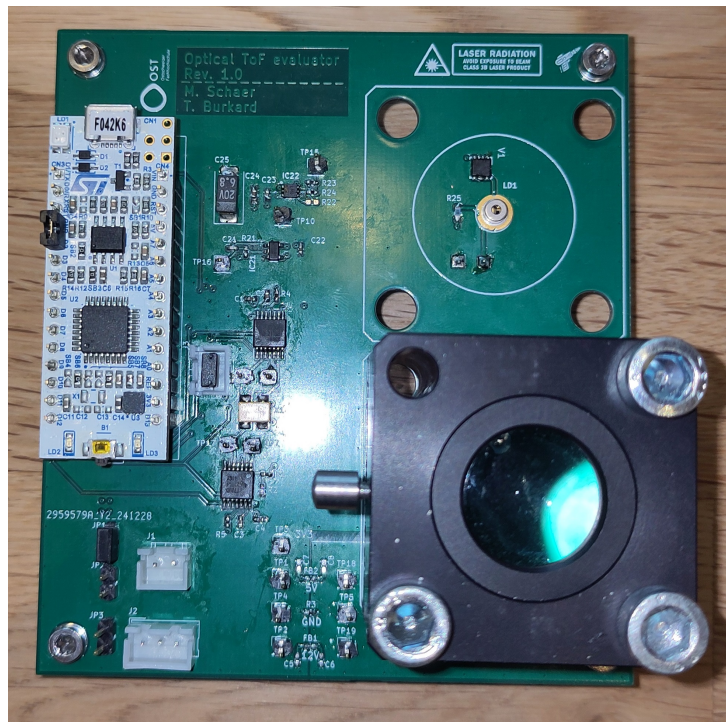


Abbildung 67: Demonstrator from above

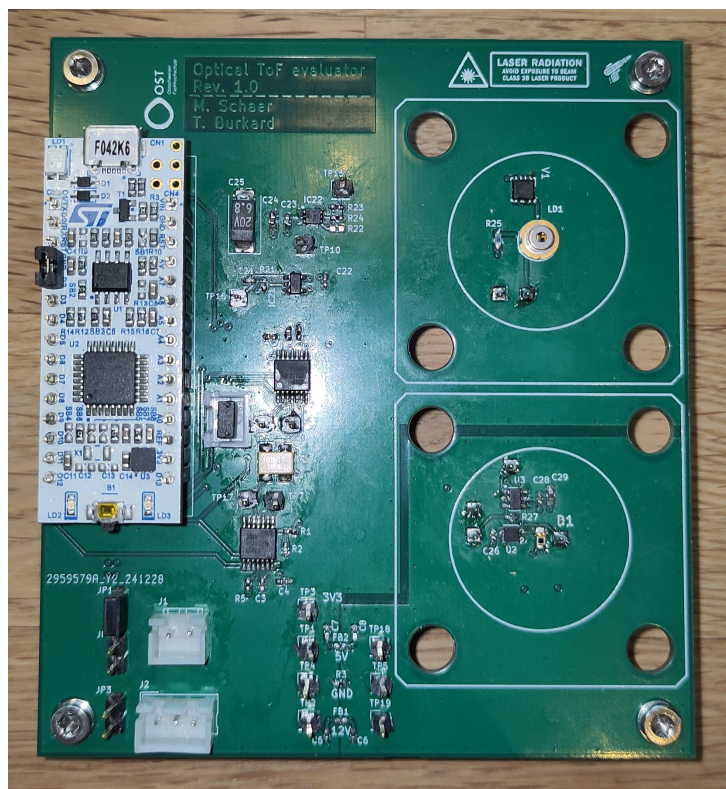


Abbildung 68: Demonstrator from above without lens

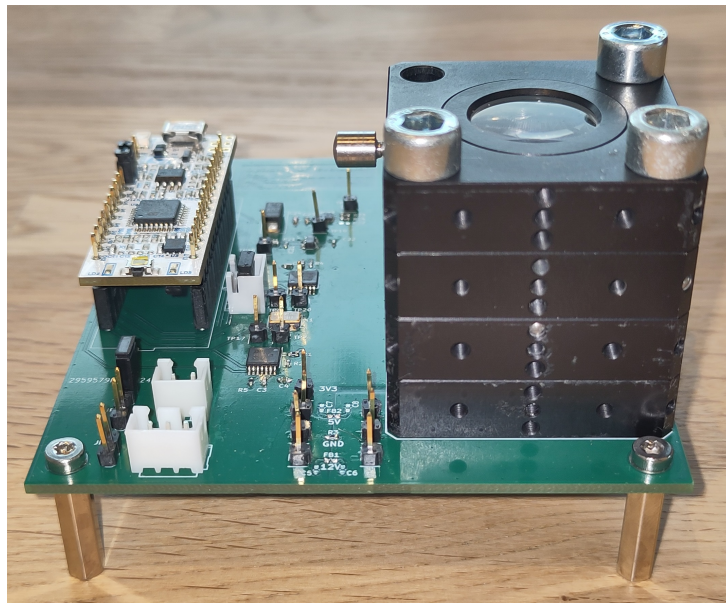


Abbildung 69: Demonstrator from the front

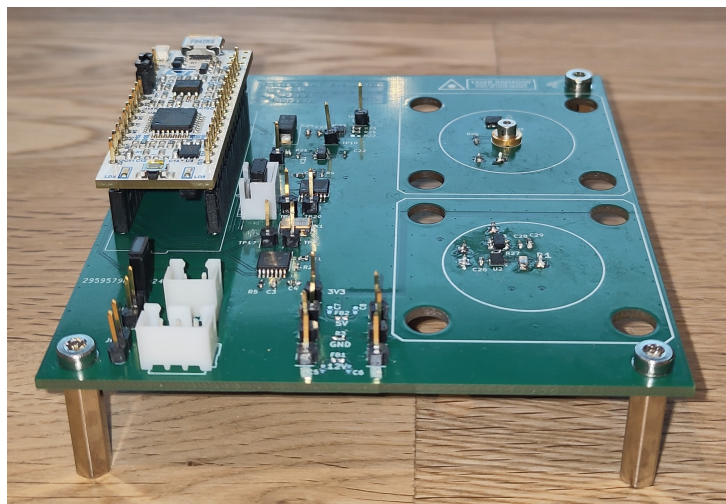


Abbildung 70: Demonstrator from the front without lens

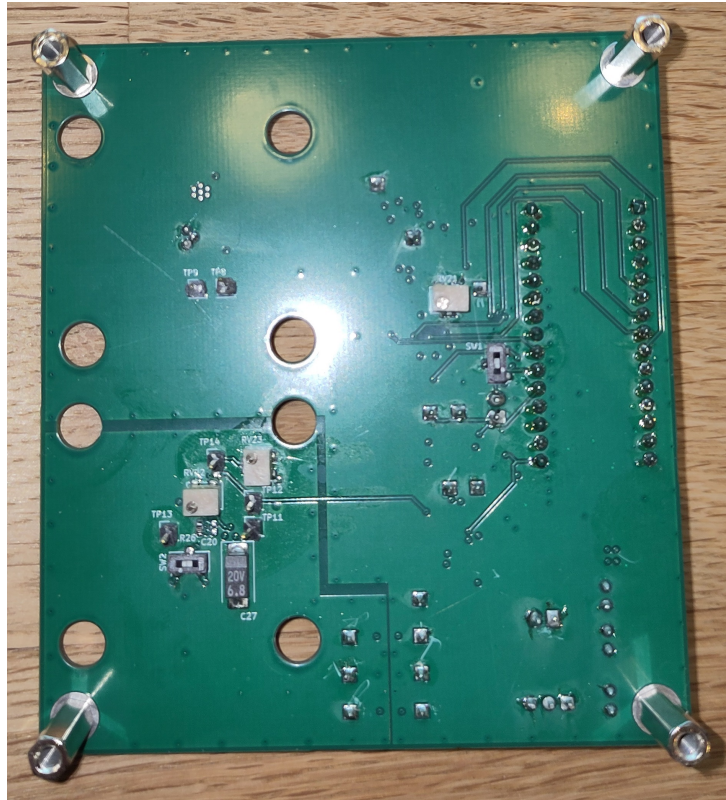


Abbildung 71: Demonstrator from below

List of References

- Bourns. (2024). *3224W-2-10XE Datasheet*. Zugriff auf <https://www.mouser.ch/datasheet/2/54/3224-776900.pdf> (aufgerufen am 26.12.2024)
- Diodes Inc. (2020). *74LVC1G08Q Datasheet*. Zugriff auf https://www.mouser.ch/datasheet/2/115/DI0D_S_A0010762531_1-2543394.pdf (aufgerufen am 26.12.2024)
- Epson Timing. (2024). *SG5032CAN Datasheet*. Zugriff auf https://www.mouser.ch/datasheet/2/137/SG5032CAN_en-961596.pdf (aufgerufen am 26.12.2024)
- firewall.xc. (2025). *Propagation Delay*. Zugriff auf <https://www.firewall.cx/networking/ethernet/propagation-delay.html> (aufgerufen am 12.01.2025)
- JLC. (2025). *JLPCPB*. Zugriff auf <https://jlcpcb.com/> (aufgerufen am 18.01.2025)
- JRC. (2014). *NJL6401R-3 Datasheet*. Zugriff auf https://www.mouser.ch/datasheet/2/294/NJL6401R_3_E-1019028.pdf (aufgerufen am 26.12.2024)
- KiCad. (2025). *KiCad EDA*. Zugriff auf <https://www.kicad.org/> (aufgerufen am 16.01.2025)
- Nidec Comp. (2024). *CAS-120A1 Datasheet*. Zugriff auf <https://www.mouser.ch/datasheet/2/972/cas-1628136.pdf> (aufgerufen am 26.12.2024)
- Qioptiq. (2024a). *G061041000 Datasheet*. Zugriff auf <https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/692/G061041000.pdf> (aufgerufen am 26.12.2024)
- Qioptiq. (2024b). *G061041000 Shop-Seite*. Zugriff auf <https://www.qioptiq-shop.com/en/Optomechanics/LINOS-Microbench-30-mm-Cage-System/Mounting-Plates/Mounting-Plate-25-with-Mounting-Holes.html> (aufgerufen am 26.12.2024)
- ROHM. (2019). *RLD65NZX1 Datasheet*. Zugriff auf https://fscdn.rohm.com/en/products/databook/datasheet/opto/laser_diode/red/rld65nzx100a008-e.pdf (aufgerufen am 26.12.2024)
- ROHM. (2020). *RLD94PZJ5 Datasheet*. Zugriff auf https://fscdn.rohm.com/en/products/databook/datasheet/opto/laser_diode/infrared/rld94pzj5-e.pdf (aufgerufen am 26.12.2024)
- Siemens. (2025). *Xpedition AMS*. Zugriff auf <https://resources.sw.siemens.com/en-US/fact-sheet-xpedition-ams/> (aufgerufen am 16.01.2025)
- ST. (2017). *STM32F042K6 Datasheet*. Zugriff auf <https://www.st.com/resource/en/datasheet/stm32f042k6.pdf> (aufgerufen am 13.01.2025)
- ST. (2019). *NUCLEO-F042K6 Datasheet*. Zugriff auf <https://www.mouser.ch/datasheet/2/389/nucleo-f031k6-1484037.pdf> (aufgerufen am 26.12.2024)
- ST. (2020). *Description of STM32F0 HAL*. Zugriff auf https://www.st.com/resource/en/user_manual/dm00122015-description-of-stm32f0-hal-and-low-layer-drivers-stmicroelectronics.pdf (aufgerufen am 12.01.2025)

- ST. (2024). *NUCLEO-F042K6 Usermanual*. Zugriff auf https://www.st.com/resource/en/user_manual/um1956-stm32-nucleo32-boards-mb1180-stmicroelectronics.pdf (aufgerufen am 26.12.2024)
- ST. (2025). *STM32CubeIDE*. Zugriff auf <https://www.st.com/en/development-tools/stm32cubeide.html> (aufgerufen am 18.01.2025)
- TDK. (2019). *MPZ0402S100 Datasheet*. Zugriff auf https://product.tdk.com/system/files/dam/doc/product/emc/emc/beads/catalog/beads_commercial_power_mpz0402_en.pdf (aufgerufen am 26.12.2024)
- TI. (2016a). *CSD17578Q3A Datasheet*. Zugriff auf <https://www.ti.com/lit/ds/symlink/csd17578q3a.pdf?ts=1735200469410> (aufgerufen am 26.12.2024)
- TI. (2016b). *TDC7200 Datasheet*. Zugriff auf <http://www.ti.com/lit/gpn/tdc7200> (aufgerufen am 25.12.2024)
- TI. (2016c). *TLV3501 Datasheet*. Zugriff auf <https://www.ti.com/lit/ds/symlink/tlv3501.pdf?ts=1735168630258> (aufgerufen am 26.12.2024)
- TI. (2018). *OPA858 Datasheet*. Zugriff auf <https://www.ti.com/lit/ds/symlink/opa858.pdf?ts=1735178891996> (aufgerufen am 26.12.2024)
- TI. (2024a). *CSD17578Q3A*. Zugriff auf <https://www.ti.com/product/CSD17578Q3A> (aufgerufen am 28.12.2024)
- TI. (2024b). *LMG1025-Q1*. Zugriff auf <https://www.ti.com/product/LMG1025-Q1> (aufgerufen am 28.12.2024)
- TI. (2024c). *LMG1025-Q1 Datasheet*. Zugriff auf <https://www.ti.com/lit/ds/symlink/lmg1025-q1.pdf?ts=1735200095732> (aufgerufen am 26.12.2024)
- TI. (2024d). *OPA858*. Zugriff auf <https://www.ti.com/product/OPA858> (aufgerufen am 28.12.2024)
- Vishay Semic. (2024). *BPV23NF Datasheet*. Zugriff auf <https://www.vishay.com/docs/81513/bpv23nf.pdf> (aufgerufen am 26.12.2024)